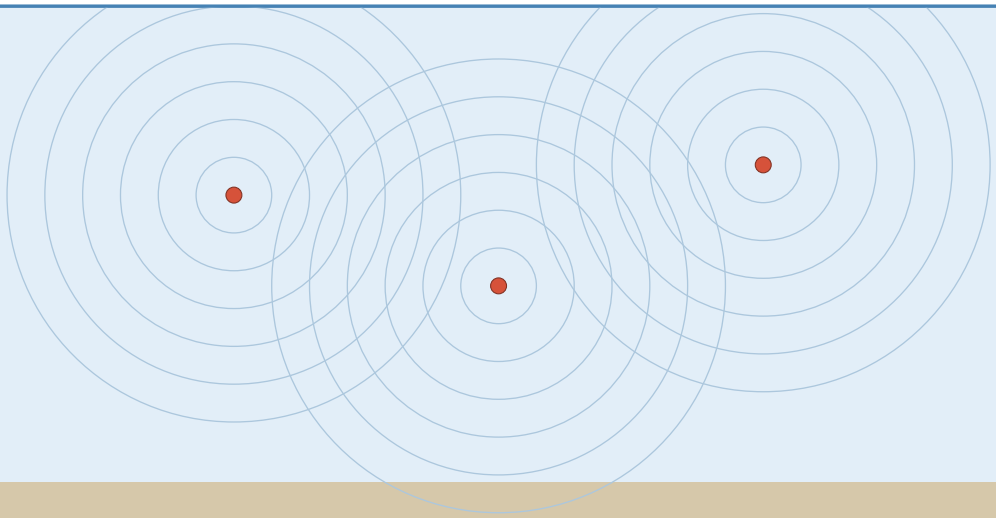


off-coustics

Underwater noise of offshore wind turbines, wind farms and point sources

User and developer guide of the Fortran solver

Raúl Sanz Ramírez

ETSIAE, Universidad Politécnica de Madrid
ERC project off-coustics

October 2026. This guide describes the code of `FORTRAN/src` and `FORTRAN/app` and the plotting package of `FORTRAN/plotting`. Every formula refers to the routine that implements it, and every number quoted from a run comes from `docs/figures/make_figures.py`.

Contents

List of Figures v

List of Tables viii

How to read this guide x

I User guide 1

1 What the solver computes 2
1.1 The problem . . . 2
1.2 Frame, units and conventions . . . 2
1.3 The field of a point dipole . . . 3
1.4 What is not modelled . . . 4

2 Installing and running 5
2.1 Dependencies . . . 5
2.2 Building . . . 5
2.3 Running . . . 6
2.4 What a run prints . . . 6

3 Input files 8
3.1 The input directory . . . 8
3.2 Solver.nml . . . 9
3.3 Environment.nml . . . 10
3.4 Emitters . . . 10
3.5 Computation setups . . . 16
3.6 Acoustic methods . . . 19
3.7 Worked examples . . . 20

4 Emitters 24
4.1 Wind turbine . . . 24
4.2 Wind farm . . . 26
4.3 FrequencySource . . . 27
4.4 TimeSource . . . 27

5 Acoustic methods 28
5.1 Image method . . . 28
5.2 Analytical normal modes . . . 30
5.3 Choosing a method . . . 31

6 Post-processing steps **32**
6.1 Reference points . . . 32
6.2 Masked observers . . . 32
6.3 The steps . . . 33
6.4 Metrics and their biases . . . 35

7 Outputs **37**
7.1 File names . . . 37
7.2 The HDF5 file . . . 37
7.3 The `.dat` summary . . . 38
7.4 Updating an existing output . . . 40

8 The plotting package **42**
8.1 Command line . . . 42
8.2 The case . . . 42
8.3 The plots . . . 43

9 Validation **47**
9.1 Free-field dipole . . . 47
9.2 Image method against normal modes . . . 48
9.3 The FFT of the time-domain sources . . . 48
9.4 Input modes and parser . . . 48
9.5 What is not verified here . . . 49

10 Troubleshooting **50**
10.1 The run stops at once . . . 50
10.2 A namelist value is silently ignored . . . 50
10.3 Sources and observers . . . 50
10.4 Turbines and farms . . . 51
10.5 Manual sources . . . 51
10.6 Builds and binaries . . . 52
10.7 Outputs . . . 52

II Developer guide **53**

11 Architecture **54**
11.1 Layers . . . 54
11.2 Class hierarchies . . . 54
11.3 Program flow . . . 56

12 Data model **58**
12.1 Kinds and constants . . . 58
12.2 The configuration . . . 59
12.3 The sources . . . 59
12.4 Run-time state of a solver . . . 59

13 Emitters in depth **61**
13.1 The emitter contract . . . 61
13.2 The turbine, step by step . . . 61
13.3 The FFT . . . 62
13.4 The farm . . . 63
13.5 Manual sources . . . 63

13.6 The expression parser . . . 63

14 Acoustic methods in depth **65**
14.1 The solver contract . . . 65
14.2 Image method . . . 65
14.3 Analytical normal modes . . . 66
14.4 The water column . . . 67

15 The computation engine **68**
15.1 The step sequence . . . 68
15.2 Anatomy of a step . . . 68
15.3 Masked observers . . . 69
15.4 Blocks and threads . . . 69
15.5 A whole-step override: the sphere . . . 70

16 The output layer **71**
16.1 Sequence of a run . . . 71
16.2 HDF5Utils . . . 71
16.3 The `.dat` file . . . 72
16.4 Updating a file . . . 72
16.5 Rules for new outputs . . . 73

17 Extending the code **74**
17.1 A turbine type . . . 74
17.2 An emitter . . . 74
17.3 An acoustic method . . . 75
17.4 A post-processing step . . . 76

18 Verification and style **77**
18.1 The verification protocol . . . 77
18.2 Style rules . . . 78
18.3 Pitfalls met in this code . . . 78

19 Roadmap **80**
19.1 Manual sources . . . 80
19.2 Methods with a non-uniform environment . . . 80
19.3 The sphere . . . 81
19.4 Items found while writing this guide . . . 81

A Symbols **82**

B Glossary **84**

C Command line and error messages **86**
C.1 Command line . . . 86
C.2 How a run stops . . . 86
C.3 Messages . . . 86

D HDF5 datasets **89**

References **92**

Index of the input variables **93**

Index of the input variables **93**

List of Figures

1.1 Frame and waveguide. The water occupies $-H \le z \le 0$; the free surface is $z = 0$ and the seabed $z = -H$. A dipole $\boldsymbol{F}$ at $\boldsymbol{x}_s$ radiates to an observer at $\boldsymbol{x}$ at the distance $R = |\boldsymbol{x} - \boldsymbol{x}_s|$ along $\hat{\boldsymbol{r}}$. The image method adds the fields of the images of the source across both planes (open circles); the first image across the surface is at $z = -z_s$ and the first across the seabed at $z = -2H - z_s$ (Section 5.1). Azimuths are measured anticlockwise from $+x$ (top view, right). 3

3.1 The input directory. `simulation_type` chooses the emitter file and its computation setup (the one with the suffix `_CS`); `method` chooses the file of the acoustic method. Only the files of the chosen emitter and method are read. The arrows give the reading order. 8

3.2 SPL of a vertical dipole of 1000 N at 100 Hz, at $z = -10$ m in 30 m of water (image method, `N_images` = 30, bottom attenuation 0.5), on the plane $y = 0$ (`sliceXZ`, 301 × 91 points; vertical scale exaggerated). Look at the null on the horizontal through the source, where $\boldsymbol{F} \cdot \hat{\boldsymbol{r}} = 0$ for the direct path, and at the interference lobes made by the images. 20

3.3 SPL on the horizontal plane through two F_x dipoles at $(0, \mp 1.25, -15)$ m, 150 Hz (image method). (a) In phase: symmetric in y. (b) Source 2 (at $+y$) lagging 90°: the field is stronger towards $+y$. The vertical line $x = 0$ is the null of an F_x dipole (drawn at the lowest level of the colour scale). 21

3.4 A `TimeSource` burst $500\,\mathrm{e}^{-((t-1)/0.05)^2}\cos(2\pi\,80\,t)$ N: (a) the series, (b) the amplitude of its spectrum after `compute_rfft` (Hann window, $\Delta f = 0.5$ Hz), (c) the SPL at the three default observers (10, 250 and 500 m at mid-depth). 22

3.5 (a) The force spectrum given as files and the one recovered from `Source_Force` of the output, $|F_k|^2/(2\Delta f)$ (equal to 2×10^{-16}). (b) The PSD of the pressure at the three default observers. 23

4.1 From an OpenFAST run to the dipoles of a turbine. The structural data and the optional frequency filter belong to the turbine type; everything else is shared by every type (`emitters/WindTurbine.f90`). 25

4.2 Pressure spectrum of the DTU 10 MW monopile at 11.4 m/s, 250 m from the axis at mid-depth, with every FFT bin ($\Delta f = 0.1$ Hz, 10 000 bins) and with the bins kept by `filter_freqs` (880). Look at how the kept bands coincide with the drivetrain peaks above 10 Hz, while every bin below `freqs_over` = 10 Hz is kept. Last 10 % of the OpenFAST run (`From` = 0.9), 25 wet nodes. 26

5.1 The images of a source at depth z_s for `N_images` = 2: two chains, one starting with a reflection across the seabed and one across the surface, alternate between the two planes. Each image carries the sign of the boundary conditions and the attenuations of the planes it was reflected across, and its vertical force is reversed at every reflection. The total is $1 + 2N$ dipoles per source. 29

5.2 Convergence of the image method to the normal modes in the ideal 30 m waveguide (surface $p = 0$, rigid seabed, both attenuations 1): relative difference of the pressure against `N_images` for a dipole $\boldsymbol{F} = (1, 0, 1)$ N at $z = -12$ m, observers at $z = -20$ m and three ranges. Look at the $1/N$ slope and at how the convergence starts later at long range. 29
5.3 Analytical normal modes. (a) The first four modal shapes (5.2) in 30 m of water, $\rho = 1025$ kg/m^3: zero at the surface, zero slope at the seabed. (b) Number of propagating modes against frequency for four depths: the automatic `Nmodes` is this number at the highest frequency plus 5. 30
5.4 Level against range for a horizontal dipole of 1000 N at 100 Hz in 30 m of water. Normal modes and the image method with ideal boundaries and 2000 images agree; the shipped image method (30 images, bottom attenuation 0.5) loses the energy of the bottom reflections and decays like a lossy guide; the free field decays as $1/r$ with no interference. Look at the modal interference pattern beyond 100 m. 31

6.1 Geometry of the steps, in top view (circles, lines and horizontal planes) or side view (water column between the surface, blue, and the seabed, brown). Blue dots: observers; red: the sources or their reference point; red cross: the centre of the circle or plane. Every step can be moved and resized with its group of the computation setup; the defaults are those of Section 3.5. 34

7.1 Content of an output file, grouped by the routine that writes it. Only the datasets of the emitter and of the steps that ran are present; every step also writes `Freqs`. 38
7.2 What happens to an existing output file at the start of a run (`resolve_save_path`, `check_update`) and at its end (`update_parameters`). 41

8.1 `turbine.py plot_mn_IM30_ws11.4 --save --no-show --format pdf`: the DTU 10 MW monopile at 11.4 m/s with the drivetrain filter, image method (input in `docs/figures/inputs/gallery_turbine/`). Look at the level of the drivetrain bands in (a) and at the difference between the free-field and in-situ source levels in (d), the effect of the waveguide below the first cut-on frequency. . 44
8.2 Top: `farm.py Farm_2_DTU10MN_IM30_ws8.0-11.4 --save --no-show --format pdf`, the farm of `input_files/Emitters/WindFarm.nml` (input in `docs/figures/inputs/gallery_farm/`). Bottom: `source.py Source_Ns1_Nf3_IM30_R0.5 --save --no-show --format pdf`, the `FrequencySource` case of `input_files/`. Look at the interference between the two coherent turbines in (b). 45
8.3 A comparison, `turbine.py plot_mn_IM30_ws11.4.hdf5 plot_mn_ANM24_ws11.4.hdf5 --save`: the same turbine with the image method (30 images, bottom attenuation 0.5) and the normal modes (24 modes), SPL in 1/3 octave bands. Look at how the lossy seabed of the image method lowers the level at the far observers. 46

9.1 Free-field dipole: (a) magnitude along $+x$, solver (markers) and closed form (1.4) (lines), showing the change of slope from $1/R^2$ to $1/R$; (b) normalised magnitude on a circle of 50 m against $|\cos\theta|$. Input: `docs/figures/inputs/freefield/`. . 48
9.2 (a) A tone through the Hann window of `compute_rfft`: on a bin (blue) and half a bin off (red, hatched). (b) SPL at the three default observers of the `TimeSource` (lines) and of the `FrequencySource` fed with its spectrum (markers): identical. Inputs: `docs/figures/inputs/hann_*/`. 49

11.1 Layers and modules. Arrows point from a layer to the layers it uses; within a layer the order is that of the Makefile. `AcousticSolver_output` and `AcousticSolver_steps` are submodules of `AcousticSolver`. 54
11.2 The two class hierarchies. Dashed boxes are `ABSTRACT` types; the bindings in italics are `DEFERRED`. A child also overrides some non-deferred bindings (e.g. `init` of the turbine types, `run_sphere` of `ImageMethod_t`). 55
11.3 Flow of `main.f90`. The four branches build the emitter; from `to_source` on, everything is common. The solver takes the arrays of the sources with `MOVE_ALLOC` and frees the configuration in its `init`; `check_sources_in_water` runs there, once the seabed of the method is known. 57

12.1 Composition of `Config_t` (`types/ConfigTypes.f90`) and `AcousticSource_t` (`types/SourceTypes.f90`). `Config_t % method` is polymorphic: the parser allocates the extension of the chosen method. Only the field of the chosen emitter is filled. 58

13.1 Compilation of `'100/(1+(f/50)**2)'`: the parse tree (left), the postfix list that `compile_expression` emits (centre, the codes of `OP_NUMBER`, `OP_VAR` and the operators) and the stack during the evaluation (right), whose largest depth, 4, is `expr % depth`. Evaluated at $f = 0, 50, 100$ Hz the expression gives 100, 50 and 20 (checked with the compiled parser). 64

List of Tables

2.1 Software used to build and run the solver and the plotting package. The versions are those of the machine where this guide was produced; other recent versions are expected to work. . . . 5

3.1 Variables of `&solver_config` . . . 9
3.2 Variables of `&environment_config` . . . 10
3.3 Variables of `&turbine_config` . . . 10
3.4 Variables of `&farm_config` . . . 12
3.5 Variables of every `&turbine` group of `Emitters/WindFarm.nml` . . . 12
3.6 Input modes of `FrequencySource`. The limits of the modes written by hand are the sizes of the parse buffers `MAX_PARSE_SRC` = 20 and `MAX_PARSE_FREQ` = 200 (`Kinds.f90`); what is read from files has no limit. . . . 13
3.7 Variables of `&frequency_source_config` . . . 13
3.8 Variables of `&time_source_config` . . . 14
3.9 Variables of `&do_config` . . . 16
3.10 Variables of `&spectrums_config` . . . 16
3.11 Variables of `&polar_config` . . . 17
3.12 Variables of `&cylinder_config` . . . 17
3.13 Variables of `&decay_config` . . . 17
3.14 Variables of `&line_config` . . . 17
3.15 Variables of `&sliceXY_config` . . . 18
3.16 Variables of `&sliceXZ_config` . . . 18
3.17 Variables of `&sliceVertical_config` . . . 18
3.18 Variables of `&sphere_config` . . . 19
3.19 Variables of `&ImagesMethod_config` . . . 19
3.20 Variables of `&AnalyticalNormalModes_config` . . . 20

4.1 Turbine types (`case_type`) and the data each one holds (`emitters/TurbineTypes/`). 24

5.1 When to use each method. . . . 31

6.1 Reference points of each emitter, used by the automatic values of the steps. . . . 32

7.1 Parts of the default output name. . . . 37

8.1 Options of the entry scripts (`plotting/plot_utils/cli.py`; `--ws` only in `turbine.py`). . . . 43
8.2 Plots of a single case (`plotting/plots/`). Every one takes `mode` (`'OASPL'`, `'SPL'`, `'PSD'`, `'ABS'`, `'REAL'`, `'IMAG'`, `'PHASE'` at `target_frequency`) and `absorption`, and the frequency filters `filter_under`, `filter_over`; the comparison versions (`plotting/compare/plots.py`) draw one panel per case. . . . 46

9.1 Summary of the checks and the agreement reached. 47

12.1 `AcousticSource_t` and its `GeometricData_t`. 59

15.1 Bits of the pressure against the run with `block_size` = 128 and one thread (shipped `FrequencySource` case with its steps; the clamp makes the blocks of the 4-thread runs differ from the 1-thread ones). 70

A.1 Symbols used in this guide, with their unit and the input or code name when there is one. 82

C.1 Error messages of common wrong inputs, as printed (`<...>`: the values of the case). 86

D.1 Datasets of an output file, by writer. 89

How to read this guide

The guide has two parts. **Part I** is for the people who run the solver: what it computes, how to build it, every input variable, what each emitter, acoustic method and post-processing step does, what the outputs contain, how to plot them, how the results were validated and what the usual error messages mean. **Part II** is for the people who change the code: its layers and class hierarchies, its data model, the emitters and methods in depth, the post-processing engine, the output layer, checklists to extend it, the verification protocol and the roadmap. The appendices collect the symbols, a glossary, the command line and the error messages, and the reference of every HDF5 dataset. The index at the end lists every input variable.

Conventions of the text.

- Names of the code, of files and of datasets are written in `typewriter`; input variables of the namelists are also entered in the index, e.g. `depth_base`.
- The place where a formula or a behaviour is implemented is given as file.f90:routine, with the path relative to `FORTRAN/src` (or `FORTRAN/app` for `main.f90`). Paths of the plotting package are relative to `FORTRAN/plotting`.
- Shapes of arrays are given in Fortran order: `Force(Nfreqs, Nsources, 3)`. The HDF5 files store them in that order, so the Python package `h5py` sees the axes reversed (Section 7.2).
- Boxes mark notes and pitfalls:

> **Note** A note adds context that is useful but not required.

> **Pitfall** A pitfall describes a mistake that has happened in practice and how to avoid it.

How the content was checked. Every statement was checked against the code; where a comment of the code and the code disagree, the code is described and the disagreement is listed for the maintainers. The numbers that come from runs (errors, tolerances, levels) were produced by `docs/figures/make_figures.py`, which writes the complete input of every run into `docs/figures/inputs/<case>/`, runs the solver on a copy of it in a scratch directory and writes the figures into `docs/figures/data/` and the numbers into `docs/figures/data/numbers.txt`.

Building this guide.

```
cd FORTRAN/docs
make            # main.pdf (pdflatex, makeindex, pdflatex, pdflatex), copied to
    ↪ FORTRAN/Off_coustics_guide.pdf
make figures    # regenerates the data figures (runs the solver in a scratch
    ↪ directory)
make clean
```

Part I

User guide

Chapter 1

What the solver computes

1.1 The problem

The solver computes the underwater sound radiated by point dipoles placed in a layer of water: the complex acoustic pressure, frequency by frequency, at sets of observer points (lines, circles, planes, cylinders, spheres). The dipoles come from one of four *emitters*:

- **a wind turbine** (`WindTurbine`): the accelerations of the wet nodes of the support structure, computed by OpenFAST (SubDyn), are taken to the frequency domain and multiplied by the mass of each node to give the force that the structure applies on the water (Section 4.1);
- **a wind farm** (`WindFarm`): several turbines, each one computed as above, merged into one set of sources on a common frequency grid (Section 4.2);
- **hand-made sources given in frequency** (`FrequencySource`): point dipoles with a complex force per frequency, written in the namelist, read from files or given as expressions of the frequency (Section 4.3);
- **hand-made sources given in time** (`TimeSource`): point dipoles whose forces are real time series, taken to the frequency domain with the same FFT as the turbines (Section 4.4).

The sound is propagated by one of two *acoustic methods* (Chapter 5):

- **the image method** (`ImageMethod`): every dipole is mirrored across the free surface and the seabed, repeatedly, and the free-field fields of the real and image dipoles are added; each reflection can be attenuated by a constant factor;
- **analytical normal modes** (`AnalyticalNormalModes`): the exact field of a dipole in an ideal waveguide (pressure release surface, rigid seabed) as a sum of modes.

The emitter and the method are independent: any emitter runs with any method. The field is then sampled by the *post-processing steps* (Chapter 6) and written to an HDF5 file and a text summary (Chapter 7), which the plotting package turns into spectra, maps and source levels (Chapter 8).

1.2 Frame, units and conventions

Frame. The frame is Cartesian and right-handed: x and y are horizontal, z points upwards. The free surface is the plane $z = 0$ and the seabed the plane $z = -H$, with H the water depth `depth_base` of `Environment.nml` (Figure 1.1). Sources and observers lie in $-H \leq z \leq 0$. Azimuths and wind directions are in degrees, measured anticlockwise from the $+x$ axis (a wind direction of 0° points to $+x$).

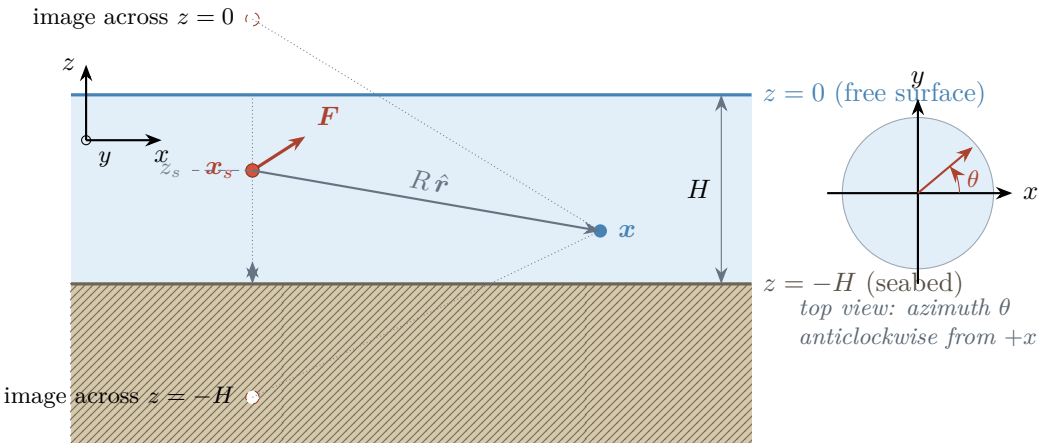


Figure 1.1: Frame and waveguide. The water occupies $-H \le z \le 0$; the free surface is $z = 0$ and the seabed $z = -H$. A dipole $\boldsymbol{F}$ at $\boldsymbol{x}_s$ radiates to an observer at $\boldsymbol{x}$ at the distance $R = |\boldsymbol{x} - \boldsymbol{x}_s|$ along $\hat{\boldsymbol{r}}$. The image method adds the fields of the images of the source across both planes (open circles); the first image across the surface is at $z = -z_s$ and the first across the seabed at $z = -2H - z_s$ (Section 5.1). Azimuths are measured anticlockwise from $+x$ (top view, right).

Units. All inputs and outputs are in SI units: m, s, Hz, kg, N, Pa, m/s, kg/m^3. Levels are in dB re the reference pressure `reference_pressure` of `Environment.nml`, $p_{\mathrm{ref}} = 1\,\mu$Pa by default.

Time factor and amplitudes. Every quantity of frequency f is a complex *peak* amplitude with the time factor $\mathrm{e}^{-\mathrm{i}\omega t}$, $\omega = 2\pi f$:

$$F(t) = \mathrm{Re}\left\{\hat{F}\,\mathrm{e}^{-\mathrm{i}\omega t}\right\}, \qquad p(t) = \mathrm{Re}\left\{\hat{p}\,\mathrm{e}^{-\mathrm{i}\omega t}\right\}. \tag{1.1}$$

So a force $A\cos(\omega t)$ is $\hat{F} = A$ and a force $A\sin(\omega t)$ is $\hat{F} = \mathrm{i}A$ (it lags the cosine by 90°). The FFT of the code follows the same convention (utils/SignalProcessing.f90:compute_rfft conjugates the FFTW output, Section 13.3). The root-mean-square value of a tone is $|\hat{p}|/\sqrt{2}$, and the sound pressure level used by the plotting package is

$$L_p = 20\log_{10}\frac{|\hat{p}|/\sqrt{2}}{p_{\mathrm{ref}}} \quad [\mathrm{dB\ re}\ p_{\mathrm{ref}}], \tag{1.2}$$

floored at 0 dB (plotting/utils/AcousticUtils.py:pressure_to_SPL).

The force. The force $\boldsymbol{F}$ of a source is the force that the source applies *on the fluid*, in newtons, in the global frame. For a turbine it is the force of the structure on the water at a node; for the hand-made sources it is whatever the user gives.

1.3 The field of a point dipole

A point force $\boldsymbol{F}\,\delta(\boldsymbol{x} - \boldsymbol{x}_s)$ applied on the fluid enters the linear acoustic equations as a source of momentum. With the time factor (1.1) the pressure satisfies the inhomogeneous Helmholtz equation

$$\nabla^2\hat{p} + k^2\hat{p} = \nabla\cdot\left[\boldsymbol{F}\,\delta(\boldsymbol{x} - \boldsymbol{x}_s)\right], \qquad k = \frac{\omega}{c}, \tag{1.3}$$

with c the speed of sound `speed_sound_base`. With the free-field Green's function $G(R) = \mathrm{e}^{\mathrm{i}kR}/(4\pi R)$, the solution of $\nabla^2 G + k^2 G = -\delta$ that radiates outwards, the pressure is $\hat{p} = -\boldsymbol{F}\cdot\nabla G$,

that is

$$\boxed{\hat{p}(\boldsymbol{x}) = (\boldsymbol{F}\cdot\hat{\boldsymbol{r}})\left(\frac{1}{R}-\mathrm{i}k\right)\frac{\mathrm{e}^{\mathrm{i}kR}}{4\pi R}}, \qquad \boldsymbol{x}-\boldsymbol{x}_s = R\,\hat{\boldsymbol{r}}. \tag{1.4}$$

This is the kernel of the image method, solvers/methods/ImageMethod.f90:dipole_pressure_images, which evaluates it as $(\boldsymbol{F}\cdot(\boldsymbol{x}-\boldsymbol{x}_s))\,(1/R-\mathrm{i}k)\,\mathrm{e}^{\mathrm{i}kR}/(4\pi R^2)$. It has a near field that decays as $1/R^2$ and a far field $-\mathrm{i}k(\boldsymbol{F}\cdot\hat{\boldsymbol{r}})\mathrm{e}^{\mathrm{i}kR}/(4\pi R)$; the two are equal at $kR=1$. The directivity is $\cos\theta$, with θ the angle between $\boldsymbol{F}$ and $\hat{\boldsymbol{r}}$: a dipole does not radiate in the plane normal to its force. Section 9.1 checks the solver against (1.4) to 7×10^{-16}.

Every method of the code evaluates the field of dipoles only: the sources carry a multipole label (`SOURCE_DIPOLE` = 1, `SOURCE_MONOPOLE` = 0, `types/SourceTypes.f90`) and both methods stop if a source is not a dipole.

1.4 What is not modelled

The model is deliberately simple; these effects are outside it:

- **Monopoles** and higher multipoles: the label exists, the kernels do not (Chapter 19).
- **A depth-dependent sound speed or density**: c and ρ are constants of `Environment.nml`.
- **A variable bathymetry**: the seabed is the plane $z=-H$. The code is prepared for it (a method only has to override the seabed query `z_seabed`, Section 14.4).
- **A sediment**: the seabed is either rigid (normal modes) or a plane with a constant reflection factor per bounce (`attenuation_bottom` of the image method), with no frequency or angle dependence and no penetration into the bottom.
- **Absorption in the water** inside the solver: k is real. The plotting package can subtract a Thorp-type absorption (Section 6.4).
- **Scattering by the structures** and the coupling between the sound and the motion of the structure: the turbine forces come from the OpenFAST accelerations and a frequency-dependent correction of the radiation of a cylinder (Section 13.2); the field of one turbine is not scattered by another.
- **Sources other than the forces of the structure**: piling, vessels, ambient noise, airborne noise transmitted through the surface, flow noise.
- **Non-linear acoustics**: the field is linear and the frequencies are independent.

Chapter 2

Installing and running

2.1 Dependencies

Table 2.1: Software used to build and run the solver and the plotting package. The versions are those of the machine where this guide was produced; other recent versions are expected to work.

Component	Version tested	Used for
gfortran (GNU Fortran)	13.3.0	the compiler (Fortran 2008 with submodules)
HDF5 [4] with Fortran bindings	1.10.10 (`h5fc`)	every output file (`utils/HDF5Utils.f90`)
FFTW3 [2]	3.3.10	the real FFT of `compute_rfft` (`-lfftw3`)
OpenMP	GCC's `libgomp`	parallel observers (`-fopenmp`)
GNU make	any	`FORTRAN/Makefile`
fpm (optional)	0.13.0	alternative build (`FORTRAN/fpm.toml`)
Python	3.9	the plotting package
numpy, h5py, matplotlib, scipy	2.0.2, 3.14.0, 3.9.4, 1.13.1	idem (and `docs/figures/make_figures.py`)
LaTeX with `dvipng`	TeX Live 2023	text of the figures (`text.usetex`) and this guide

On Ubuntu the libraries come from the packages `gfortran`, `libhdf5-dev` (it provides `h5fc`) and `libfftw3-dev`. On a cluster, load a Fortran-enabled HDF5 module (so that `h5fc` or `h5pfc` is on the `PATH`) and an FFTW3 module before building: the Makefile stops with *Could not find h5fc or h5pfc on PATH* otherwise.

2.2 Building

With make (from `FORTRAN/`):

```
make                                # builds ./off_coustics
make -j8                            # the same, in parallel
make FFLAGS="-O0 -g -fopenmp -fcheck=all -fbacktrace"   # debug build
make clean                          # removes build_make/ and the executable
make run ARGS=./my_inputs           # builds, then runs on the namelists of
    ↪ ./my_inputs
```

The Makefile compiles every module in dependency order into `build_make/` (objects, `.mod` and the `.smod` of the submodules) and links with `h5fc`, which adds the HDF5 paths by itself. The default flags are `-O3 -fopenmp`.

> **Pitfall** When overriding `FFLAGS`, keep `-fopenmp`: without it every module compiles but the link fails with undefined references to `omp_get_wtime_` and `omp_get_max_threads_`.

With fpm:

```
fpm build --flag "-O3 -fopenmp"
fpm run --flag "-O3 -fopenmp" -- ./path_to_input_files
```

fpm finds the sources of `src/` by itself, links HDF5 through its metapackage and FFTW3 through `link = ["fftw3"]` of `fpm.toml`. Its executable is named after the package, `Off_coustics`, and lives under `build/`.

> **Pitfall** After editing any module, rebuild before running. A binary older than its sources once accepted the new `simulation_type` labels in the parser while `main.f90` still expected the old ones: no emitter was built and the run crashed. `make` rebuilds what depends on the edited files; when in doubt, `make clean && make`.

2.3 Running

Command line.

```
./off_coustics                    # reads ./input_files/
./off_coustics ./my_case          # reads the namelists of ./my_case/
OMP_NUM_THREADS=8 ./off_coustics ./my_case
```

The only argument is the directory of the namelists (input/Args_parser.f90:get_input_path; a missing trailing `/` is added). More than one argument, or an empty one, stops the program. The directory must contain `Solver.nml`, `Environment.nml` and the folders `Emitters/`, `ComputationSetups/` and `AcousticMethods/` (Chapter 3).

Working directory. Every path written inside the namelists (OpenFAST files, `freq_file`, `source_file`, `save_path`, ...) and the default output folders (`./turbine_acoustic_data/`, `./farm_acoustic_data/`, `./source_acoustic_data/`) are relative to the directory where the program is started, *not* to the input directory. The turbine types also read their rpm curve from `../wind_speed_curves_DTU_10MW/rpm_ws.csv` relative to the working directory (Section 4.1).

> **Pitfall** A run deletes and rewrites any output file of the same name (unless `update_existing` keeps it, Section 7.4). Run test cases in a separate directory, not where your results are.

Threads. The observers are computed in parallel with OpenMP (Section 15.4); `OMP_NUM_THREADS` sets the number of threads. A run is deterministic: the same inputs, `block_size` and number of threads give the same bits. The image method gives the same bits for any block size and number of threads; the normal-mode method can differ in the last bits (measured: 3.6×10^{-16} of the peak pressure) when the blocks change (Section 15.4).

2.4 What a run prints

Listing 2.1 is the output of the `TimeSource` case of `input_files/` with the normal-mode method. The lines starting with `>>` come from the emitter and the solver; the mode table is printed by solvers/methods/AnalyticalNormalModes.f90:print_mode_summary; every step prints a header `[k/N]`, a geometry line, the WARNING lines of the observers it masks (Section 6.2), the norm of the pressure when `debug` is `.true.`, and the file it writes. The `.dat` summary is written when the parameters are saved, before the steps, and completed at the end (Section 7.3).

Listing 2.1: Standard output of a run (shortened where marked).

```
 === Off_coustics_fortran Initialization ===
 >> Reading the input files from: ./input_files/
 >> Instantiating TimeSource...
 >> [TimeSource] Time axis: 2000 samples, time step dt =  1.00000E-03 s (sampling frequency
    ↪ 1.00000E+03 Hz)
 >> [TimeSource] Record length =  2.00000E+00 s, frequency resolution df =  5.00000E-01 Hz, Nyquist ...
 >> [TimeSource] Forces taken to the frequency domain (Hann window): 81 frequencies from  3.000E+01 to
    ↪ ...
 >> Translating time-domain forces to mathematical acoustic sources...
 >> [AcousticSource] Frequency-domain forces handed to the solver as point dipoles
 >> Launching acoustic propagation solver: AnalyticalNormalModes

 Selected method: Analytical Normal Modes
================================================================================================
                                ANALYTICAL NORMAL MODES SETUP
================================================================================================
  Water Depth (H):           30.00 m
  Speed of Sound (c):      1500.00 m/s
  Frequency Band:            30.00 Hz -      70.00 Hz
  Retained Modes (m):            8
  Mode  | k_zm [rad/m]       | Vert. WL [m] | f_cutoff [Hz] | k_rm [rad/m]         | ... | Status
      1 | 0.0524+0.0000j     |       120.00 |         12.50 | 0.2885+0.0000j       | ... | Propagating
      ...
      4 | 0.3665+0.0000j     |        17.14 |         87.50 | 0.0000+0.2199j       | ... | Evanescent
    ---> Text summary saved at ./source_acoustic_data/Source_Ns1_Nt2000_ANM8_R0.5.dat

========================================================================
 RUNNING ACOUSTIC POST-PROCESSING (9 steps)
========================================================================

[1/9] Spectrums at observer points
>> [Spectrums] computing pressure at observer points...
   Pressure norm :     1.0124                                  (only with debug = .true.)
   Saved to: ./source_acoustic_data/Source_Ns1_Nt2000_ANM8_R0.5.hdf5

[2/9] Polar contour
>> [Polar]    r=  500.00 m | center=(    0.00,    0.00) m | z=  -15.00 m | observers=72
   ...
[5/9] Spherical radiation (free-field)
>> [Sphere]     skipped, not available for: Analytical Normal Modes
   ...
 ================================================
    Total elapsed time =    0.143 (s)
 ================================================
 Simulation finished successfully!
```

Chapter 3

Input files

3.1 The input directory

A case is a directory with two namelist files and three folders (Figure 3.1). The program reads `Solver.nml` and `Environment.nml`, and then one file of each folder, chosen by the names given in `Solver.nml`: the emitter file and the computation setup by `simulation_type`, the method file by `method` (input/Args_parser.f90:read_input_files).

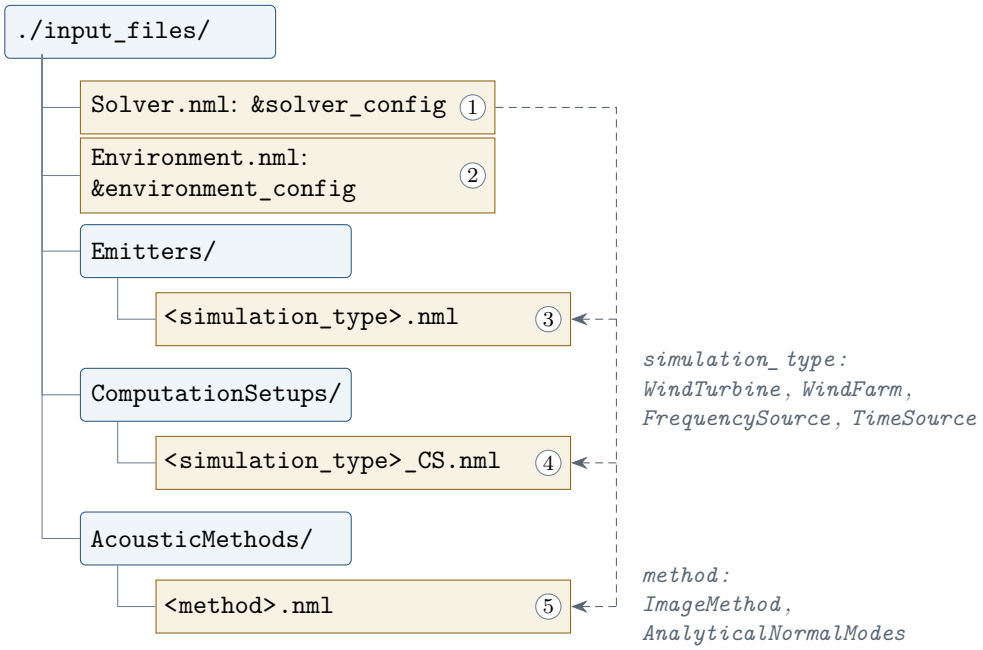


Figure 3.1: The input directory. `simulation_type` chooses the emitter file and its computation setup (the one with the suffix `_CS`); `method` chooses the file of the acoustic method. Only the files of the chosen emitter and method are read. The arrows give the reading order.

The values of `simulation_type` are `'WindTurbine'`, `'WindFarm'`, `'FrequencySource'` and `'TimeSource'`; those of `method` are `'ImageMethod'` and `'AnalyticalNormalModes'`. They are compared exactly, capitals included, and are also the file names. Any other value stops the run with *Unknown simulation_type* or *Unknown method*.

3.1.1 Rules shared by every namelist

- **Groups.** Each file holds one or more namelist groups, `&name ... /`. A reader rewinds the file and reads the first group of its name, so the order of the groups in a file does not

matter and a repeated group is ignored. Text outside the groups is ignored.

- **Comments** start with `!`. A namelist group ends at the first `/` that is not inside a quoted string or a comment, so `//` is *not* a comment: it ends the group and the lines that follow are silently not read (Section 10.2).
- **End of file.** The closing `/` must be followed by a newline; without it gfortran reports *End of file* and the run stops. The shipped files end with a blank line and a comment.
- **Defaults.** A variable left out takes the default of its reader, given in the tables below. Some defaults are sentinels: `-999.0` (reals) or `-999` (integers) mean *automatic*, and the value is computed at run time; `'None'` means *not used* for a file name. A few variables have no sentinel (e.g. `azimuth_sv`): `-999.0` is then used literally.
- **Strings** are read into buffers of 1024 characters and stored in shorter components (200 characters for paths, 100 for `extra_label` and `farm_name`); a longer string stops the run instead of being cut (input/InputChecks.f90:check_fits).
- **Paths** written in a namelist are relative to the working directory (Chapter 2).
- **Errors** stop the run with a line `ERROR: &group: ...` that names the variable; the complete list is in Appendix C.

3.2 Solver.nml

Table 3.1: Variables of `&solver_config` (`Solver.nml`, input/InputSolver.f90:read_solver_config).

Variable	Unit	Default	Valid values, checks	Meaning
`simulation_type`	–	`'WindTurbine'`	`'WindTurbine'`, `'WindFarm'`, `'FrequencySource'`, `'TimeSource'`	The emitter; also the name of its files
`method`	–	`'ImageMethod'`	`'ImageMethod'`, `'Analytical NormalModes'`	The acoustic method; also the name of its file
`save_path`	–	`'None'`	`'None'` or `'Default'` (any case), or a path without extension	Output file: automatic name (Section 7.1) or the given path plus `.hdf5`
`extra_label`	–	`'None'`	any text, 100 characters	Appended to the file name as `_<extra_label>`; `'None'` or empty adds nothing
`block_size`	–	128	≥ 1	Observers per parallel block (clamped so that there are at least as many blocks as threads, Section 15.4)
`in_cluster`	–	`.true.`	logical	`.true.`: observers in parallel blocks; `.false.`: one observer at a time, with a progress line every 100
`debug`	–	`.false.`	logical	Prints the norm of the pressure of every step
`update_existing`	–	`.true.`	logical	Keep an existing output of the same case and rewrite only the steps run now (Section 7.4)

Table 3.1 (continued)

Variable	Unit	Default	Valid values, checks	Meaning
`mask_distance`	m	5.0	≥ 0, else stop	Observers closer than this to a source get NaN; 0 disables it (Section 6.2)

The shipped `Solver.nml` sets `mask_distance` to 1.0 m and `debug` to `.true.`.

3.3 Environment.nml

Table 3.2: Variables of `&environment_config` (`Environment.nml`, input/InputSolver.f90: read_environment_config). All four must be > 0, otherwise the run stops printing the four values read.

Variable	Unit	Default	Valid values, checks	Meaning
`depth_base`	m	30.0	> 0	Water depth H: the seabed is $z = -H$ for every method and step
`speed_sound_base`	m/s	1500.0	> 0	Speed of sound c
`density_base`	kg/m^3	1025.0	> 0	Density ρ of the water
`reference_pressure`	Pa	10^{-6}	> 0	Reference pressure of the levels

3.4 Emitters

3.4.1 WindTurbine.nml

Table 3.3: Variables of `&turbine_config` (`Emitters/WindTurbine.nml`, input/InputEmitters.f90:read_turbine_config).

Variable	Unit	Default	Valid values, checks	Meaning
`case_type`	–	`'DTU10MW Monopile'`	`'DTU10MWMonopile'`, `'DTU10MWFloating'`, `'SAITEC2MWFloating'`; else *Unsupported turbine type*	Turbine type (Section 4.1)
`rootname`	–	`'DTU_DeltaWind_mn_ws11.4'`	200 characters	OpenFAST run name, without extension
`OP_output_dir`	–	`'../OP_output/'`	must end with `/`	Directory of the OpenFAST files
`OP_output_filename`	–	`'None'`	`'None'`/`'Default'` or a full path; the file must exist	OpenFAST output; by default `OP_output_dir//rootname//'.outb'` (or `'.out'`)

Table 3.3 (continued)

Variable	Unit	Default	Valid values, checks	Meaning
`OP_SubDyn_filename`	–	`'None'`	idem	SubDyn summary; by default `OP_output_dir//rootname//'.SD.sum.yaml'`
`Binary`	–	`.true.`	logical	The extension of the default output name: `.outb` or `.out` (the reader follows the actual extension)
`WindSpeed`	m/s	11.4	–	Wind speed: rpm of the drivetrain filter and the `_ws` part of the file name
`WindDir`	deg	0.0	–	Wind direction, anticlockwise from $+x$; the structure is rotated by it
`Nmembers`	–	8	> 0 (checked when reading OpenFAST)	Members of the SubDyn summary, rigid links included
`Nnodes`	–	5	> 0	Nodes per member
`AxisPos`	m	0.0, 0.0	–	Position (x, y) of the turbine axis
`BariPos`	m	0.0, 0.0	exactly $(0, 0)$ = automatic	Barycentre (x, y); automatic: mean (x, y) of the nodes
`From`	–	0.0	$0 \leq$ `From` $<$ `Upto` ≤ 1	Fraction of the time series where reading starts
`Upto`	–	1.0	idem	Fraction where reading stops
`skip`	–	1	≥ 1	Keep one time sample out of `skip`
`skipf`	–	1	≥ 1	Keep one frequency bin out of `skipf`
`filter_freqs`	–	`.true.`	logical	Keep only the bins near the drivetrain excitation (Section 4.1)
`freqs_over`	Hz	10.0	–	Bins below this are never filtered out

The water depth of the turbine is not an input of this file: it is `depth_base` of `Environment.nml`.

3.4.2 WindFarm.nml

A farm file has one `&farm_config` group with the settings shared by every turbine and one `&turbine` group per turbine, in any number (input/InputEmitters.f90:read_farm_config). The reader counts the `&turbine` groups, then reads them in file order; the defaults of Table 3.5 are reset before each one, so a value left out never carries over from the previous turbine. Every turbine gets the six reading and filtering settings of `&farm_config` and the depth of `Environment.nml`; they cannot be set per turbine.

Table 3.4: Variables of `&farm_config` (`Emitters/WindFarm.nml`).

Variable	Unit	Default	Valid values, checks	Meaning
`farm_name`	–	`'None'`	100 characters	`'None'`: automatic file name; otherwise the head of the name becomes `Farm_<farm_name>`
`From`	–	0.0	as in Table 3.3	Start fraction, every turbine
`Upto`	–	1.0	idem	Stop fraction
`skip`	–	1	≥ 1	Time decimation
`skipf`	–	1	≥ 1	Frequency decimation
`filter_freqs`	–	`.true.`	logical	Drivetrain frequency filter
`freqs_over`	Hz	10.0	–	Lower limit of the filter

Table 3.5: Variables of every `&turbine` group of `Emitters/WindFarm.nml`. A group with any other variable (e.g. `From`) stops the run with its number.

Variable	Unit	Default	Valid values, checks	Meaning
`case_type`	–	`'DTU10MW Monopile'`	as in Table 3.3	Turbine type
`rootname`	–	`'None'`	mandatory: `'None'` stops the run	OpenFAST run name
`OP_output_dir`	–	`'../OP_ output/'`	ends with `/`	Directory of its files
`OP_output_ filename`	–	`'None'`	as in Table 3.3	OpenFAST output
`OP_SubDyn_ filename`	–	`'None'`	idem	SubDyn summary
`Binary`	–	`.true.`	logical	`.outb` or `.out`
`WindSpeed`	m/s	11.4	–	Wind speed of this turbine
`WindDir`	deg	0.0	–	Wind direction of this turbine
`AxisPos`	m	0.0, 0.0	–	Axis position
`BariPos`	m	0.0, 0.0	$(0, 0)$ = automatic	Barycentre
`Nmembers`	–	8	> 0	Members
`Nnodes`	–	5	> 0	Nodes per member

Two groups with the same `rootname` print a WARNING: they are *replicas*, the same OpenFAST time series at two positions, so their forces have the same phases and the two turbines radiate as fully coherent sources (they interfere instead of adding their powers).

3.4.3 FrequencySource.nml

The group `&frequency_source_config` describes point dipoles with a complex force per frequency (input/InputEmitters.f90:read_frequency_source_config). Each of its three items, the positions, the frequency axis and the forces, is given in one of the ways of Table 3.6. A way is chosen by giving its values (an entry of a list, a value other than `-999.0`, a file name other than `'None'`, an expression other than `'0'`); two ways of the same item stop the run with a message that names both (*the frequency axis is given in more than one way (freqs and freq_file): give only one of them*), and with none the written list and forces are taken.

Table 3.6: Input modes of `FrequencySource`. The limits of the modes written by hand are the sizes of the parse buffers `MAX_PARSE_SRC` = 20 and `MAX_PARSE_FREQ` = 200 (`Kinds.f90`); what is read from files has no limit.

Item	Written in the namelist	From files, a grid or expressions
Positions	(a) `source_xyz(i_s, 1:3)`, at most 20 sources	(b) `source_file`: one row `x y z` per source; `Nsources` is −999 or the number of rows
Frequencies	(a) `freqs`: `Nfreqs` values, at most 200	(b) a grid: `Nfreqs` ≥ 2 values from `fmin` (> 0) to `fmax`, `freq_spacing 'linear'` or `'log'`; (c) `freq_file`: one frequency per row; `Nfreqs` is −999 or the number of rows
Forces	(a) `force_re(i_f, i_s, 1:3)`, mandatory, and `force_im` (0 if left out), at most 200 frequencies and 20 sources	(b) `force_re_expr(i_s, 1:3)` and `force_im_expr`: expressions of f [Hz], `'0'` if left out, at most 20 sources; (c) `force_re_file` and `force_im_file` (`'None'`: imaginary part 0): one row per frequency, $3N_s$ columns `Fx1 Fy1 Fz1 Fx2 ...`

Table 3.7: Variables of `&frequency_source_config` (`Emitters/FrequencySource.nml`). NaN marks the entries of the written lists that were not given.

Variable	Unit	Default	Valid values, checks	Meaning
`Nsources`	–	−999	1 to 20 if written; −999 or the rows of `source_file`	Number of sources
`source_xyz`	m	NaN	every entry of the first `Nsources` rows, finite, $z \leq 0$; nothing beyond	`source_xyz(i_s, 1:3) = x, y, z`
`source_file`	–	`'None'`	must exist; 3 columns; finite, $z \leq 0$	Positions read from a text file
`Nfreqs`	–	−999	1 to 200 for a list; ≥ 2 for a grid; −999 or the rows of `freq_file` (needed if the forces are written)	Number of frequencies
`freqs`	Hz	NaN	`Nfreqs` finite values, positive, strictly increasing	The list of frequencies
`fmin`	Hz	−999.0	> 0; with `fmax`	First frequency of the grid
`fmax`	Hz	−999.0	$>$ `fmin`	Last frequency of the grid (up to rounding)
`freq_spacing`	–	`'linear'`	`'linear'` or `'log'` (any case); ignored with a WARNING without a grid	Constant step $f_i = f_{\min} + (i-1)\Delta f$ or constant ratio $f_i = 10^{\log_{10} f_{\min} + (i-1)\delta}$
`freq_file`	–	`'None'`	must exist; 1 column; positive and strictly increasing	Frequencies read from a text file
`force_re`	N	NaN	every `(i_f, i_s)` line of the first `Nfreqs` and `Nsources`; finite	Real part, `force_re(i_f, i_s, 1:3) = Fx, Fy, Fz`
`force_im`	N	0.0	finite	Imaginary part

Table 3.7 (continued)

Variable	Unit	Default	Valid values, checks	Meaning
`force_re_expr`	N	`'0'`	valid expression of `f`, 256 characters, finite at every frequency	Real part as a function of f
`force_im_expr`	N	`'0'`	idem	Imaginary part as a function of f
`force_re_file`	–	`'None'`	must exist; $3N_s$ columns; one row per frequency; finite	Real part read from a text file
`force_im_file`	–	`'None'`	only with `force_re_file`; same layout	Imaginary part read from a text file

Conventions of the force. The force is the peak complex amplitude with the time factor $\mathrm{e}^{-\mathrm{i}\omega t}$ (Section 1.2): $F(t) = \mathrm{Re}\{(F_{\mathrm{re}} + \mathrm{i}F_{\mathrm{im}})\mathrm{e}^{-\mathrm{i}\omega t}\}$, so $A\cos\omega t$ is `force_re = A` and $A\sin\omega t$ is `force_im = +A`. The relative phase between sources and between components is kept.

Uniform frequencies and the PSD. Every frequency is a tone. When the frequencies are uniformly spaced by Δf (a list or a file with a constant step, or a `'linear'` grid), the plotting package also draws the PSD and the 1/3 octave bands, taking each tone as the power of a band Δf wide (Section 8.2). To represent a continuous force spectrum $S_F(f)$ [N^2/Hz], give

$$|F_k| = \sqrt{2\, S_F(f_k)\, \Delta f}, \tag{3.1}$$

as in the example of Section 3.7.4. For isolated tones the PSD depends on Δf and has no physical meaning: use the SPL.

Text files. `source_file`, `freq_file`, `force_re_file` and `force_im_file` are read by utils/CSVReader.f90:read_table: values separated by blanks, tabs or commas; blank lines and lines starting with `#` or `!` are skipped; a row with another number of values, or a value that is not a number, stops the run with the number of its line.

3.4.4 TimeSource.nml

The group `&time_source_config` describes point dipoles whose forces are real series in time (input/InputEmitters.f90:read_time_source_config); they are taken to the frequency domain with the FFT of the turbines (Section 4.4).

Table 3.8: Variables of `&time_source_config` (`Emitters/TimeSource.nml`).

Variable	Unit	Default	Valid values, checks	Meaning
`Nsources`	–	−999	1 to 20 if written; −999 or the rows of `source_file`	Number of sources
`source_xyz`	m	NaN	as in Table 3.7	Positions
`source_file`	–	`'None'`	as in Table 3.7	Positions read from a text file
`t_start`	s	−999.0	finite; needed without `time_file`	First time of the uniform axis
`t_end`	s	−999.0	$>$ `t_start`	End of the axis, not included
`Nt`	–	−999	≥ 4	Samples: $\Delta t = (t_{\mathrm{end}} - t_{\mathrm{start}})/N_t$

Table 3.8 (continued)

Variable	Unit	Default	Valid values, checks	Meaning
`time_file`	–	`'None'`	must exist; uniformly spaced to 10^{-6}; ≥ 4 rows	Times read from a text file (then `t_start`, `t_end`, `Nt` are ignored with a WARNING)
`force_expr`	N	`'0'`	valid expression of `t`, finite at every time; at most 20 sources	Force `force_expr(i_s, 1:3)` as a function of t
`force_file`	–	`'None'`	must exist; N_t rows and $3N_s$ columns	Forces read from a text file (then `force_expr` is ignored with a WARNING)
`skipf`	–	1	≥ 1	Keep the bins $1, 1+$`skipf`, ... (a tone can be dropped)
`fmin`	Hz	-999.0	≥ 0 or -999 (all)	Lowest frequency kept
`fmax`	Hz	-999.0	> 0, $>$ `fmin`, or -999 (up to Nyquist)	Highest frequency kept

The time step, the record length $T = N_t \Delta t$, the frequency resolution $\Delta f = 1/T$ and the Nyquist frequency $1/(2\Delta t)$ are printed when the run starts. A tone gets its full amplitude only if its frequency is a multiple of Δf (Section 9.3).

3.4.5 The expression language

`force_expr` (time) and `force_re_expr`, `force_im_expr` (frequency) are text expressions of one variable, `t` [s] or `f` [Hz], compiled when the namelist is read (utils/ExpressionParser.f90:compile_expression) and evaluated on the whole axis at once (`evaluate_expression`). They may use:

- numbers: `12`, `0.5`, `.5`, `1.0e-3`, `2d3`;
- the variable (`t` or `f`) and the constant `pi`;
- the operators `+ - * / **` and parentheses; `**` is right-associative and binds tighter than the unary minus, so `-t**2` is $-(t^2)$ and `2**3**0.5` is $2^{(3^{0.5})}$;
- the functions `sin cos tan asin acos atan sinh cosh tanh exp log log10 sqrt abs floor` of one argument, `atan2(y, x)`, `min(a, b)`, `max(a, b)`, `mod(a, b)` (the sign of `a`, as Fortran's `mod`) and `modulo(a, b)` (the sign of `b`), and `step(x)` (1 if $x \geq 0$, else 0) and `sgn(x)` (-1, 0 or 1).

Upper and lower case are the same and blanks are ignored. An expression has at most 256 characters and 256 operations. Examples:

```
force_expr(1, 1:3) = '0', '0', '1000*sin(2*pi*50*t)'                    ! a 50
    ↪ Hz tone
force_expr(2, 1:3) = '500*exp(-((t-1.0)/0.1)**2)*cos(2*pi*80*t)', '0', '0'   ! a
    ↪ burst
force_expr(3, 1:3) = '0', '0', '200*step(t-0.5)*sgn(sin(2*pi*10*t))'    ! a
    ↪ switched square wave
force_re_expr(1, 1:3) = '0', '0', '100/(1+(f/50)**2)'                   ! a
    ↪ low-pass spectrum
force_im_expr(1, 1:3) = '0', '0', '50*exp(-((f-80)/10)**2)'             ! a band
    ↪ around 80 Hz
```

A syntax error or an unknown name stops the run when the namelist is read, naming the variable, the source, the component and the position in the text, e.g. *force_re_expr(1, Fz): unknown*

*name 't' (use f, pi or a function) at position 14 of 'sin(2*pi*50*t)'*. A value that is not finite (a division by zero, a logarithm of a negative number) stops the run naming the first time or frequency where it happens.

> **Pitfall** In a time-domain force, tones should be multiples of $\Delta f = 1/T$; any other frequency is shared between the two nearest bins (Figure 9.2). And `skipf` > 1 keeps one bin out of `skipf` starting with the first: a tone on a dropped bin disappears. Use `fmin` and `fmax` to limit the band instead.

3.5 Computation setups

The file `ComputationSetups/<simulation_type>_CS.nml` switches the post-processing steps on and gives their parameters (input/InputPostProcessing.f90:read_computation_config). The group `&do_config` is always read; the group of a step is read only if the step runs, so a file may leave out the groups of the steps it does not run (`WindFarm_CS.nml` has no decay or sphere group). A missing group of a step that runs stops the run, except `&sliceVertical_config`, whose defaults are then used with a WARNING. The geometry of each step is described in Chapter 6.

Table 3.9: Variables of `&do_config`.

Variable	Unit	Default	Valid values, checks	Meaning
`do_all`	–	`.true.`	logical	Master switch: `.true.` runs every step and ignores the others
`do_spectrums`	–	`.true.`	logical	Spectra at observer points
`do_polar`	–	`.true.`	logical	Horizontal circle
`do_cylinder`	–	`.true.`	logical	Vertical cylinder
`do_decay`	–	`.true.`, `.false.` for a farm	logical; always off for a farm	Line along the wind direction
`do_line`	–	`.true.`	logical	Line between two points
`do_sliceXY`	–	`.true.`	logical	Horizontal plane
`do_sliceXZ`	–	`.true.`	logical	Vertical plane $y =$ constant
`do_sliceVertical`	–	`.false.`	logical	Vertical plane at any azimuth
`do_sphere`	–	`.true.`, `.false.` for a farm	logical; always off for a farm; image method only	Free-field sphere

In the tables below, *centre* and *origin* are the reference points of the emitter (Table 6.1), and $z_{\mathrm{bot}} = -H$ is the seabed.

Table 3.10: Variables of `&spectrums_config`.

Variable	Unit	Default	Valid values, checks	Meaning
`n_observers`	–	-999	-999 or 1 to 200	-999: three points $(10, 0)$, $(250, 0)$, $(500, 0)$ m at mid-depth (absolute positions)
`observers_x`	m	NaN	`n_observers` values; more: WARNING	x of the observers

Table 3.10 (continued)

Variable	Unit	Default	Valid values, checks	Meaning
`observers_y`	m	NaN	idem	y
`observers_z`	m	NaN	idem; $z > 0$: WARNING and NaN	z

Table 3.11: Variables of `&polar_config`.

Variable	Unit	Default	Valid values, checks	Meaning
`r_polar`	m	-999.0 (500)	–	Radius of the circle
`z_polar`	m	-999.0 ($-H/2$)	–	Depth of the circle
`n_polar`	–	-999 (72)	-999 or ≥ 1	Points: $\theta_i = (i-1)\,360°/n$
`center_polar`	m	-999.0 $\times 2$ (centre)	both or none	Centre (x, y)

Table 3.12: Variables of `&cylinder_config`. The levels go from the seabed to the surface, both included.

Variable	Unit	Default	Valid values, checks	Meaning
`r_cyl`	m	-999.0 (500)	–	Radius
`n_cyl`	–	-999 (72)	-999 or ≥ 1	Azimuthal points
`nz_cyl`	–	-999 (20)	-999 or ≥ 1	Vertical levels
`center_cyl`	m	-999.0 $\times 2$ (centre)	both or none	Axis (x, y)

Table 3.13: Variables of `&decay_config`. The line starts at the origin and follows the direction of the emitter.

Variable	Unit	Default	Valid values, checks	Meaning
`distance_decay`	m	-999.0 $\times 2$ (10, 500)	both or none; start ≤ 0: WARNING, set to 1 m; log spacing needs end > 0	Start and end distance
`n_decay`	–	-999 (200)	-999 or ≥ 1	Points
`z_decay`	m	-999.0 ($-H/2$)	–	Depth
`logspace_decay`	–	`.true.`	logical, no sentinel	Logarithmic spacing

Table 3.14: Variables of `&line_config`. Each component of `p1_line` and `p2_line` is resolved on its own.

Variable	Unit	Default	Valid values, checks	Meaning
`p1_line`	m	-999.0 $\times 3$ $(100, 0, 0)$	$\neq$ `p2_line`	Start point
`p2_line`	m	-999.0 $\times 3$ $(100, 0, -H)$	idem	End point
`n_line`	–	-999	-999 or ≥ 1	Points; automatic: $\lfloor L/5 \rfloor$ if $L > 200$ m, else $\max(2, \lfloor L \rfloor)$

Table 3.14 (continued)

Variable	Unit	Default	Valid values, checks	Meaning
`logspace_line`	–	`.false.`	logical, no sentinel	Logarithmic spacing, from $10^{-6}L$ to L

Table 3.15: Variables of `&sliceXY_config`.

Variable	Unit	Default	Valid values, checks	Meaning
`z_xy`	m	-999.0 ($-H/2$)	> 0 is changed to $-z$ with a message	Depth
`nx_xy`	–	-999 (26)	-999 or ≥ 1	Points along x
`ny_xy`	–	-999 (26)	-999 or ≥ 1	Points along y
`xlim_xy`	m	-999.0 $\times 2$ (centre ± 500)	both or none	x limits
`ylim_xy`	m	-999.0 $\times 2$ (centre ± 500)	both or none	y limits
`center_xy`	m	-999.0 $\times 2$	both or none; always set to the middle of the limits	Informative only

Table 3.16: Variables of `&sliceXZ_config`.

Variable	Unit	Default	Valid values, checks	Meaning
`y_xz`	m	-999.0 (y of the origin)	–	y of the plane
`nx_xz`	–	-999 (26)	-999 or ≥ 1	Points along x
`nz_xz`	–	-999 (26)	-999 or ≥ 1	Points along z
`xlim_xz`	m	-999.0 $\times 2$ (x of the centre ± 500)	both or none	x limits
`zlim_xz`	m	-999.0 $\times 2$ ($0, -H$)	each component on its own	First and last level (from the surface down by default)

Table 3.17: Variables of `&sliceVertical_config`. The plane contains the vertical and the horizontal axis $(-\sin\phi, \cos\phi)$; its normal is $(\cos\phi, \sin\phi)$.

Variable	Unit	Default	Valid values, checks	Meaning
`azimuth_sv`	deg	0.0	no sentinel	Azimuth ϕ of the normal: $0°$ is the plane YZ
`center_sv`	m	-999.0 $\times 2$ (origin)	both or none	Centre of the plane
`width_sv`	m	-999.0	–	Horizontal span; automatic: every source projected on the plane plus `margin_sv` on both sides
`zlim_sv`	m	-999.0 $\times 2$ ($-H, 0$)	clipped to the water column	Depth limits (min, max)

Table 3.17 (continued)

Variable	Unit	Default	Valid values, checks	Meaning
`nu_sv`	–	−999	−999 or ≥ 1	Points along the span; automatic: $\max(30, \lceil \text{span}/5\,\text{m} \rceil)$
`nz_sv`	–	−999	−999 or ≥ 1	Points along z; automatic: $\max(30, \lceil \text{depth}/2\,\text{m} \rceil)$
`margin_sv`	m	10.0	no sentinel	Margin of the automatic width

Table 3.18: Variables of `&sphere_config` (image method only).

Variable	Unit	Default	Valid values, checks	Meaning
`r_sphere`	m	−999.0	−999 or 0: automatic	Radius; automatic: $a_{\max} + 3/k_{\text{far}}$, $a_{\max}$ the largest distance of a source to the centre
`n_theta_sphere`	–	−999 (144)	−999 or ≥ 1	Azimuthal cells
`nz_sphere`	–	−999 (40)	−999 or ≥ 1	Bands of equal height (equal area)
`f_farfield_sphere`	Hz	−999.0 (10)	> 0	Frequency of $k_{\text{far}} = 2\pi f/c$ in the automatic radius
`center_sphere`	m	−999.0 ×3	all or none	Centre; automatic: the middle of the box that holds every source

3.6 Acoustic methods

Table 3.19: Variables of `&ImagesMethod_config` (`AcousticMethods/ImageMethod.nml`, input/InputSolver.f90:read_ImageMethod_config). The reader does not check their values.

Variable	Unit	Default	Valid values, checks	Meaning
`N_images`	–	30	≥ 0	Image levels on each side: $1 + 2N$ dipoles per source; 0 is free field
`UpperBC`	–	−1	−1 or +1	Surface: −1 pressure release ($p = 0$), +1 rigid
`LowerBC`	–	+1	−1 or +1	Seabed: same codes
`attenuation_surface`	–	1.0	0 to 1	Factor applied at every surface reflection
`attenuation_bottom`	–	0.5	0 to 1	Factor applied at every seabed reflection
`epsilon`	m	10^{-6}	> 0	The distance R is floored at this value

Table 3.20: Variables of `&AnalyticalNormalModes_config` (`AcousticMethods/AnalyticalNormalModes.nml`, input/InputSolver.f90:read_AnalyticalNormalModes_config).

Variable	Unit	Default	Valid values, checks	Meaning
`Nmodes`	–	-999	-999 or ≥ 1	Modes; automatic: the modes that propagate at the highest frequency plus 5
`epsilon`	m	10^{-12}	> 0	The horizontal distance is floored at this value (the shipped file sets 10^{-6})

3.7 Worked examples

The four examples below are complete cases: their files are in `docs/figures/inputs/<case>/` and `make_figures.py` runs them. Only the emitter file is shown; the other files set `simulation_type`, the method (`ImageMethod` with the default values of Table 3.19), the steps, a depth of 30 m and the default water properties.

3.7.1 A tone radiated by one dipole

A vertical force of 1000 N at 100 Hz, 10 m deep, and the vertical slice through it (Figure 3.2):

```
&frequency_source_config
    Nsources = 1
    source_xyz(1, 1:3) = 0.0, 0.0, -10.0
    Nfreqs = 1
    freqs = 100.0
    force_re(1, 1, 1:3) = 0.0, 0.0, 1000.0
/


! end
```

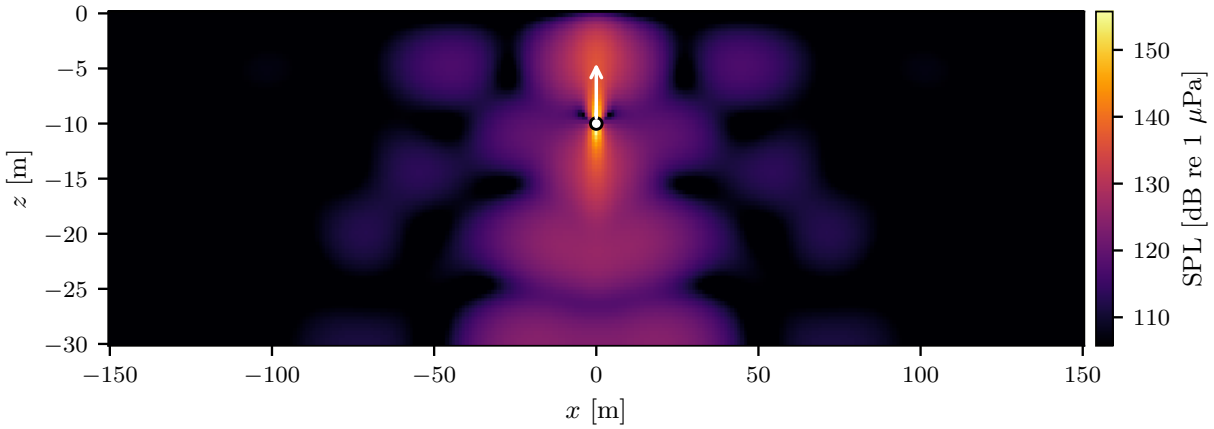


Figure 3.2: SPL of a vertical dipole of 1000 N at 100 Hz, at $z = -10$ m in 30 m of water (image method, `N_images` = 30, bottom attenuation 0.5), on the plane $y = 0$ (`sliceXZ`, 301 × 91 points; vertical scale exaggerated). Look at the null on the horizontal through the source, where $\boldsymbol{F} \cdot \hat{\boldsymbol{r}} = 0$ for the direct path, and at the interference lobes made by the images.

3.7.2 Two sources with a phase shift

Two horizontal dipoles $F_x = 100$ N at 150 Hz, a quarter of a wavelength apart along y ($\lambda/4 = 2.5$ m), the second lagging by 90° (`force_im = +100`, a sine against the cosine of the first):

```
&frequency_source_config
    Nsources = 2
    source_xyz(1, 1:3) = 0.0, -1.25, -15.0
    source_xyz(2, 1:3) = 0.0, 1.25, -15.0
    Nfreqs = 1
    freqs = 150.0
    force_re(1, 1, 1:3) = 100.0, 0.0, 0.0
    force_re(1, 2, 1:3) = 0.0, 0.0, 0.0
    force_im(1, 2, 1:3) = 100.0, 0.0, 0.0
/


! end
```

In the direction $+y$ the delay of the second source compensates its shorter path, and the two add; in the direction $-y$ they cancel (Figure 3.3). The plane $x = 0$ is an exact null of both dipoles.

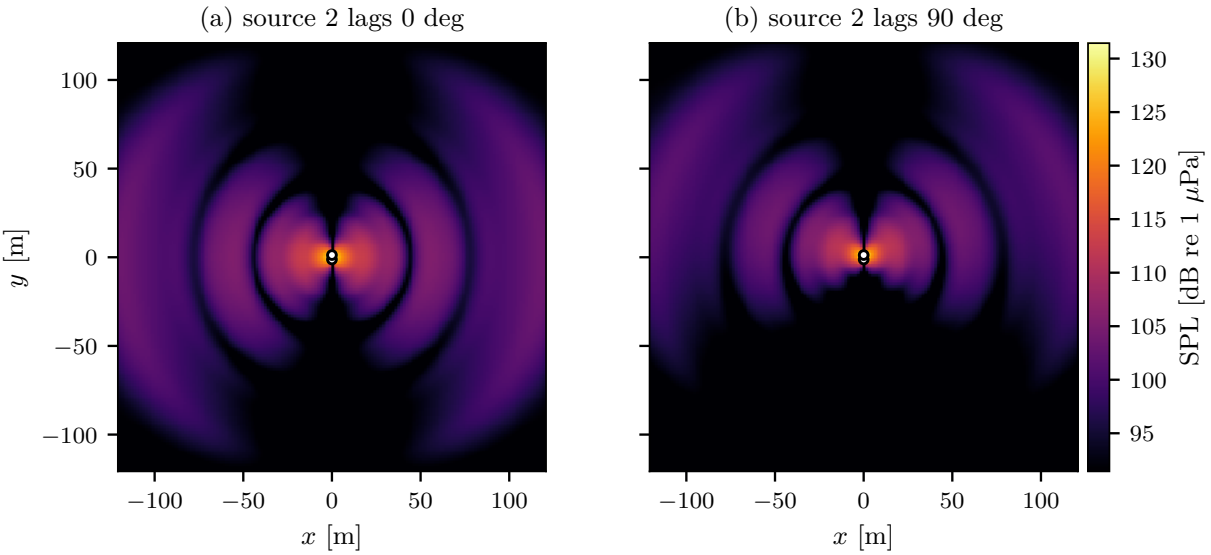


Figure 3.3: SPL on the horizontal plane through two F_x dipoles at $(0, \mp 1.25, -15)$ m, 150 Hz (image method). (a) In phase: symmetric in y. (b) Source 2 (at $+y$) lagging 90°: the field is stronger towards $+y$. The vertical line $x = 0$ is the null of an F_x dipole (drawn at the lowest level of the colour scale).

3.7.3 A burst in the time domain

A Gaussian burst of 80 Hz, 2 s sampled at 2 kHz, with the band 20 to 200 Hz passed to the solver (Figure 3.4):

```
&time_source_config
    Nsources = 1
    source_xyz(1, 1:3) = 0.0, 0.0, -10.0
    t_start = 0.0
    t_end = 2.0
    Nt = 4000
    force_expr(1, 1:3) = '0', '0', '500*exp(-((t-1.0)/0.05)**2)*cos(2*pi*80*t)'
```

```
    skipf = 1
    fmin = 20.0
    fmax = 200.0
/


! end
```

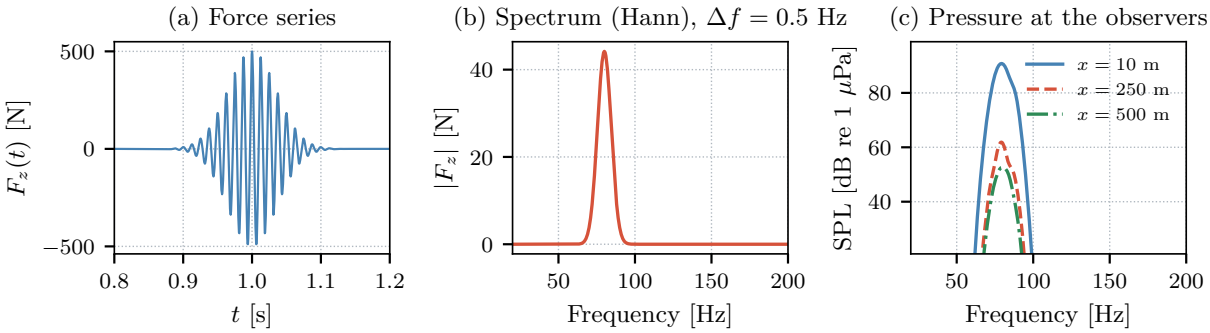


Figure 3.4: A `TimeSource` burst $500\,\mathrm{e}^{-((t-1)/0.05)^2}\cos(2\pi\,80\,t)$ N: (a) the series, (b) the amplitude of its spectrum after `compute_rfft` (Hann window, $\Delta f = 0.5$ Hz), (c) the SPL at the three default observers (10, 250 and 500 m at mid-depth).

3.7.4 A force spectrum read from files

A continuous vertical force spectrum $S_F(f) = 10^4/(1 + (f/30)^2)$ N^2/Hz sampled every hertz from 1 to 300 Hz, 300 frequencies, more than the 200 that can be written by hand, given as two files with $|F_k| = \sqrt{2 S_F \Delta f}$ (3.1):

```
&frequency_source_config
    Nsources = 1
    source_xyz(1, 1:3) = 0.0, 0.0, -10.0
    freq_file = 'freqs.txt'
    force_re_file = 'force_re.txt'
/


! end
```

`freqs.txt` has one frequency per row and `force_re.txt` three columns, `Fx Fy Fz`:

```
# generated by make_figures.py
0.0  0.0  141.34285422946365
0.0  0.0  141.10813025753959
...
```

The frequencies are uniformly spaced, so the pressure PSD is meaningful (Figure 3.5).

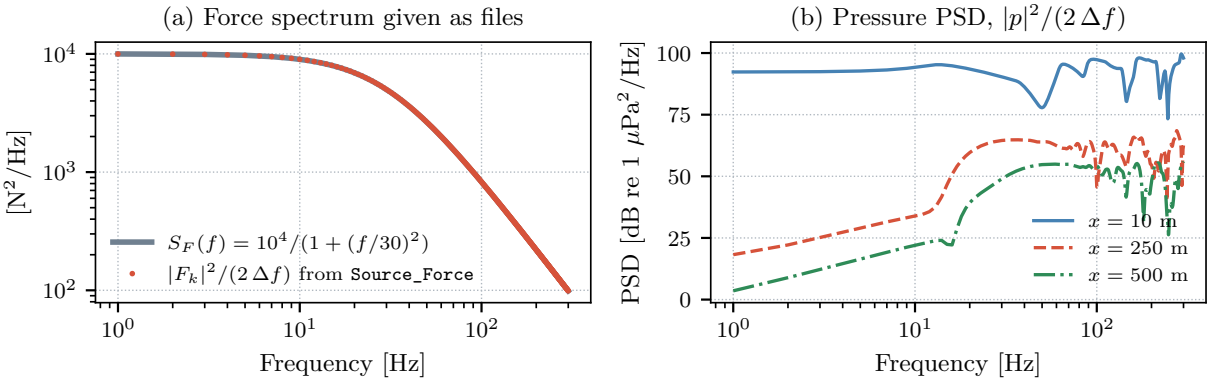


Figure 3.5: (a) The force spectrum given as files and the one recovered from `Source_Force` of the output, $|F_k|^2/(2\Delta f)$ (equal to 2×10^{-16}). (b) The PSD of the pressure at the three default observers.

Chapter 4

Emitters

An emitter turns its input into a set of point dipoles: positions $\boldsymbol{x}_s$, a frequency vector f_k and the complex forces $\boldsymbol{F}_s(f_k)$, the `AcousticSource_t` of Section 12.3. This chapter tells what each emitter does with its input; Chapter 13 gives the implementation.

4.1 Wind turbine

4.1.1 What is read

A turbine is computed from an OpenFAST [6] simulation with SubDyn:

- the **SubDyn summary** `<rootname>.SD.sum.yaml`: the list of members (the line starting with `#Member I Joint1_ID Joint2_ID`, one row per member with its node identifiers) and the node coordinates (the block after the header `# Node_[#] X_[m] Y_[m] Z_[m]`). `Nmembers` and `Nnodes` tell how many members and nodes per member to take;
- the **OpenFAST output** `<rootname>.outb` (binary) or `<rootname>.out` (text): the time and the accelerations of every node, the channels `M<m>N<n>TAxe`, `M<m>N<n>TAye` and `M<m>N<n>TAze` for every member m and node n (emitters/OpenFASTReader.f90:read_input_SD). The reader follows the extension of the file.

The OpenFAST run must therefore output those SubDyn channels for every member and node. Only the window $[\,\mathrm{int}(N_T\,\texttt{From})+1, \mathrm{int}(N_T\,\texttt{Upto})\,]$ of the N_T rows is read, and one row out of `skip`; the time step is the difference of the first two times read.

4.1.2 Turbine types

Table 4.1: Turbine types (`case_type`) and the data each one holds (`emitters/TurbineTypes/`).

`case_type`	**Structure and model**
`DTU10MWMonopile`	DTU 10 MW [1] on a monopile of outer diameter 9 m (steel 8500 kg/m^3, inner diameter 8.69 to 8.80 m below the surface). Wet nodes: $z \le 0$, duplicated nodes between members removed.
`DTU10MWFloating`	DTU 10 MW on the floating platform of the CENER DeltaWind version: three columns of 14.5 m diameter and three pontoons of 10.875×7 m, wall 0.023 m (steel 7850 kg/m^3); needs `Nmembers` ≥ 9.
`SAITEC2MWFloating`	One more floating turbine type, of 2 MW, with its own geometry, masses, drivetrain spectrum and OpenFAST reading.

4.1.3 From accelerations to forces

For every turbine (Figure 4.1):

1. The nodes and the accelerations are rotated by `WindDir` about z and the nodes are translated to `AxisPos` (emitters/WindTurbine.f90:translate_align_with_WindDir).
2. The wet nodes ($z \le 0$, without duplicates) get a mass per node: the mass of the structure plus the added mass of the displaced water, $\rho\,\pi(D/2)^2\,\Delta s$ per length Δs (`compute_mass` of the type).
3. The accelerations are taken to the frequency domain with `compute_rfft`: mean removed, Hann window, one-sided spectrum of peak amplitudes, frequencies $\Delta f, 2\Delta f, \ldots$ up to Nyquist, $\Delta f = 1/(N_t \Delta t)$, and one bin out of `skipf` (Section 13.3).
4. The force of node j is its mass times its acceleration spectrum, $\boldsymbol{F}_j(f) = (m_j + m_{\mathrm{a},j})\,\hat{\boldsymbol{a}}_j(f)$, the same mass for the three components.
5. If `filter_freqs` is set, only the bins near the drivetrain excitation are kept (below).
6. When the sources are handed to the solver, every force is multiplied by the magnitude of the radiation correction of a cylinder of the member's diameter, $|\alpha(ka)|$ (Section 13.2).

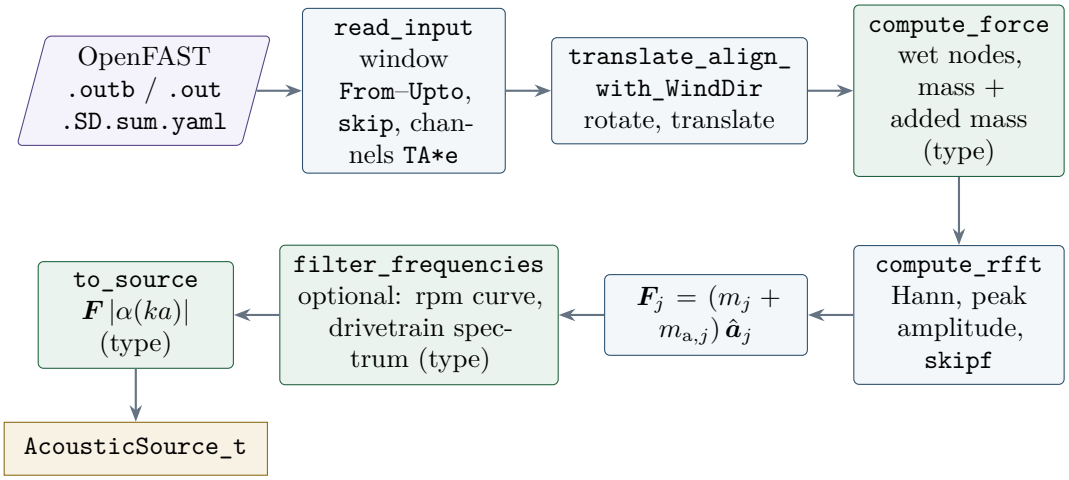


Figure 4.1: From an OpenFAST run to the dipoles of a turbine. The structural data and the optional frequency filter belong to the turbine type; everything else is shared by every type (`emitters/WindTurbine.f90`).

4.1.4 Wind speed, rpm and the drivetrain filter

The wind speed `WindSpeed` gives the rotor speed through the curve `rpm_ws.csv`, interpolated linearly (and extrapolated) by utils/CSVReader.f90:read_curve. The DTU types read it from `../wind_speed_curves_DTU_10MW/rpm_ws.csv`, a path relative to the working directory.

With `filter_freqs` the turbine keeps only the frequencies that its drivetrain excites (emitters/WindTurbine.f90:filter_frequencies):

1. its `excitation_spectrum` gives the drivetrain peaks at that rpm: for the DTU 10 MW, 13 shaft orders and 9 gear-mesh orders whose frequencies scale with the rpm, clipped to 6–9.6 rpm (emitters/DrivetrainExcitation.f90:drivetrain10MW_excitation_spectrum);
2. every gear-mesh peak, and every peak above 10 Hz, is spread into 50 lines over $f_0 \pm 4\sigma$, $\sigma = 0.02 f_0$ (`generate_timeseries_banded_sines`);

3. a bin is kept if it is below `freqs_over`, or within one bin of one of those lines (`filter_non_useful_freqs`).

The amplitudes of the excitation spectrum only select the bins; they do not weight the forces, which remain the FFT of the OpenFAST accelerations. Figure 4.2 shows the effect: the 880 bins kept out of 10 000 are the bands where the accelerations, and the radiated pressure, have their peaks.

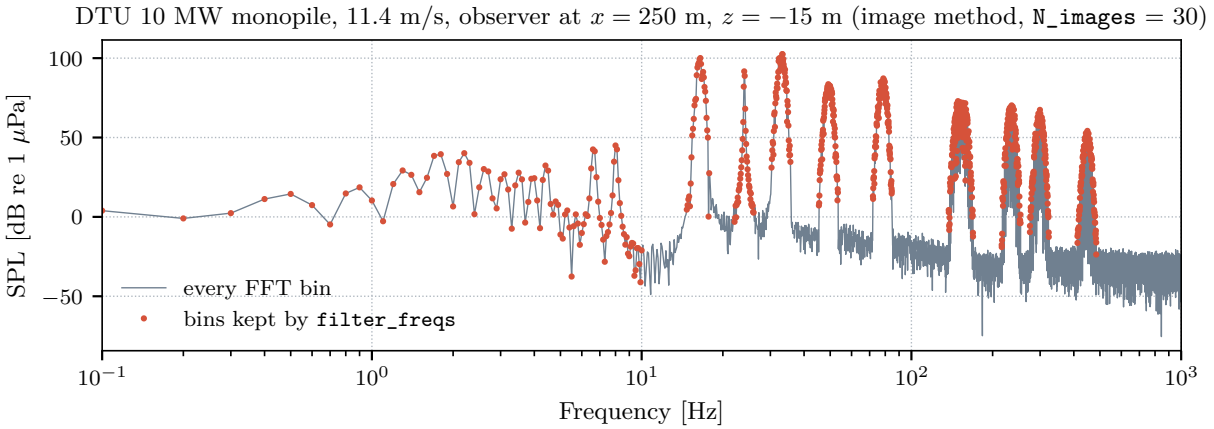


Figure 4.2: Pressure spectrum of the DTU 10 MW monopile at 11.4 m/s, 250 m from the axis at mid-depth, with every FFT bin ($\Delta f = 0.1$ Hz, 10 000 bins) and with the bins kept by `filter_freqs` (880). Look at how the kept bands coincide with the drivetrain peaks above 10 Hz, while every bin below `freqs_over` = 10 Hz is kept. Last 10 % of the OpenFAST run (`From` = 0.9), 25 wet nodes.

4.1.5 Reference points and output name

The barycentre `BariPos` (when left at $(0, 0)$, the mean (x, y) of the nodes) is the *centre* of the turbine, its axis `AxisPos` the *origin* and `WindDir` the *direction*; the steps use them as defaults (Table 6.1). The default output is `./turbine_acoustic_data/plot_mn<tag>_ws<WindSpeed>.hdf5` for a monopile and `plot_fl...` for a floating turbine, with `<tag>` the method tag (Section 7.1).

4.2 Wind farm

A farm is a list of turbines (Section 3.4.2), each computed as above, one after another, so that only one OpenFAST record is in memory at a time (emitters/WindFarm.f90:compute_forces_WindFarm). Then they are merged into one set of sources (`farm_to_source_WindFarm`):

- **Common frequency grid.** Every turbine must have the same bin width $\Delta f = 1/(N_t \Delta t)$, to a relative 10^{-9}, otherwise the run stops: their OpenFAST records must have the same length and time step. The farm grid is the union of the bins of all the turbines; a turbine that does not have a bin (e.g. because its drivetrain filter, at another wind speed, kept other bins) has a zero force there, and a WARNING says in how many bins.
- **Coherent sum.** All the dipoles go to the solver together, so the field of the farm is the coherent sum of the fields of its turbines. Two turbines made from the same OpenFAST run (the same `rootname`) are fully coherent replicas.
- **Reference points.** The centre is the mean of the barycentres, the origin the mean of the axes, and the direction the circular mean of the wind directions.
- **Steps.** Decay and sphere are never run for a farm (Table 3.9).

- **Name.** `./farm_acoustic_data/Farm_<n>_<type>..._<method tag>_ws<min>[-<max>].hdf5`, with one `<n>_<type>` per turbine type (`DTU10MN`, `DTU10FL`, `SAITEC2FL`), or `Farm_<farm_name>...` when `farm_name` is given.

4.3 FrequencySource

A `FrequencySource` is the most direct emitter: its dipoles are the input. `FrequencySource_t % init` (emitters/FrequencySource.f90:init_FrequencySource) copies the positions, builds the frequency axis (the list, the grid or the rows of the file) and the forces (the written values, the expressions of f evaluated on the final axis, or the columns of the files), and checks that the frequencies are positive and strictly increasing and that every force is finite. Its forces are not windowed (no `ENBW` dataset is written) and have no radiation correction.

Besides the line *Initialized with N sources at M frequencies*, a run prints how the axis and the forces were built when they do not come from the namelist, e.g.

```
>> [FrequencySource] Frequency axis (the linear grid from fmin to fmax): 400 frequencies from
   ↪ 2.50000E+00 ...
>> [FrequencySource] Forces evaluated from the expressions of f
```

The reference points are the mean (x, y) of the sources (centre and origin) and the direction 0°; the default output is `./source_acoustic_data/Source_Ns<N>_Nf<M><tag>.hdf5`.

4.4 TimeSource

A `TimeSource` builds the force series $F(t_n)$ of every source and component on its time axis, from the expressions of t or the columns of `force_file`, and takes them to the frequency domain with `compute_rfft`, the FFT of the turbines (emitters/TimeSource.f90:take_to_frequency_domain):

- the mean of every series is removed and a symmetric Hann window applied; the spectrum is one-sided, in peak amplitudes corrected for the coherent gain of the window, at $\Delta f, 2\Delta f, \ldots$ up to the Nyquist frequency, with $\Delta f = 1/T$, $T = N_t \Delta t$;
- the zero frequency is never kept (a constant force does not radiate);
- the bins kept are those of the band [`fmin`, `fmax`], one out of `skipf`.

The forces handed to the solver are therefore windowed: `get_enbw` returns the equivalent noise bandwidth of the Hann window, 1.5 bins, which is saved as `ENBW` and used by the plotting package to correct sums over bins (Section 6.4). A tone of amplitude A on a bin gives A on that bin and about $A/2$ on the two neighbours; half a bin off it gives $0.849A$ on the two nearest bins, with the same total power (Section 9.3). The default output is `./source_acoustic_data/Source_Ns<N>_Nt<Nt><tag>.hdf5`; the file also holds the series (`Source_time`, `Source_Force_time`).

Chapter 5

Acoustic methods

Both methods compute the pressure of the same dipoles in the same water layer, $-H \le z \le 0$; they differ in the boundaries they represent and in how. The method is chosen by `method` of `Solver.nml`; its parameters are in `AcousticMethods/<method>.nml` (Section 3.6). Chapter 14 gives the formulas as implemented.

5.1 Image method

5.1.1 Principle

The image method replaces the two boundaries by images: every source is mirrored across the surface and across the seabed, the images are mirrored again across the other plane, and so on, `N_images` times on each side (Figure 5.1). The pressure is the sum of the free-field dipole fields (1.4) of the source and of its $2N$ images,

$$\hat{p}(\boldsymbol{x}) = \sum_{j=0}^{2N} s_j \left(\boldsymbol{F}_j \cdot \hat{\boldsymbol{r}}_j\right) \left(\frac{1}{R_j} - \mathrm{i}k\right) \frac{\mathrm{e}^{\mathrm{i}kR_j}}{4\pi R_j}, \tag{5.1}$$

where the image j reached after n_u reflections across the surface and n_b across the seabed has

- its force mirrored: F_z changes sign at every reflection, F_x, F_y do not;
- the sign $s_j = (\texttt{UpperBC})^{n_u}(\texttt{LowerBC})^{n_b}$, with -1 for a pressure-release plane ($p = 0$) and $+1$ for a rigid one ($\partial p/\partial n = 0$);
- the amplitude multiplied by $a_s^{n_u} a_b^{n_b}$, with a_s = `attenuation_surface` and a_b = `attenuation_bottom`.

With $s = -1$ the mirrored dipole cancels the pressure on the plane, with $s = +1$ it cancels its normal derivative, so the boundary conditions hold exactly when the attenuations are 1 and the sum is infinite. The attenuations are constant factors per reflection, independent of the frequency and of the angle: the shipped $a_b = 0.5$ is a crude model of the energy that a real seabed transmits.

5.1.2 Parameters and cost

- `N_images`: the cost is proportional to $N_s(1+2N)$ per observer and frequency; `N_images = 0` is the free field (used by the sphere, Section 6.3.9).
- `UpperBC`, `LowerBC`: -1 (pressure release) or $+1$ (rigid). The physical choice is -1 at the surface and $+1$ at a hard seabed.
- `attenuation_surface`, `attenuation_bottom`: factors per reflection, normally between 0 and 1.

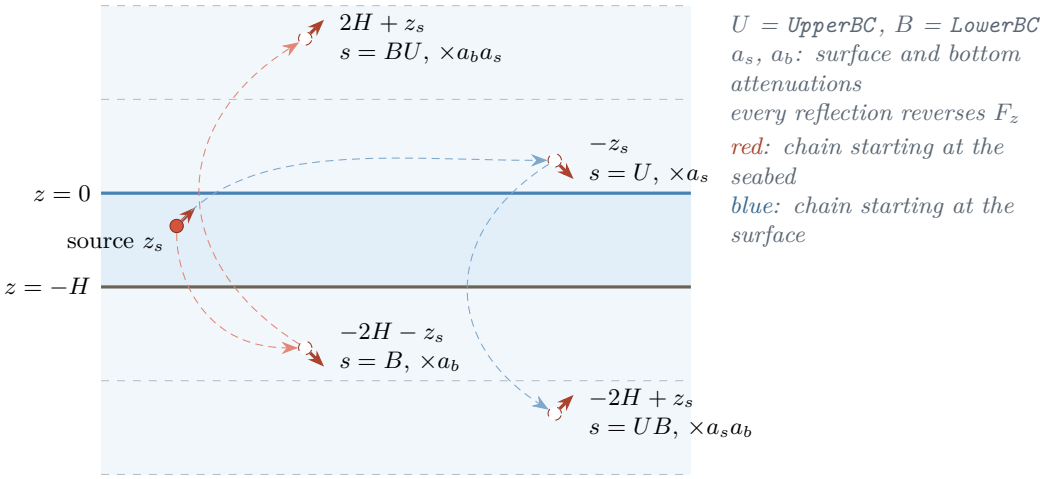


Figure 5.1: The images of a source at depth z_s for `N_images` = 2: two chains, one starting with a reflection across the seabed and one across the surface, alternate between the two planes. Each image carries the sign of the boundary conditions and the attenuations of the planes it was reflected across, and its vertical force is reversed at every reflection. The total is $1 + 2N$ dipoles per source.

- `epsilon`: the distance R_j is floored at this value, so that an observer on a source does not divide by zero; such observers are normally masked anyway (`mask_distance`, Section 6.2).

The reader does not check these values; an unusual one (e.g. a boundary code of 0) is used as given.

5.1.3 Convergence of the image sum

The images of order n are at a distance of about $2nH$, so their fields decay as $1/(nH)$ while their number grows linearly. With attenuations of 1 the sum converges slowly and oscillating: Figure 5.2 compares it with the normal modes, which solve the same ideal waveguide exactly. The difference decreases as $1/N$, and is larger at long range, where more images are needed before their distance exceeds the range: with the default `N_images = 30`, it is 2 to 12 % at 50–200 m but 13 to 71 % at 1 km (`numbers.txt` of the figures). With $a_b < 1$ the images of high order are damped geometrically, the sum converges fast, and the field is the one of a lossy seabed rather than the ideal guide (Figure 5.4).

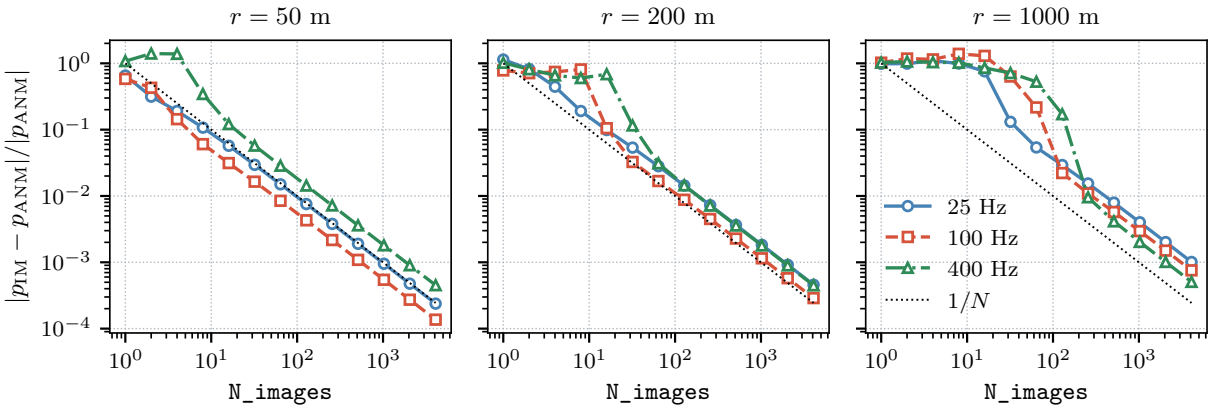


Figure 5.2: Convergence of the image method to the normal modes in the ideal 30 m waveguide (surface $p = 0$, rigid seabed, both attenuations 1): relative difference of the pressure against `N_images` for a dipole $\boldsymbol{F} = (1, 0, 1)$ N at $z = -12$ m, observers at $z = -20$ m and three ranges. Look at the $1/N$ slope and at how the convergence starts later at long range.

5.2 Analytical normal modes

5.2.1 Principle

In a layer with a pressure-release surface and a rigid seabed, the field is a sum of modes [5]. Mode m has the vertical shape

$$\Psi_m(z) = \sqrt{\frac{2\rho}{H}}\,\sin(k_{zm}z), \qquad k_{zm} = \frac{(2m-1)\pi}{2H}, \tag{5.2}$$

which vanishes at $z = 0$ and has zero slope at $z = -H$ (Figure 5.3a), and travels horizontally with the wavenumber $k_{rm} = \sqrt{k^2 - k_{zm}^2}$. A mode propagates when $k > k_{zm}$, that is above its cut-on frequency

$$f_m = \frac{(2m-1)\,c}{4H}, \tag{5.3}$$

and is evanescent below it (k_{rm} imaginary, decaying as $\mathrm{e}^{-|k_{rm}|r}$). At the frequency f there are $\lfloor 2Hf/c + 1/2 \rfloor$ propagating modes (Figure 5.3b): in 30 m of water with $c = 1500$ m/s the cut-on frequencies are 12.5, 37.5, 62.5 Hz…, in steps of $c/(2H) = 25$ Hz. The pressure of a dipole is (Section 14.3)

$$\hat{p} = \frac{\mathrm{i}}{4\rho}\sum_{m=1}^{M}\Psi_m(z)\left[F_z\Psi'_m(z_s)\,H_0^{(1)}(k_{rm}r) + \Psi_m(z_s)\,k_{rm}\,H_1^{(1)}(k_{rm}r)\,\frac{F_x\,\Delta x + F_y\,\Delta y}{r}\right], \tag{5.4}$$

with r the horizontal distance and $H_n^{(1)}$ the Hankel functions of the first kind.

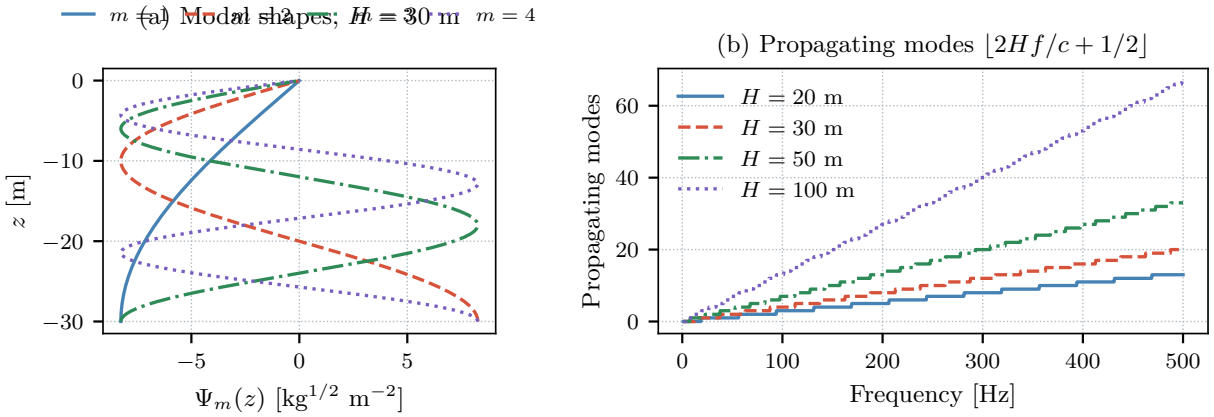


Figure 5.3: Analytical normal modes. (a) The first four modal shapes (5.2) in 30 m of water, $\rho = 1025$ kg/m^3: zero at the surface, zero slope at the seabed. (b) Number of propagating modes against frequency for four depths: the automatic `Nmodes` is this number at the highest frequency plus 5.

5.2.2 Parameters

- `Nmodes`: −999 (automatic) takes the modes that propagate at the highest frequency of the sources plus 5 evanescent ones; the sum also stops, frequency by frequency, at the first evanescent mode that has decayed by e^{-40} relative to the dominant one. In Section 9.2 the automatic 21 modes and 400 modes give identical results. The number of modes is part of the file name (`_ANM21`).
- `epsilon`: the horizontal distance is floored at this value (the Hankel functions are singular at $r = 0$).

The boundaries are fixed: the method has no attenuation and no rigid surface. At an exact cut-on ($k = k_{zm}$, where $H_n^{(1)}(0)$ is singular) the mode is evaluated a relative distance 10^{-6} from it.

5.3 Choosing a method

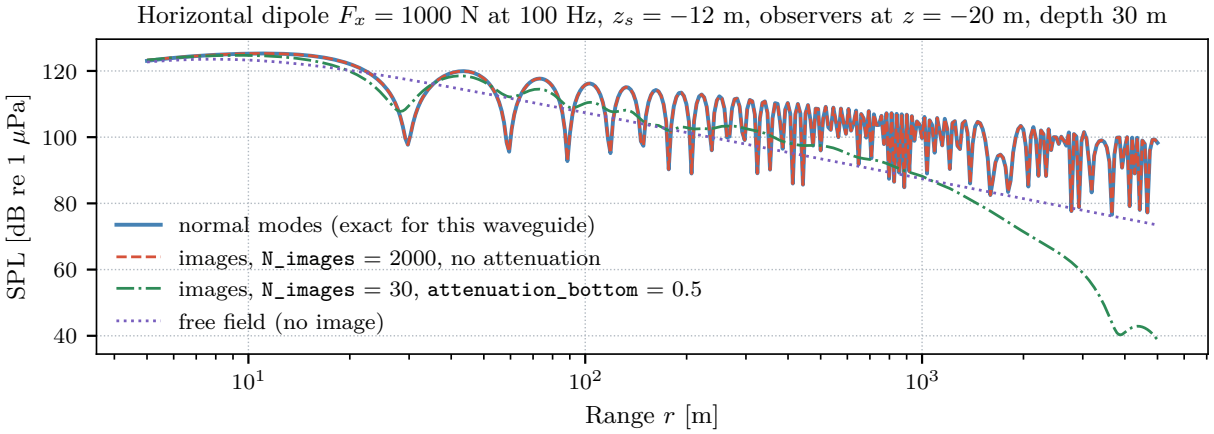


Figure 5.4: Level against range for a horizontal dipole of 1000 N at 100 Hz in 30 m of water. Normal modes and the image method with ideal boundaries and 2000 images agree; the shipped image method (30 images, bottom attenuation 0.5) loses the energy of the bottom reflections and decays like a lossy guide; the free field decays as $1/r$ with no interference. Look at the modal interference pattern beyond 100 m.

Table 5.1: When to use each method.

	Image method	**Analytical normal modes**
Boundaries	$p = 0$ or rigid on each plane, attenuation per reflection	$p = 0$ surface, rigid seabed only
Accuracy	exact for the free field (`N_images` = 0); for the ideal guide, error $\sim 1/N$, worse at long range	exact for the ideal guide with enough modes
Cost	$\propto N_s(1 + 2N)$ per observer and frequency	$\propto N_s M$, M grows with f; Hankel functions per mode
Sphere step	yes (free field)	no (skipped)
Typical use	near field, a lossy seabed modelled by `attenuation_bottom`, free-field source levels	long ranges over a hard seabed, reference solution

Chapter 6

Post-processing steps

A *step* builds a set of observer points, computes the pressure there at every frequency of the sources and writes it to the HDF5 output with the geometry (`solvers/AcousticSolver_steps.f90`). The steps are switched on in `&do_config` and configured in their own groups (Section 3.5); they run in this fixed order:

`spectrums`, `polar`, `decay`, `line`, `sphere`, `sliceXY`, `sliceXZ`, `sliceVertical`, `cylinder`.

The header `[k/N]` of each step is counted before any step runs, so a step that skips itself (the sphere with the normal modes, a vertical slice equal to `sliceXZ`) still has its number.

6.1 Reference points

The automatic values of the steps are built from three references that the emitter provides (Table 6.1) and from the seabed $z_{\text{bot}} = -H$; mid-depth is $z_{\text{bot}}/2$.

Table 6.1: Reference points of each emitter, used by the automatic values of the steps.

Emitter	**centre**	**origin**	**direction**
Wind turbine	barycentre `BariPos`	axis `AxisPos`	`WindDir`
Wind farm	mean of the barycentres	mean of the axes	circular mean of the `WindDir`
Point sources	mean (x, y) of the sources	the same	$0°$ $(+x)$
Used by	polar, cylinder, sliceXY limits, sliceXZ x limits	decay start, sliceXZ y, sliceVertical centre	decay

6.2 Masked observers

Every step computes the pressure through solvers/AcousticSolver.f90:compute_pressure_masked, which leaves out two kinds of observers and gives them the value NaN at every frequency:

- observers closer than `mask_distance` to a source (3-D distance to the real sources, not to the images), where the point-dipole field is meaningless: for a turbine they lie inside the structure;
- observers outside the water column, $z > 0$ or $z < z_{\text{bot}}$ (with a tolerance of 10^{-9} m, so points on the surface and on the seabed are kept).

For each criterion that masks something, one line is printed:

```
WARNING [SliceVert] 4 of 900 observers closer than  1.0 m to a source node: masked (NaN)
WARNING [Sphere] 4464 of 5760 observers outside the water column: masked (NaN)
```

The grids keep their shape, so masked points appear as holes in the maps. The plotting package treats NaN as holes and leaves them out of the sums that it reports (Section 6.4).

6.3 The steps

Figure 6.1 shows the geometry of each step. Below, N_f is the number of frequencies; the datasets are given with their Fortran shapes (Section 7.2) and every step also writes `Freqs`.

6.3.1 Spectra at points: `spectrums`

A list of points (`observers_x`, `observers_y`, `observers_z`) or, by default, the three points $(10, 0)$, $(250, 0)$, $(500, 0)$ m at mid-depth, in absolute coordinates whatever the emitter. A point above the surface prints a WARNING and is masked. *Datasets*: `P_spectrums` (N_f, N_{obs}), `Obs_spectrums` $(N_{\text{obs}}, 3)$.

6.3.2 Horizontal circle: `polar`

n points $\boldsymbol{x}_i = \boldsymbol{x}_c + r(\cos\theta_i, \sin\theta_i)$ at depth z, with $\theta_i = (i-1)\,360°/n$, anticlockwise from $+x$ (0° included, 360° excluded). Defaults: $r = 500$ m, $n = 72$, the centre, mid-depth. *Datasets*: `P_polar` (N_f, n), `R_polar`, `Z_polar`, `Theta_deg_polar` (n) [deg], `Obs_polar` $(n, 3)$, `Center_polar` (2).

6.3.3 Vertical cylinder: `cylinder`

n azimuths as in the polar step times n_z levels $z_j = z_{\text{bot}} + (j-1)\Delta z$ from the seabed to the surface, both included ($\Delta z = H/(n_z - 1)$). Defaults: $r = 500$ m, $n = 72$, $n_z = 20$, the centre. The element of area $\Delta A = r\,\Delta z\,2\pi/n$ is that of an inner level; a surface integral weights the first and last levels by 1/2. *Datasets*: `P_cylinder` (N_f, n, n_z), `R_cylinder`, `Z_cylinder` (n_z), `Theta_deg_cylinder` (n), `Obs_cylinder` $(n, n_z, 3)$, `Center_cylinder`, `dA_cylinder`.

6.3.4 Decay along the direction: `decay`

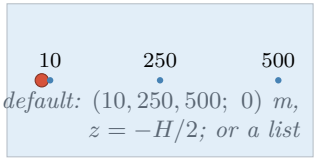
n points $\boldsymbol{x}_i = \boldsymbol{x}_o + d_i(\cos\beta, \sin\beta)$ from the origin $\boldsymbol{x}_o$ along the direction β of the emitter, at depth z; the distances d_i go from d_1 to d_2 in logarithmic (default) or linear steps. A start $d_1 \leq 0$ is moved to 1 m with a WARNING. Defaults: $d = [10, 500]$ m, $n = 200$, mid-depth. Not available for a wind farm. *Datasets*: `P_decay` (N_f, n), `Z_decay`, `Distance_decay` (2), `Distances_decay` (n), `Obs_decay` $(n, 3)$, `Logspace_decay` (0 or 1).

6.3.5 Line between two points: `line`

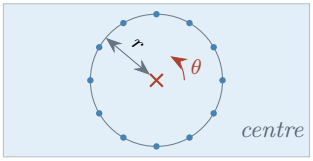
n points from $\boldsymbol{p}_1$ to $\boldsymbol{p}_2$, at distances $d_i = (i-1)L/(n-1)$ from $\boldsymbol{p}_1$, or logarithmic from $10^{-6}L$ to L. Defaults: the vertical from $(100, 0, 0)$ down to the seabed, $n = \lfloor L/5 \rfloor$ for $L > 200$ m and $\max(2, \lfloor L \rfloor)$ otherwise. Coincident points stop the run. *Datasets*: `P_line` (N_f, n), `P1_line`, `P2_line` (3), `Obs_line` $(n, 3)$, `Distance_line` (L), `Distances_line` (n), `Logspace_line`.

6.3.6 Horizontal plane: `sliceXY`

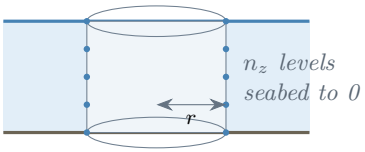
A regular grid of $n_x \times n_y$ points at depth z (a positive z is changed to $-z$ with a message), limits included; x varies fastest. Defaults: 26×26 points over the centre ± 500 m, mid-depth. The centre `center_xy` is always the middle of the limits. *Datasets*: `P_slicexy` (N_f, n_x, n_y), `Obs_slicexy` $(n_x, n_y, 3)$, `X_slicexy`, `Y_slicexy`, `Z_slicexy`, `Center_slicexy`, `Nx_slicexy`, `Ny_slicexy`.



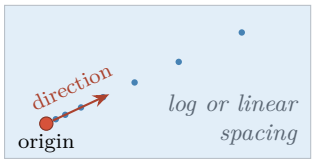
(a) `spectrums`: observer points



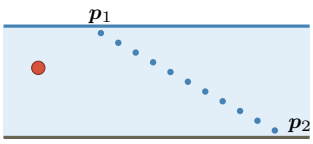
(b) `polar`: circle at depth z



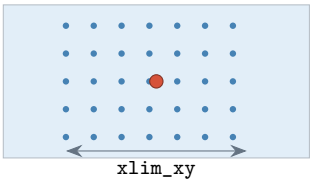
(c) `cylinder`: radius r, full depth



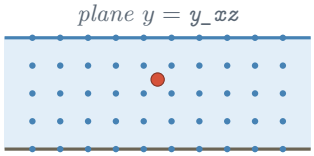
(d) `decay`: from the origin along the direction



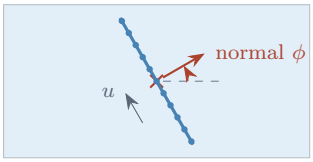
(e) `line`: from $\boldsymbol{p}_1$ to $\boldsymbol{p}_2$ (any 3-D line)



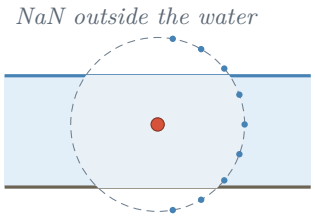
(f) `sliceXY`: horizontal plane at z



(g) `sliceXZ`: vertical plane $y =$ constant



(h) `sliceVertical`: plane of normal $(\cos\phi, \sin\phi)$, top view



(i) `sphere` (image method): free field

Figure 6.1: Geometry of the steps, in top view (circles, lines and horizontal planes) or side view (water column between the surface, blue, and the seabed, brown). Blue dots: observers; red: the sources or their reference point; red cross: the centre of the circle or plane. Every step can be moved and resized with its group of the computation setup; the defaults are those of Section 3.5.

6.3.7 Vertical plane y = constant: `sliceXZ`

A grid of $n_x \times n_z$ points on the plane $y = y_{xz}$; x varies fastest and z goes from `zlim_xz(1)` to `zlim_xz(2)`. Defaults: 26×26 points, x over the centre ± 500 m, y of the origin, z from 0 down to the seabed. *Datasets*: `P_slicexz` (N_f, n_x, n_z), `Obs_slicexz` $(n_x, n_z, 3)$, `X_slicexz`, `Z_slicexz`, `Y_slicexz`, `Nx_slicexz`, `Nz_slicexz`.

6.3.8 Vertical plane at any azimuth: `sliceVertical`

The plane through the centre $\boldsymbol{x}_c$ (default: the origin) with the horizontal normal $(\cos\phi, \sin\phi)$, ϕ = `azimuth_sv`, spanned by $\boldsymbol{u} = (-\sin\phi, \cos\phi)$ and the vertical: points $\boldsymbol{x}_c + u_i\boldsymbol{u}$ at depths z_j, with z varying fastest. With $\phi = 0°$ the plane is YZ and u runs along $+y$; with $\phi = -90°$ it is the plane XZ with u along $+x$. The automatic width projects every source on $\boldsymbol{u}$ and adds `margin_sv` on both sides; the depth range is clipped to the water column. The step is skipped, with a message, when `sliceXZ` also runs on the same plane ($\phi = \pm 90°$ within 10^{-3} degrees and the same y within 1 mm). *Datasets*: `P_sliceVertical` (N_f, n_u, n_z), `Coords_sliceV` $(n_u, n_z, 3)$, `U_sliceV`, `Z_sliceV`, `Azimuth_sliceV`, `Center_sliceV`.

6.3.9 Free-field sphere: `sphere`

Only the image method runs it (the normal modes print *skipped*). The pressure is computed in free field: the image system is rebuilt with `N_images = 0` for this step and restored afterwards (solvers/methods/ImageMethod.f90:run_sphere). The sphere of radius r about the centre $\boldsymbol{x}_c$ is divided into n_z bands of equal height and n_θ sectors, and the observers are the centres of the cells: by Archimedes' theorem every cell has the same area $\Delta A = 4\pi r^2/(n_\theta n_z)$. Defaults: $n_\theta = 144$, $n_z = 40$, the centre of the box that holds every source, and the radius $r = a_{\max} + 3/k_{\text{far}}$, with $a_{\max}$ the largest distance from a source to the centre and $k_{\text{far}} = 2\pi f_{\text{far}}/c$, f_{far} = `f_farfield_sphere` = 10 Hz (71.6 m for one source in water with $c = 1500$ m/s). A given radius is used as it is. *Datasets*: `P_sphere` (N_f, n_θ, n_z), `Obs_sphere` $(n_\theta, n_z, 3)$, `Theta_sphere` (n_θ) [deg], `Z_sphere` (n_z), `R_sphere`, `Center_sphere` (3), `dA_sphere`.

> **Pitfall** The sphere is computed in free field but its observers are still masked outside the water column: a sphere of 71.6 m about a source in 30 m of water has most of its points above the surface or below the seabed, and they are NaN (4464 of the 5760 points of the shipped `FrequencySource` case). The free-field power is then integrated over the wet part only (Section 6.4); for a source level, use a deep enough `depth_base` or a smaller radius.

6.4 Metrics and their biases

The solver writes complex pressures; the levels are computed by the plotting package (`plotting/utils/AcousticUtils.py`, `plotting/plots/metrics.py`, `plotting/plots/specific/sphere.py`). Every quantity below is the one implemented.

SPL of each bin, (1.2): $L = 20\log_{10}(\max(|\hat{p}|/\sqrt{2}, p_{\text{ref}})/p_{\text{ref}})$.

The ENBW. The forces of turbines, farms and `TimeSource` come from a Hann-windowed FFT scaled for amplitude: a tone on a bin gets its full amplitude, but a broadband signal is spread so that the sum of $|\hat{F}|^2$ over the bins is ENBW = 1.5 [3] times its power (Section 9.3). The plotting package reads the dataset `ENBW` (1.5) when it exists; a file with `Method` but no `ENBW`, i.e. forces without a window (`FrequencySource`), uses 1.0; an older file with neither uses 1.5.

PSD of each bin: $L - 10\log_{10}(\mathrm{ENBW}\,\Delta f)$, with Δf the bin width of the case (Section 8.2). It needs a bin width: a `FrequencySource` with frequencies that are not uniformly spaced has none, and its PSD plots are skipped with a message.

OASPL : $10\log_{10}\left(\sum_k 10^{L_k/10} + 10^{-6}\right) - 10\log_{10}\mathrm{ENBW}$, the energetic sum over the bins, corrected by the ENBW. A NaN observer stays NaN.

1/3 octave bands : centres $f_c = 1000 \cdot 2^{n/3}$ Hz between the lowest and the highest frequency, edges $f_c 2^{\pm 1/6}$, a bin belongs to $[f_{\mathrm{low}}, f_{\mathrm{high}})$; a band with no bin takes the level of the nearest bin, and the bins beyond the outer edges are left out. The PSD bands divide the band power by the ENBW and the band width; the SPL bands are the plain energetic sum of the bin levels.

> **Note** For a Hann-windowed spectrum the SPL band levels therefore include the factor ENBW (+1.76 dB) that the OASPL, the PSD bands and the equivalent source of `compute_point_source` remove.

Absorption (option `absorption` of the plots, on by default): α [dB/km] $= 0.0033 + 0.11 f^2/(1+f^2) + 44 f^2/(4100+f^2) + 0.0003 f^2$ with f in kHz (Thorp's formula [7]), times a distance that depends on the plot (the horizontal distance to the origin for the spectra, the radius of the circle, the distance along the line from $\boldsymbol{p}_1, \ldots$). It is switched off, with a message, for cases without a single origin (several point sources, farms).

Radiated power through a closed surface, from the far-field intensity $|\hat{p}|^2/(2\rho c)$:

$$W = \sum_{\text{cells}} \frac{|\hat{p}|^2}{2\rho c}\,\Delta A, \tag{6.1}$$

over the sphere (equal cells, NaN cells left out and their fraction reported) or the cylinder (trapezoid rule in θ and z, NaN cells counted as zero). The equivalent source level is SL $= 10\log_{10}(\rho c W/(4\pi p_{\mathrm{ref}}^2))$ dB re 1 µPa^{2}m^2, *free field* from the sphere (the source itself) and *in situ* from the cylinder (the power injected into the actual water column), in `compute_point_source`, which divides the sums over bins by the ENBW. `compute_sphere_metrics` and `compute_cylinder_metrics` print the same power without that division.

Known biases of these powers, which the plotting package reports as warnings (`compute_point_source`):

- **Near field**: $|\hat{p}|^2/(2\rho c)$ is the intensity of a plane wave; on a sphere with $kr < 3$ the dipole power is overestimated by up to $10\log_{10}(1 + 1/(kr)^2)$.
- **Finite size**: a sphere that hugs an extended source overestimates the power by a geometric factor $10\log_{10}\left(\frac{1}{2a}\ln\frac{1+a}{1-a}\right)$, $a = a_{\max}/r$; keep $r \gtrsim 3a_{\max}$ to stay below about 0.2 dB.
- **Oblique modes on the cylinder**: in a waveguide only the radial intensity carries power, so the cylinder overestimates by about 0.5 to 4.7 dB (quantified on one structure of the project); exact values would need the active intensity $\frac{1}{2}\mathrm{Re}\{\hat{p}\,\hat{v}_r^*\}$.
- **Masked cells**: the power through the masked part of the surface is missing.

Chapter 7

Outputs

A run writes two files with the same name: an HDF5 file with every number, and a text summary `.dat` with the input that defines the case and the parameters actually used by every step.

7.1 File names

With `save_path = 'None'` (the default) the name is built from labels of the emitter around the tag of the method (solvers/AcousticSolver_output.f90:resolve_save_path):

```
<out_dir><name_tag><method tag><name_suffix>[_<extra_label>].hdf5
```

Table 7.1: Parts of the default output name.

Emitter	out_dir	name_tag	name_suffix
Wind turbine	./turbine_acoustic_data/	plot_mn or plot_fl	_ws<WindSpeed>
Wind farm	./farm_acoustic_data/	Farm_<n>_<type>... or Farm_<farm_name>	_ws<min> or _ws<min>-<max>
FrequencySource	./source_acoustic_data/	Source_Ns<N> _Nf<Nfreqs>	–
TimeSource	./source_acoustic_data/	Source_Ns<N> _Nt<Nt>	–

The method tag is `_IM<N_images>` or `_ANM<Nmodes>`, and `extra_label` adds `_<extra_label>` unless it is `'None'` or empty. Examples: `plot_mn_IM30_ws11.4.hdf5`, `Farm_2_DTU10MN_ANM24_ws8.0-11.4.hdf5`, `Source_Ns1_Nf3_IM30_R0.5.hdf5`. With any other `save_path` the file is `<save_path>[_<extra_label>].hdf5`. The folder is created if needed; the paths are relative to the working directory.

> **Pitfall** Both floating turbine types are called `plot_fl`: two floating types run at the same wind speed with the same method and label write the same file. Use `extra_label` to tell them apart.

7.2 The HDF5 file

All datasets are at the root of the file (Figure 7.1); Appendix D lists every one with its shape. The conventions are:

- arrays are written with their Fortran shape, e.g. `P_slicexy` (N_f, n_x, n_y); a reader in a row-major language (`h5py`, C) sees the axes reversed, (n_y, n_x, N_f);
- complex arrays are stored as real arrays with an extra first Fortran axis of length 2 (real, imaginary): `P_polar` $(2, N_f, n)$ in Fortran is $(n, N_f, 2)$ in `h5py`;
- scalars are datasets of one element; strings are fixed-length byte strings;
- a dataset written again (e.g. `Freqs`, which every step writes) replaces the previous one.

A minimal reader:

```
import h5py, numpy as np
with h5py.File("source_acoustic_data/Source_Ns1_Nf3_IM30_R0.5.hdf5") as f:
    a = f["P_polar"][()]                        # (n, Nf, 2) in h5py
    P = np.transpose(a[..., 0] + 1j * a[..., 1])   # (Nf, n): the Fortran shape
    freqs = f["Freqs"][()]
    theta = f["Theta_deg_polar"][()]
```

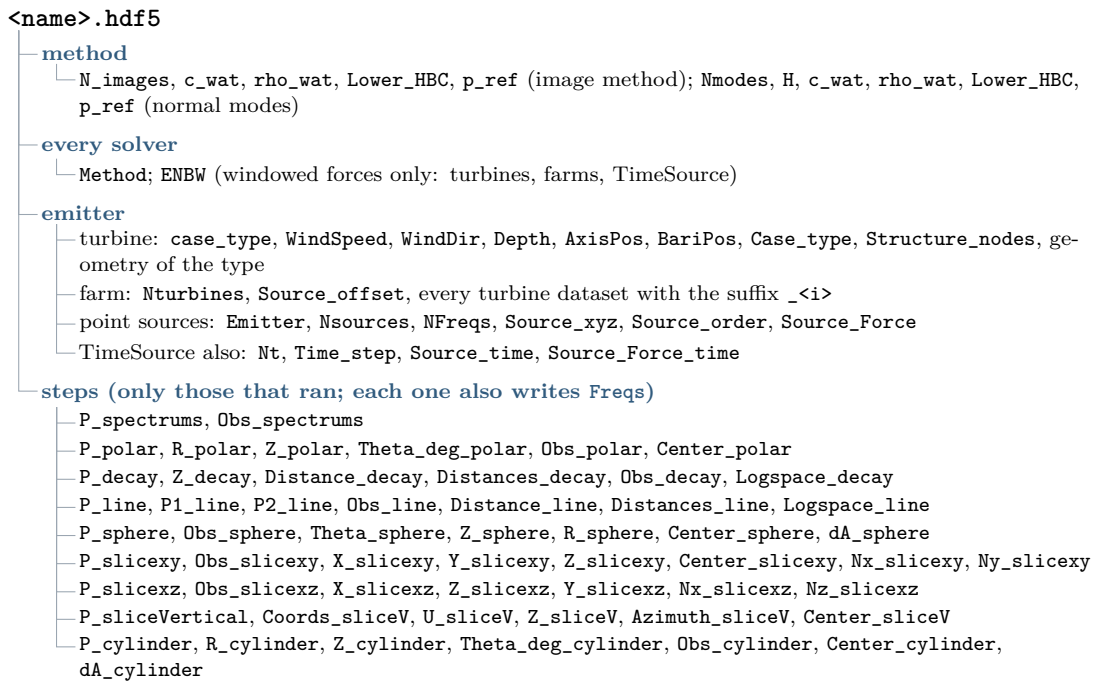


Figure 7.1: Content of an output file, grouped by the routine that writes it. Only the datasets of the emitter and of the steps that ran are present; every step also writes `Freqs`.

7.3 The `.dat` summary

The text file next to the HDF5 file has, between separators of 70 characters:

1. a title and the name of the HDF5 file;
2. the block of the emitter (`write_txt` of the emitter): for a turbine its type, wind, position and specific parameters; for a farm a table of the turbines and the block of each; for the point sources the positions and the forces (for a `FrequencySource` written by hand, the full table of complex forces; otherwise, or when it has more than 200 frequencies or 20 sources, checksums at full precision of the force table; for a `TimeSource`, the time axis, the expressions or file and checksums of the series);

3. the `ENVIRONMENT` block and the `SOLVER` block of the method;
4. `POST-PROCESSING STEPS EXECUTED`: one block per step that ran, with the values actually used, the automatic ones resolved;
5. a final separator.

Items 1 to 3 are the *header*, which `update_existing` compares. Listing 7.1 is the summary of the `FrequencySource` case of `input_files/` with every step.

Listing 7.1: The `.dat` summary of the shipped `FrequencySource` case with every step (`docs/figures/data/shipped_fs.dat`).

```
======================================================================
OFF_COUSTICS SIMULATION SUMMARY
======================================================================
HDF5 file: ./source_acoustic_data/Source_Ns1_Nf3_IM30_R0.5.hdf5

----------------------------------------------------------------------
FREQUENCY-DOMAIN SOURCES
----------------------------------------------------------------------
N sources   : 1
N freqs     : 3
Multipole   : dipole

   #         x [m]         y [m]         z [m]
   1      0.000000      0.000000    -10.000000

Force [N]: peak complex amplitude, time factor exp(-i*w*t)
   f   s   Freq [Hz]          Re(Fx)          Im(Fx)          Re(Fy)          Im(Fy)          Re(Fz)
     ↪       Im(Fz)
   1   1     15.0000  1.00000000E+01  0.00000000E+00  0.00000000E+00  0.00000000E+00  0.00000000E+00
     ↪ 0.00000000E+00
   2   1     50.0000  3.00000000E+02  0.00000000E+00  0.00000000E+00  0.00000000E+00  0.00000000E+00
     ↪ 0.00000000E+00
   3   1    100.0000  1.00000000E+02  0.00000000E+00  0.00000000E+00  0.00000000E+00  0.00000000E+00
     ↪ 0.00000000E+00

----------------------------------------------------------------------
ENVIRONMENT
----------------------------------------------------------------------
Speed of sound   :  1500.00 m/s
Density          :  1025.00 kg/m^3
Reference press. :   1.00E-06 Pa
Depth            :    30.00 m

----------------------------------------------------------------------
SOLVER: Images Method
----------------------------------------------------------------------
N images     : 30
Lower HBC    :   -30.00 m
Upper atten. :   1.0000
Lower atten. :   0.5000
Block size   : 128
In cluster   : T

----------------------------------------------------------------------
POST-PROCESSING STEPS EXECUTED
----------------------------------------------------------------------

Spectrums:
   n_observers : 3
   obs 1 (x,y,z) :    10.00,     0.00,   -15.00 m
   obs 2 (x,y,z) :   250.00,     0.00,   -15.00 m
   obs 3 (x,y,z) :   500.00,     0.00,   -15.00 m

Polar:
   r_polar      :   500.00 m
   z_polar      :   -15.00 m
   n_polar      : 72
   center_polar :      0.000     0.000
```

The post-processing block goes on with one block per step, down to the last one:

```
Sphere:
   r_sphere       :    71.62 m
   n_theta_sphere : 144
   nz_sphere      : 40
   f_farfield     :    10.00 Hz
   center_sphere  :      0.000     0.000   -10.000

=========================================================================
```

7.4 Updating an existing output

With `update_existing` = `.true.`, an existing output of the same case is completed instead of rebuilt: only the steps run now are rewritten, and the others are kept (Figure 7.2). This is how to add a step, or to recompute one with other parameters, without recomputing the rest:

1. `resolve_save_path` deletes any old `.dat.prev`; if the HDF5 file exists it is kept and its `.dat` renamed `.dat.prev` (with `.false.` the file is deleted);
2. `check_update`, before anything is written, keeps the file only if it holds the same case: the same `Freqs` (same number, relative difference below 10^{-9}) and the same `.dat` header, line by line (the lines `Block size` and `In cluster` excepted, at most 256 characters per line); otherwise it prints a WARNING with the reason, e.g. *different setting: "..." -> "..."*, and deletes the file and its `.dat.prev`;
3. the steps run now overwrite their datasets; the datasets of the other steps stay;
4. `update_parameters` writes the blocks of the steps run now and copies the blocks of the other steps from `.dat.prev`, marked `(previous run)`, then deletes `.dat.prev`.

The header lists the inputs of the emitter (the forces included, or their checksums), the environment and the method, so changing them rebuilds the file, with the exceptions of the pitfall below. `Method` and `ENBW` are not in the `.dat` and take no part in the comparison.

> **Pitfall** Some settings that change the results are not in the header, or not at enough precision, and an update does not see them: `UpperBC`, `LowerBC` and `epsilon` of the image method, `mask_distance`, and changes of the speed of sound, the density or the depth below 0.005 (they are printed with two decimals). The file then mixes steps computed with the old and the new values (checked in Section 16.4). After changing any of them, delete the output or set `update_existing` = `.false.`.

> **Note** The force table of a `FrequencySource` written by hand has nine significant digits: a change of a force beyond them is not seen (the frequencies are also compared directly, to a relative 10^{-9}). Forces given by files, grids or expressions are also compared through checksums at full precision.

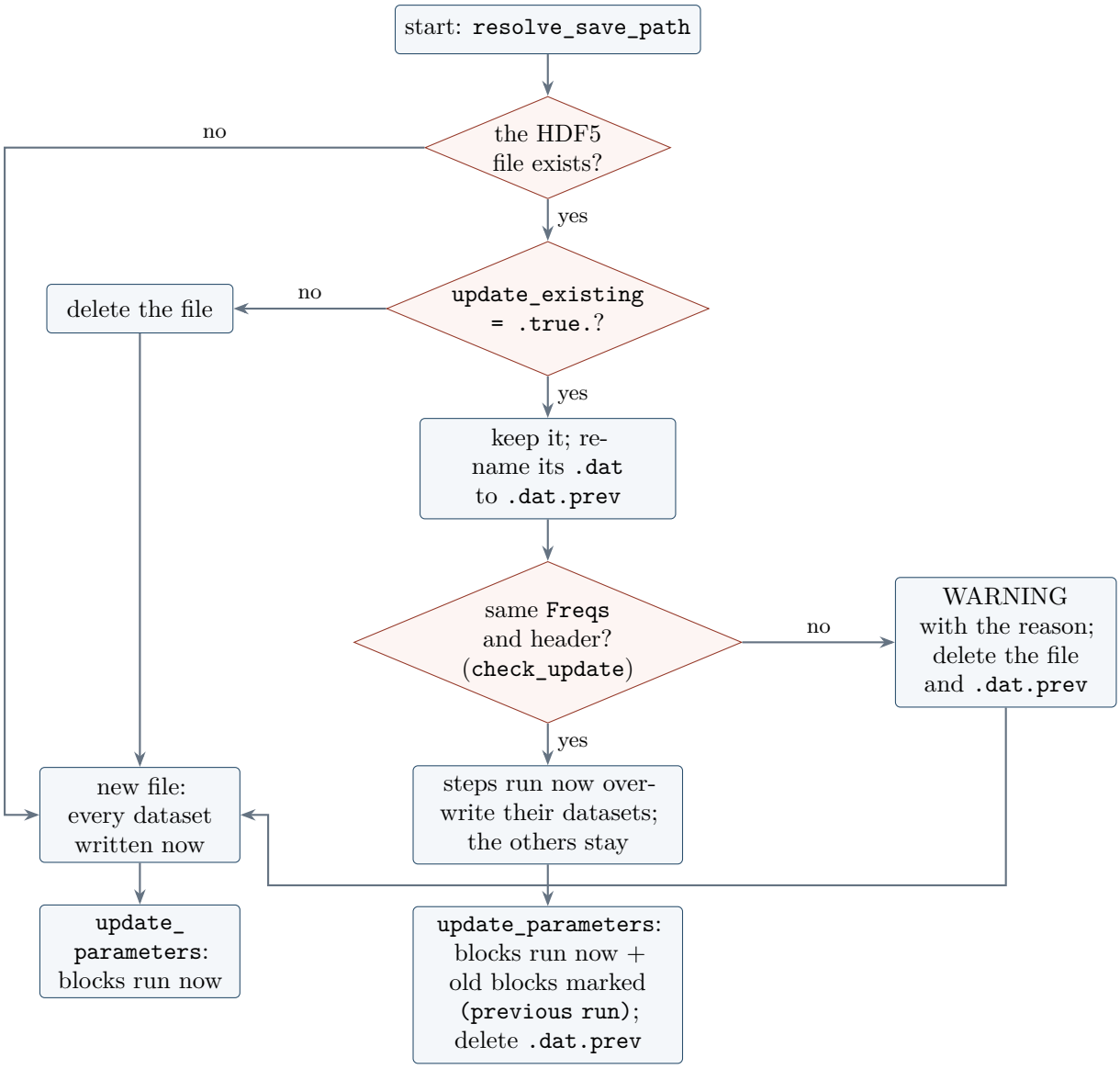


Figure 7.2: What happens to an existing output file at the start of a run (`resolve_save_path`, `check_update`) and at its end (`update_parameters`).

Chapter 8

The plotting package

The Python package `FORTRAN/plotting` reads the HDF5 outputs and draws spectra, maps and source levels. It has three entry scripts, one per kind of emitter:

Script	Reads	Default folder
`plotting/turbine.py`	a single wind turbine	`FORTRAN/turbine_acoustic_data/`
`plotting/farm.py`	a wind farm	`FORTRAN/farm_acoustic_data/`
`plotting/source.py`	`FrequencySource` and `TimeSource`	`FORTRAN/source_acoustic_data/`

The default folders are absolute paths next to the package, whatever the working directory. A file of another emitter is rejected with the name of the script that reads it. The package needs numpy, h5py and matplotlib, and LaTeX with `dvipng` for the text of the figures (without them it falls back to matplotlib's mathtext with a warning).

8.1 Command line

```
python plotting/source.py                                    # the newest file of the
    ↪ default folder
python plotting/source.py Source_Ns1_Nf3_IM30_R0.5           # one file: single case
python plotting/turbine.py a.hdf5 b.hdf5                     # several files:
    ↪ comparison
python plotting/turbine.py --list                            # the files of the folder
python plotting/turbine.py plot_mn_IM30_ws11.4 --ws          # every wind speed of
    ↪ that case
python plotting/farm.py Farm_2_DTU10MN_IM30_ws8.0-11.4 --save --no-show --format
    ↪ pdf png
```

Choosing the plots. The plots are the calls listed in `main()` of each script, one list for a single case and one for a comparison: edit them (comment out a call, change its options) to choose what is drawn. A plot of a step that is not in the file is skipped with a message, and so is a plot that does not apply (a PSD without a bin width, the sphere of a farm).

8.2 The case

Each file becomes a *case*, a dictionary with the pressure of every step and a few derived keys (`plotting/wrappers/case.py`). Two of them decide which levels make sense:

Table 8.1: Options of the entry scripts (`plotting/plot_utils/cli.py`; `--ws` only in `turbine.py`).

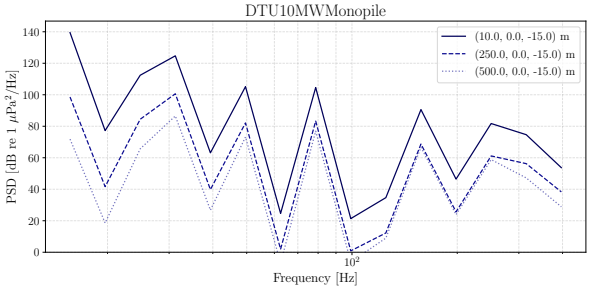
Option	Effect
`files`	One file: single case; several: comparison; none: the newest `.hdf5` of `--data_dir`. A name is tried as given, then in `--data_dir`, then with `.hdf5` added
`--data_dir DIR`	Folder of the files (default: the folder of the emitter)
`--list`	Lists the files of the folder, newest first, with their emitter, method, wind speed or sources, depth and the steps they hold, and exits
`--save`	Saves every figure in `--output_dir/<case>/<figure>.<format>`, or `comparison_<a>_vs_<b>...` for a comparison
`--output_dir DIR`	Folder of the saved figures (default `FORTRAN/figures/`)
`--format F ...`	`png` (default), `pdf`, `svg`, `eps`; several at once
`--no-show`	Does not open the windows (e.g. on a cluster, with `--save`)
`--verbose`	Lists the datasets of every file read
`--ws`	Line plots (spectrum, polar, decay, line) draw every wind speed of the case: the files of the same folder whose names differ only in `ws<speed>`

- `bin_width`, the Δf of the PSD: the smallest spacing of the frequencies for turbines and farms (an FFT grid, possibly with gaps), $1/(N_t \Delta t)$ for a `TimeSource`, and for a `FrequencySource` the spacing of its frequencies if it is uniform (relative tolerance 10^{-6}), otherwise none: then the PSD, the 1/3 octave bands and `compute_point_source` are skipped;
- `ENBW`: the dataset `ENBW` if present (1.5, Hann); 1.0 for a file with `Method` and without `ENBW` (forces without a window); 1.5 for older files with neither.

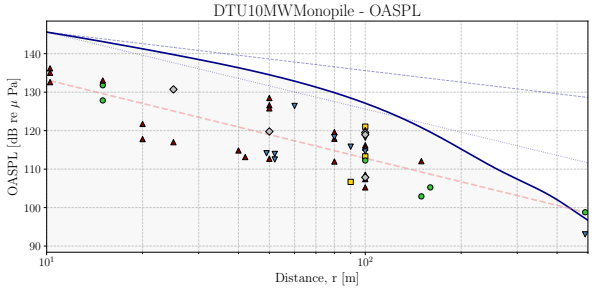
The `origin` (the axis of a turbine, the source if there is only one) is the reference of the distances, and of the absorption; a farm or several sources have none, and the absorption is then switched off with a message.

8.3 The plots

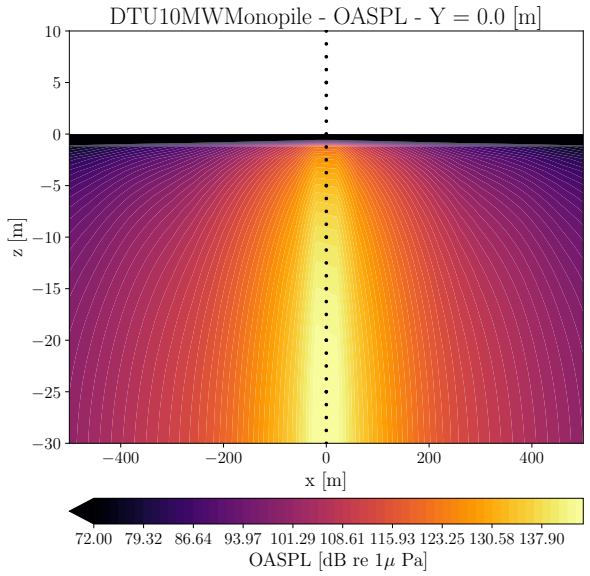
The figures below were made by running the scripts with `--save --no-show --format pdf` on the runs of `make_figures.py` (`fig_gallery`); their names are the ones the scripts give.



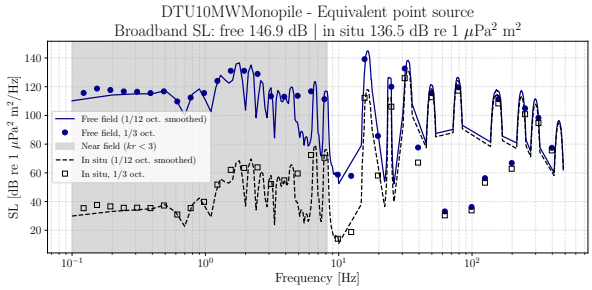
(a) `spectrum_PSD_octave`: PSD in 1/3 octave bands at the three default observers.



(b) `decay_OASPL`: OASPL along the wind with $1/r$, $1/r^2$ and measured data.

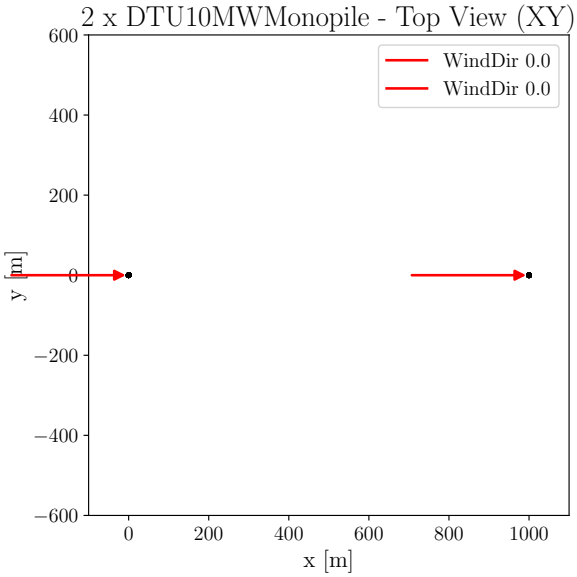


(c) `sliceXZ_OASPL`: OASPL on the vertical plane through the axis.

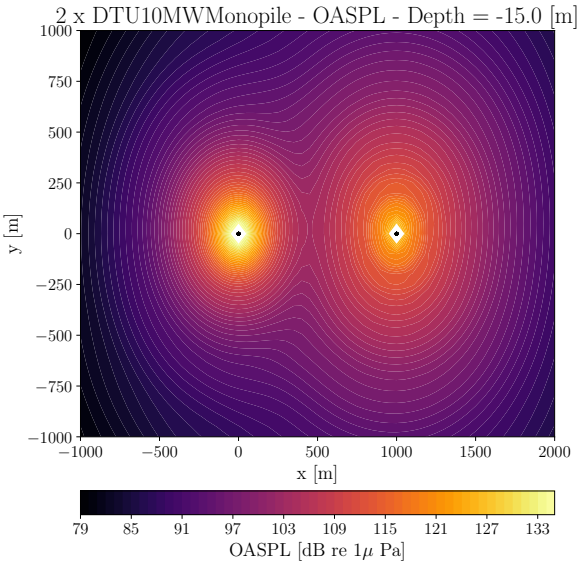


(d) `point_source_SL`: equivalent source level, free field (sphere) and in situ (cylinder); the shaded band is the near field $kr < 3$.

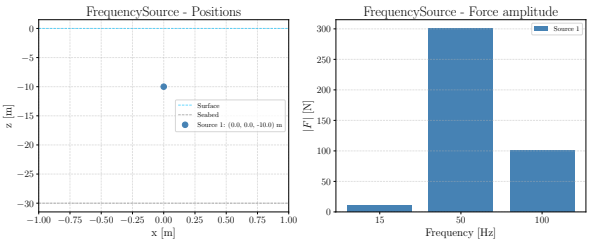
Figure 8.1: `turbine.py plot_mn_IM30_ws11.4 --save --no-show --format pdf`: the DTU 10 MW monopile at 11.4 m/s with the drivetrain filter, image method (input in `docs/figures/inputs/gallery_turbine/`). Look at the level of the drivetrain bands in (a) and at the difference between the free-field and in-situ source levels in (d), the effect of the waveguide below the first cut-on frequency.



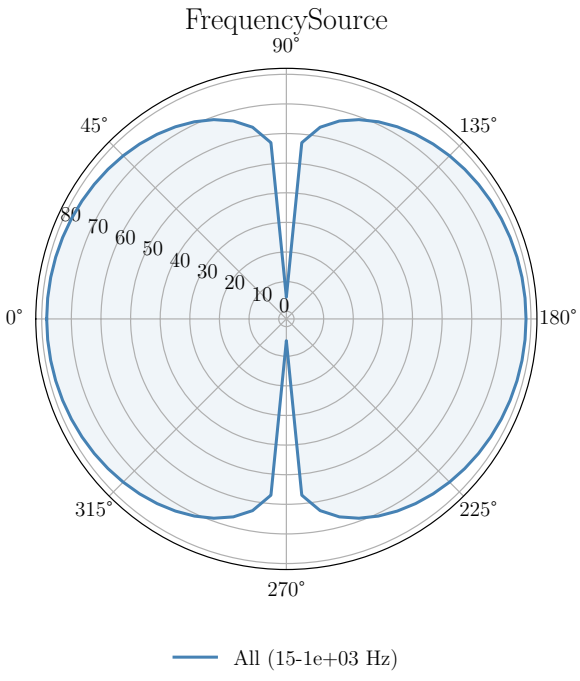
(a) `structure_xy`: the two turbines and their wind directions.



(b) `sliceXY_OASPL`: OASPL at mid-depth over the farm.



(c) `source.py`: `sources`, the dipole and its force at the three tones.



(d) `source.py`: `polar_OASPL`, the $\cos\theta$ directivity of the F_x dipole at 500 m.

Figure 8.2: Top: `farm.py Farm_2_DTU10MN_IM30_ws8.0-11.4 --save --no-show --format pdf`, the farm of `input_files/Emitters/WindFarm.nml` (input in `docs/figures/inputs/gallery_farm/`). Bottom: `source.py Source_Ns1_Nf3_IM30_R0.5 --save --no-show --format pdf`, the `FrequencySource` case of `input_files/`. Look at the interference between the two coherent turbines in (b).

Table 8.2: Plots of a single case (`plotting/plots/`). Every one takes `mode` (`'OASPL'`, `'SPL'`, `'PSD'`, `'ABS'`, `'REAL'`, `'IMAG'`, `'PHASE'` at `target_frequency`) and `absorption`, and the frequency filters `filter_under`, `filter_over`; the comparison versions (`plotting/compare/plots.py`) draw one panel per case.

Function	What it draws
`plot_spectrum`	SPL or PSD at the `spectrums` observers, narrow band (optionally smoothed over 1/12 octave) or in 1/3 octave bands, with the OASPL of each observer; observers at the same distance get their mean; `do_thresholds` adds hearing thresholds of marine species
`plot_polar`	OASPL (by bands, `bands=[10, 100]` Hz by default) or a level at one frequency on the circle
`plot_cylinder`	the unrolled cylinder, azimuth against depth
`plot_distance_decay`	the decay line, with the $1/r$ and $1/r^2$ slopes; `turbine_data=True` adds published measurements of turbines
`plot_line`	the line from $\boldsymbol{p}_1$ to $\boldsymbol{p}_2$
`plot_sliceXY`, `plot_sliceXZ`, `plot_sliceVertical`	maps of the planes, with the sources
`plot_structure`	the members of a turbine or farm and the wind direction (`'3D'`, `'xy'`, `'xz'`)
`plot_sources`	positions and force amplitude of the point sources (and the series of a `TimeSource`)
`compute_sphere_metrics`, `compute_cylinder_metrics`	print the OASPL, power and levels of the surfaces
`compute_point_source`	equivalent source level, free field and in situ, propagation loss, with three figures (Section 6.4)

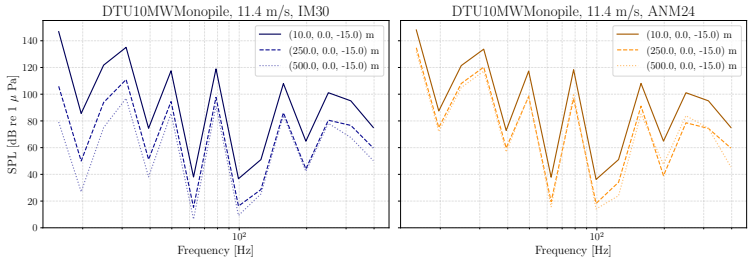


Figure 8.3: A comparison, `turbine.py plot_mn_IM30_ws11.4.hdf5 plot_mn_ANM24_ws11.4.hdf5 --save`: the same turbine with the image method (30 images, bottom attenuation 0.5) and the normal modes (24 modes), SPL in 1/3 octave bands. Look at how the lossy seabed of the image method lowers the level at the far observers.

Chapter 9

Validation

This chapter collects the checks of the solver against closed forms, between its two methods and between its input modes. The figures and the numbers come from `docs/figures/make_figures.py` (`docs/figures/data/numbers.txt`); the checks of the input modes were made when they were added (October 2026).

Table 9.1: Summary of the checks and the agreement reached.

Check	Reference	Agreement
Free-field dipole (image method, `N_images = 0`)	closed form (1.4)	6.6×10^{-16} (relative, maximum over 272 points and 2 frequencies)
Image method, ideal boundaries	normal modes	$\sim 1/N$: 10^{-4}–10^{-3} with 4096 images; 2–71 % with 30
Normal modes, automatic `Nmodes`	400 modes	identical
Hann tone through `compute_rfft`	amplitude and power	full amplitude on a bin; $\sum \lvert F_k \rvert^2 = A^2$ ENBW to 5×10^{-4}
`TimeSource` → `FrequencySource` (the spectrum given as files)	the `TimeSource` itself	bit-identical pressures
`FrequencySource` input modes (list, grid, file, expressions, positions from a file)	each other	bit-identical datasets (14 comparisons, IM and ANM)
Expression parser	Fortran intrinsics	bit-identical (62 expressions of t and f)

9.1 Free-field dipole

A horizontal dipole $\boldsymbol{F} = (100, 0, 0)$ N at $z = -15$ m radiates at 50 and 200 Hz; the image method with `N_images = 0` computes its free field on the decay line along $+x$ from 1 to 1000 m and on a circle of 50 m. Figure 9.1 compares it with (1.4): the largest relative difference is 6.6×10^{-16}, the rounding of double precision. The near field ($1/R^2$, below $kR = 1$, i.e. 4.8 m at 50 Hz) and the far field ($1/R$) are both reproduced, and so is the $\lvert \cos\theta \rvert$ directivity with its nulls at 90° and 270°.

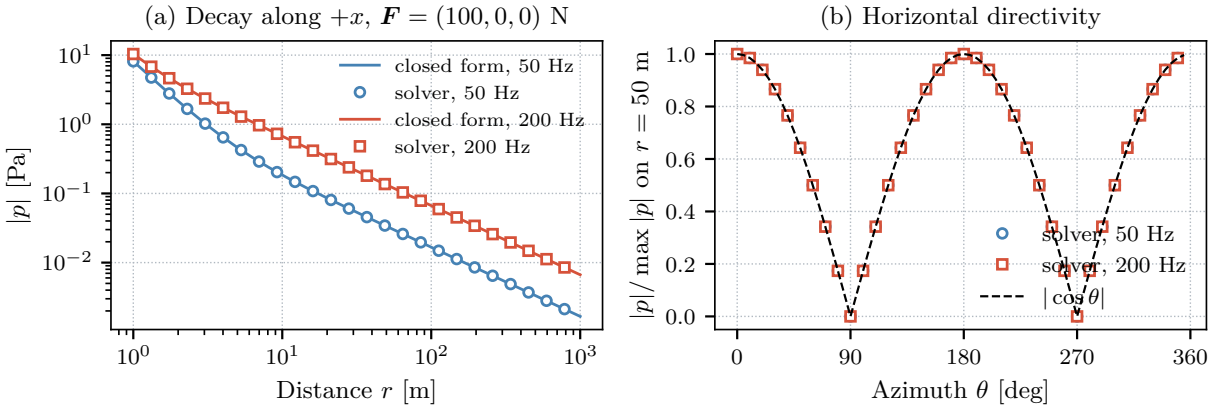


Figure 9.1: Free-field dipole: (a) magnitude along $+x$, solver (markers) and closed form (1.4) (lines), showing the change of slope from $1/R^2$ to $1/R$; (b) normalised magnitude on a circle of 50 m against $|\cos\theta|$. Input: `docs/figures/inputs/freefield/`.

9.2 Image method against normal modes

With $p = 0$ at the surface, a rigid seabed and both attenuations equal to 1, the image method and the normal modes solve the same problem. Figure 5.2 (Section 5.1.3) shows the image sum converging to the modes as $1/N$, more slowly at long range. The normal modes themselves are converged: the automatic number of modes (21 at 400 Hz in 30 m of water) and 400 modes give the same pressure to the last bit, because the evanescent modes beyond the automatic ones are cut off by the decay test before they contribute. Figure 5.4 shows the two methods on a range of 5 km and the effect of the bottom attenuation of the image method.

9.3 The FFT of the time-domain sources

Two tones of amplitude 1000 N, one on a bin (50 Hz) and one half a bin off (50.25 Hz), are given to a `TimeSource` with $T = 2$ s ($\Delta f = 0.5$ Hz). Figure 9.2a shows what `compute_rfft` gives:

- on a bin: exactly 1000 N at 50 Hz and 500.375 N at the two neighbours (a periodic Hann window would give 500 N there; the code uses the symmetric one, of denominator $N_t - 1$);
- half a bin off: 848.97 N at the two nearest bins and a slowly decaying leakage beyond;
- the power is the same in both cases: $\sum_k |F_k|^2 = 1.50075 \times 10^6$ N^2, which is $A^2 \times$ ENBW with ENBW $= 1.5$ to 5×10^{-4} (1.5 is the large-N_t limit of the ENBW of the symmetric window).

The spectrum of the on-bin case, written to two files and given to a `FrequencySource` with the same frequencies, gives bit-identical pressures (Figure 9.2b): the two emitters share everything from the forces on.

9.4 Input modes and parser

When the input modes of `FrequencySource` were added, these properties were checked on copies of the tree (no run in `FORTRAN/`):

- **Regression**: the shipped `FrequencySource` and `TimeSource` cases, with both methods, give bit-identical HDF5 datasets, `.dat` and standard output (apart from the elapsed time) before and after the change, also when the old namelist is read by the new binary.

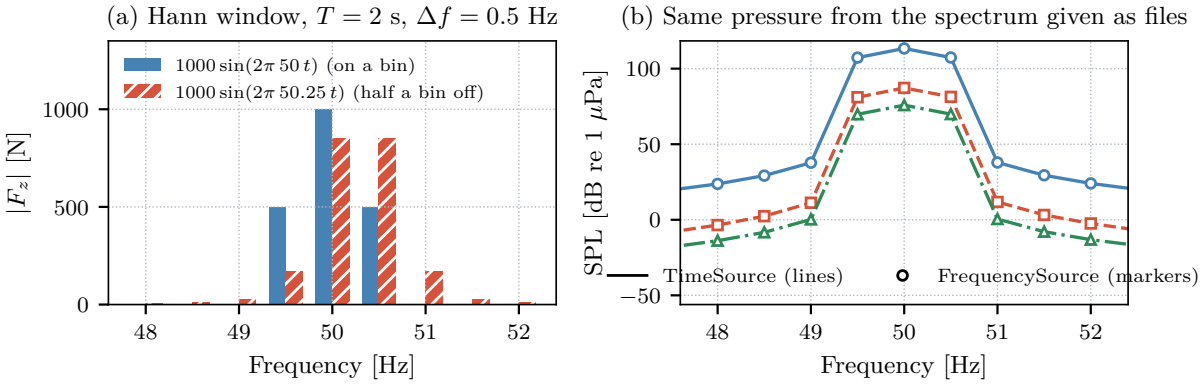


Figure 9.2: (a) A tone through the Hann window of `compute_rfft`: on a bin (blue) and half a bin off (red, hatched). (b) SPL at the three default observers of the `TimeSource` (lines) and of the `FrequencySource` fed with its spectrum (markers): identical. Inputs: `docs/figures/inputs/hann_*/`.

- **Equivalence**: two sources with complex forces and different phases, given as written lists, a linear grid, a logarithmic grid, files, expressions of f and positions from a file, give bit-identical datasets with both methods.
- **Errors**: 45 wrong inputs (missing files, wrong columns or rows, non-increasing or non-positive frequencies, two modes at once, `t` in an expression of `f`, syntax errors, an expression that is infinite on the grid, `fmin >= fmax`, a source outside the water, ...) stop with a message that names the input.
- **update_existing**: changing one value of a force file (in its 12th significant digit) rebuilds the output; the same input reuses it.
- **Parser**: 31 expressions with `t` and the same with `f` give the same bits as the Fortran intrinsics compiled with the flags of the project (`-O3 -fopenmp`); the previous version of the parser gives the same bits as the new one.
- **Builds**: `make`, `fpm build`, `-Wall` with only the accepted warnings, and `-fcheck=all -fbacktrace` with no runtime error in 79 runs.

> **Note** The parser and a Fortran expression agree to the last bit only when both are compiled the same way: with `-O3 -fopenmp` the array intrinsics use vectorised versions of `sin`, `exp`, `pow`..., which can differ by one unit in the last place from the scalar ones used at `-O0`.

9.5 What is not verified here

The turbine and farm emitters depend on OpenFAST outputs and are checked by the regression cases of the refactor (Section 18.1), not by a closed form. The radiation correction of the turbine forces (Section 13.2) and the drivetrain filter are models; they are described, not validated, in this guide.

Chapter 10

Troubleshooting

Every error stops the run with `ERROR STOP` after a message; the message names the namelist group and the variable, or the file. Appendix C lists them by source; this chapter explains the common ones.

10.1 The run stops at once

Unknown simulation_type / Unknown method. The value of `simulation_type` or `method` is not one of the names of Section 3.1; they are case-sensitive.

ERROR: Could not open input file: ... The file chosen by `Solver.nml` is missing, e.g. `ComputationSetups/TimeSource_CS.nml`, or the input directory given on the command line is wrong.

Failed to read &group: End of file. Either the group is missing from its file, or its closing `/` is the last character of the file: add a newline (and, as in the shipped files, a blank line and a comment) after it.

Failed to read &group: Cannot match namelist object name ... A variable that the group does not have, often a typo or a variable of another group (in a farm, the reading settings belong to `&farm_config`, not to the `&turbine` groups).

10.2 A namelist value is silently ignored

A `/` outside a string and outside a `!` comment ends the group: everything after it is not read, and no message says so. The usual cause is a `//` used as a comment:

```
force_re(1, 1, 1:3) = 10.0, 0.0, 0.0
// force_im(1, 2, 1:3) = 0.0, 0.0, 1000.0    <- ends the group here
...                                          <- never read
```

Comments start with `!`. A `/` inside a `!` comment or inside a quoted string (an expression such as `'100/(1+(f/50)**2)'`) is harmless.

10.3 Sources and observers

ERROR: 1 of 1 sources are outside the water column; the first one, source 1, is at z = -45.00 m (the seabed is at -30.00 m and the surface at 0 m). Every method

checks the sources against its seabed when it starts; the readers only check $z \leq 0$. Move the source or increase `depth_base`.

source 1 is at z = 5.00 m, above the free surface (z = 0). The reader rejects $z > 0$.

WARNING *... observers closer than 1.0 m to a source node: masked (NaN)* and *... outside the water column.* Not errors: those observers are NaN (Section 6.2). For point sources a small `mask_distance` is enough; for turbines it should exceed the radius of the members.

Most of the sphere is NaN. The free-field sphere is masked outside the water column (Section 6.3.9).

10.4 Turbines and farms

WindTurbine % init: output file not found ... / SubDyn file not found. The default names are `OP_output_dir//rootname//'.outb'` (`'.out'` with `Binary = .false.`) and `OP_output_dir//rootname//'.SD.sum.yaml'`; `OP_output_dir` must end with `/`. A value of `OP_output_filename` other than `'None'` is a full path.

RPM curve file not found. The message is `DTU10MWMonopile % init: RPM curve file not found: ../wind_speed_curves_DTU_10MW/rpm_ws.csv`. The rpm curve is looked for relative to the working directory: run from `FORTRAN/` (or from a directory next to `wind_speed_curves_DTU_10MW/`).

Unsupported turbine type: ... `case_type` must be one of Table 4.1.

ERROR WindFarm % farm_to_source: turbine 2 has df = ... Hz, turbine 1 has df = ... Hz. The turbines of a farm must have the same record length and time step after `From`, `Upto` and `skip`.

WARNING *... has zero force in N frequency bins.* Not an error: the turbines kept different bins (Section 4.2).

10.5 Manual sources

the frequency axis is given in more than one way (freqs and fmin/fmax). Comment out the values of the way you do not use; the same for the forces and the positions (*the positions are given twice*).

force_re(2, 1, 1:3) is missing or not finite. Every line of the written forces is mandatory for the first `Nfreqs` frequencies and `Nsources` sources.

force_re_expr(1, Fz): unknown name 't' (use f, pi or a function) at position 14 of 'sin(2*pi*50*t)'. The variable of a frequency expression is `f`; that of a time expression `t`.

the real part (force_re_expr) of the force Fz of source 1 is not finite at f = 5.00000E+01 Hz. The expression divides by zero or takes the logarithm of a negative number at that frequency.

force_re_file re.txt has 3 rows and the frequency axis has 4 frequencies / line 2 of re.txt has 5 values, 6 are required. A force file has one row per frequency and $3N_s$ columns.

force_re and force_im hold at most 200 frequencies. The forces written by hand are limited to 200 frequencies and 20 sources: give them as files or expressions.

10.6 Builds and binaries

Could not find h5fc or h5pfc on PATH. Load or install HDF5 with its Fortran bindings.

undefined reference to 'omp_get_wtime_'. `FFLAGS` was overridden without `-fopenmp`.

A segmentation fault right after reading the inputs. A binary older than the sources it was built from (Chapter 2): rebuild.

10.7 Outputs

WARNING ***check_update: ... not updated (different setting: "..." -> "..."): deleted and rebuilt.*** With `update_existing`, the existing file held another case; the message quotes the first line of the header that changed.

Holes in the maps. Masked observers (Section 6.2).

mode 'PSD' needs uniformly spaced frequencies. A `FrequencySource` whose frequencies are not uniformly spaced has no bin width: plot the SPL (Section 3.4.3).

Part II

Developer guide

Chapter 11

Architecture

The code is about 15 000 lines of Fortran 2008 in 35 modules and 2 submodules under `FORTRAN/src`, plus `FORTRAN/app/main.f90`. `FORTRAN/ARCHITECTURE.md` is the reference for the contracts; this chapter gives the picture.

11.1 Layers

The modules form layers, and a module never uses one of a higher layer (Figure 11.1). `input` and `emitters` are at the same level and do not use each other; both use the utilities (e.g. `read_table` and the expression parser serve the readers and the manual emitters). Every module is `PRIVATE` by default with an explicit `PUBLIC` list (`Kinds` excepted), and `USE ..., ONLY:` lists what is taken.

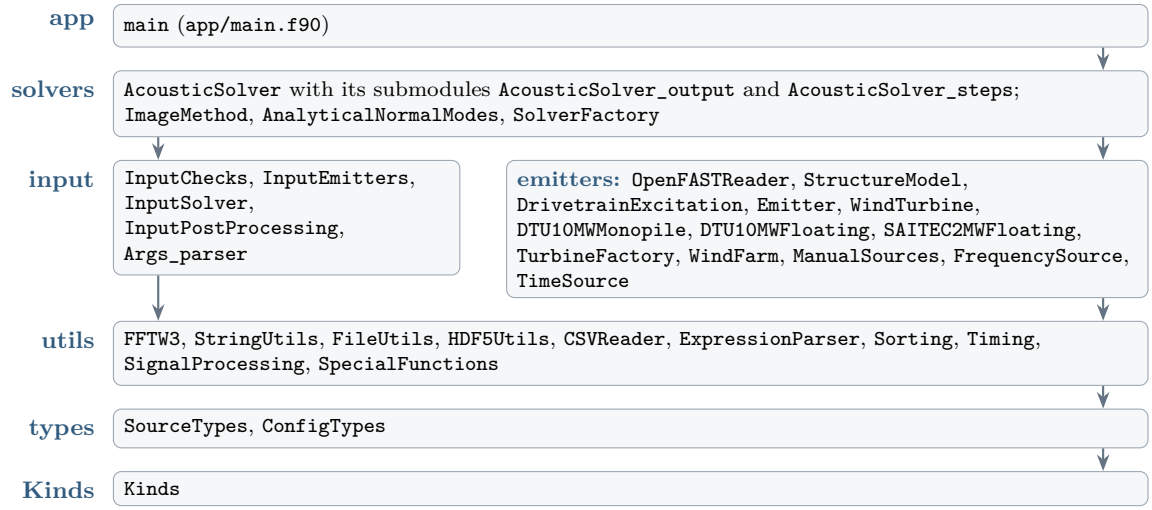
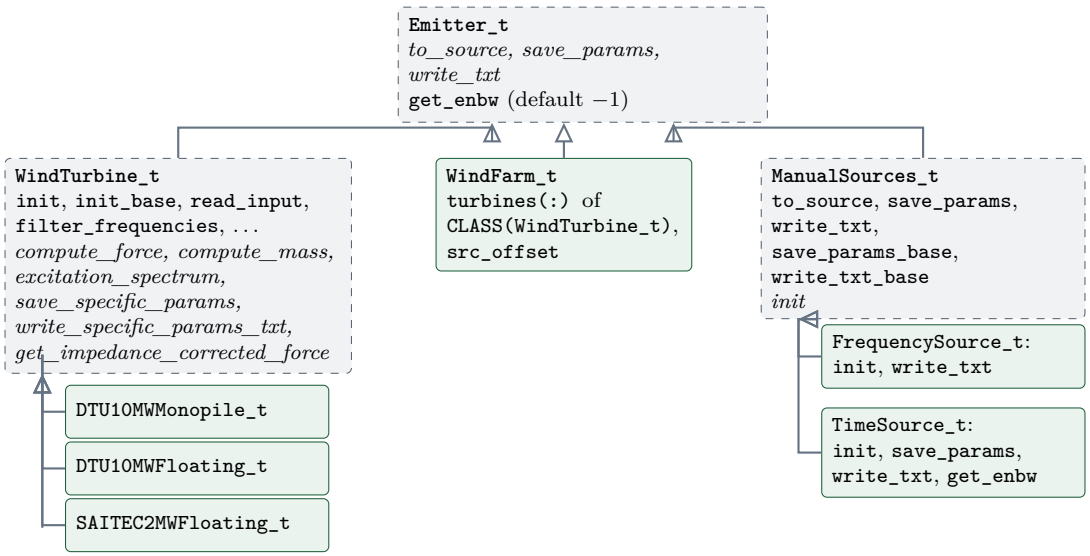


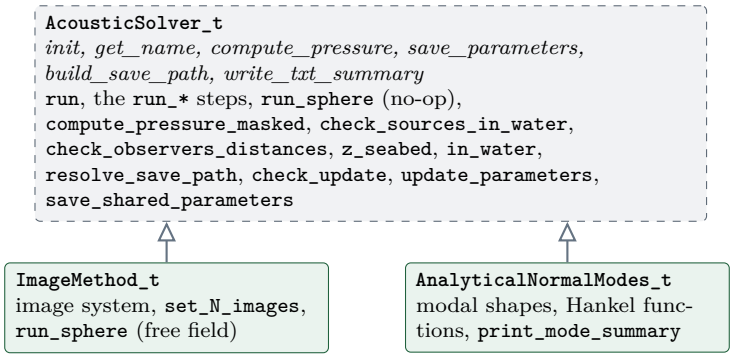
Figure 11.1: Layers and modules. Arrows point from a layer to the layers it uses; within a layer the order is that of the Makefile. `AcousticSolver_output` and `AcousticSolver_steps` are submodules of `AcousticSolver`.

11.2 Class hierarchies

Two abstract types organise the code: `Emitter_t`, what produces the sources, and `AcousticSolver_t`, what propagates them (Figure 11.2). `main.f90` only sees them through these types after building them, and the solvers only see `Emitter_t`: a new emitter needs no change in the solvers, and a new method none in the emitters.



(a) Emitters (`emitters/`)



(b) Acoustic methods (`solvers/`)

Figure 11.2: The two class hierarchies. Dashed boxes are `ABSTRACT` types; the bindings in italics are `DEFERRED`. A child also overrides some non-deferred bindings (e.g. `init` of the turbine types, `run_sphere` of `ImageMethod_t`).

Why abstract. `Emitter_t` and `AcousticSolver_t` have deferred bindings, the contract every emitter and method must fulfil (Sections 13.1 and 14.1). `WindTurbine_t` is abstract because the structural model is per type (`compute_force`, `compute_mass`, `excitation_spectrum`, `save_specific_params`, `write_specific_params_txt`, `get_impedance_corrected_force`); `ManualSources_t` because the way the forces are given is per child (its `init`).

Calling the parent version: the `*_base` aliases. A child that extends a binding of an abstract parent (e.g. `TimeSource_t % save_params`, which writes the common datasets and then its own) cannot call the parent version with the parent component: gfortran 13 rejects

```
CALL self % ManualSources_t % save_params(save_path)
! Error: Base object for type-bound procedure call at (1) is of ABSTRACT type
```

The project's idiom is a second binding of the same procedure, which the children never override:

```
PROCEDURE :: save_params      => save_params_ManualSources   ! Overridden
PROCEDURE :: save_params_base => save_params_ManualSources   ! Never overridden
...
CALL self % save_params_base(save_path)    ! In the child: the parent version
```

`WindTurbine_t` has `init_base` for the same reason, and `ManualSources_t` has `save_params_base` and `write_txt_base`.

11.3 Program flow

Figure 11.3 follows a run from the command line to the last file (`app/main.f90`). The emitter is built through a variable of its concrete type, because the abstract `active_emitter` can neither be allocated nor call `init` (which is not part of the `Emitter_t` contract); at the end of its branch it is moved (`MOVE_ALLOC`) into `active_emitter`, which `save_parameters` uses and which is freed before the steps run.

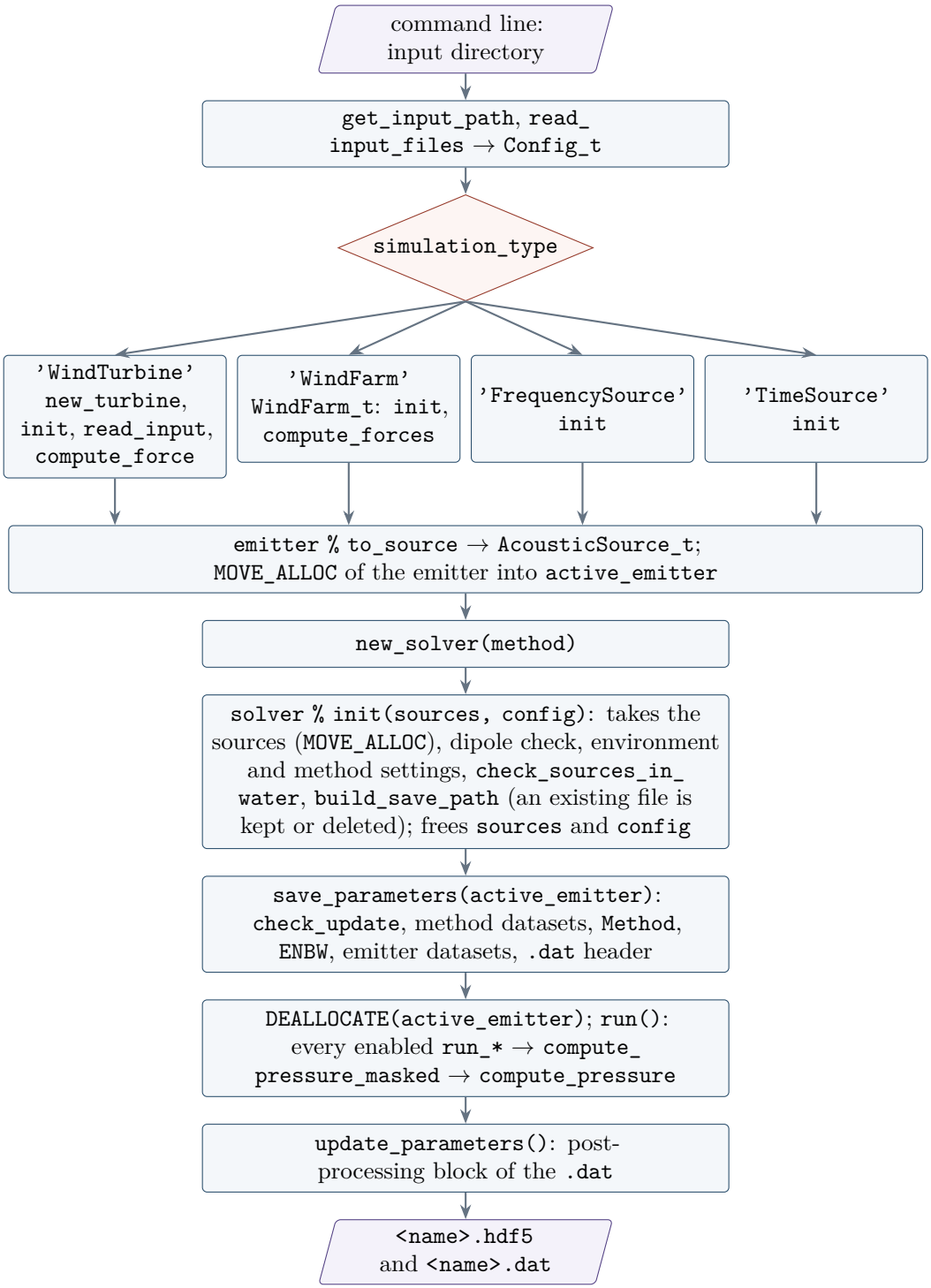


Figure 11.3: Flow of `main.f90`. The four branches build the emitter; from `to_source` on, everything is common. The solver takes the arrays of the sources with `MOVE_ALLOC` and frees the configuration in its `init`; `check_sources_in_water` runs there, once the seabed of the method is known.

Chapter 12

Data model

Three kinds of objects carry the data of a run (Figure 12.1): the configuration `Config_t`, filled by the readers and freed by the solver; the emitter, built from it; and the `AcousticSource_t`, the hand-off between the emitter and the solver.

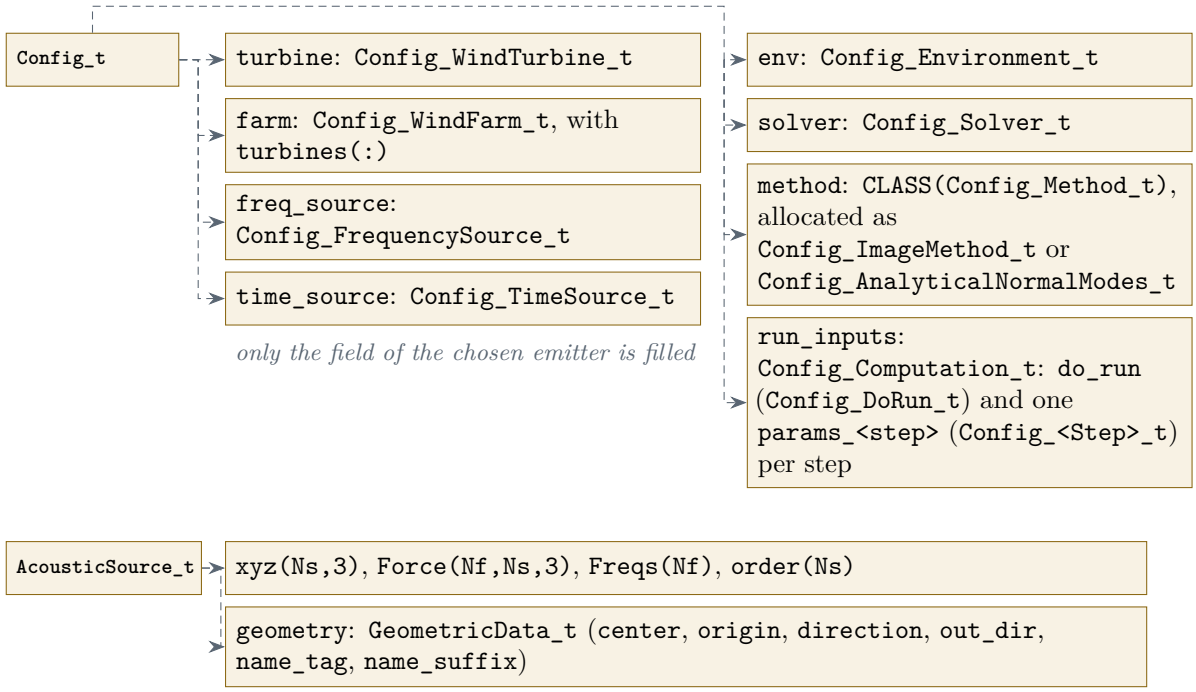


Figure 12.1: Composition of `Config_t` (`types/ConfigTypes.f90`) and `AcousticSource_t` (`types/SourceTypes.f90`). `Config_t % method` is polymorphic: the parser allocates the extension of the chosen method. Only the field of the chosen emitter is filled.

12.1 Kinds and constants

`Kinds.f90` is the only module that is public as a whole.

Name	Value	Use
I32, I64	selected_int_kind(9), (18)	integer kinds (INTEGER(I32) everywhere)
SP, DP, WP	WP = DP	working precision; must stay double: FFTW (C_DOUBLE) and HDF5 (H5T_NATIVE_DOUBLE) assume it
PI, I1	π, $(0, 1)$	constants
DEFAULT_R, DEFAULT_I	-999.0, -999	sentinels of the parser (*automatic*)
MAX_PARSE_OBS, MAX_PARSE_SRC, MAX_PARSE_FREQ	200, 20, 200	sizes of the buffers of values written by hand

12.2 The configuration

Config_t aggregates one component per input file (Figure 12.1). It is a plain container: no procedure, no run-time state. The readers of input/ fill it, main.f90 hands it to the emitter and to the solver, and the solver's init copies what it needs (env, run_inputs, the settings of solver and of its method) and deallocates it. The emitter types keep their own copy of what they need after to_source (save_params and write_txt run later).

Sentinels. The automatic values stay DEFAULT_R/DEFAULT_I in Config_t; each run_* step resolves them at run time, writes the resolved values back into run_inputs and the .dat reports them (Section 3.5).

12.3 The sources

Table 12.1: AcousticSource_t and its GeometricData_t.

Component	Shape, unit	Meaning
xyz	$(N_s, 3)$ m	positions
Force	$(N_f, N_s, 3)$ N, complex	forces on the fluid, peak amplitude, $e^{-i\omega t}$
Freqs	(N_f) Hz	frequencies, positive and increasing
order	(N_s)	multipole label: SOURCE_DIPOLE $= 1$, SOURCE_MONOPOLE $= 0$
geometry % center	(2) m	reference centre (Table 6.1)
geometry % origin	(2) m	reference origin
geometry % direction	deg	reference direction, anticlockwise from $+x$
geometry % out_dir, name_tag, name_suffix	strings	default output name

The emitter allocates it in to_source; the solver's init takes its arrays with MOVE_ALLOC, so they are never duplicated in the solver, copies geometry and frees it. Inside the solver they live in AcousticSolver_t % sources, which the steps read (xyz for the masks, the sphere and the automatic width of the vertical slice; geometry for the defaults; Freqs in every output).

12.4 Run-time state of a solver

AcousticSolver_t holds, besides the sources: the environment env, the step settings run_inputs, block_size, debug, in_cluster, update_existing, mask_distance, and solver_name, save_

`path` and `extra_label`. The methods add their own: `ImageMethod_t` its image system (the positions, forces and signs of the $N_s(1 + 2N)$ real and image dipoles, built in `init` and rebuilt when `N_images` changes), `AnalyticalNormalModes_t` the number of modes and the depth.

Chapter 13

Emitters in depth

13.1 The emitter contract

`Emitter_t` (`emitters/Emitter.f90`) has three deferred bindings and one with a default:

Binding	Contract
`to_source(self, config, sources)`	allocate and fill the `AcousticSource_t`: `xyz`, `Force`, `Freqs`, `order`, `geometry`
`save_params(self, save_path)`	write the datasets of the emitter into the HDF5 file
`write_txt(self, unit)`	write the emitter block of the `.dat`; it must list every input that defines the case (or checksums of it), because `check_update` compares it
`get_enbw(self)`	ENBW of the window of the forces; -1 (no window) by default, `HANN_ENBW` for turbines, farms and `TimeSource`

`init` is not part of the contract: each emitter has its own (Section 11.3).

13.2 The turbine, step by step

1. Initialisation (emitters/WindTurbine.f90:init). The type's override of `init` sets its data (`path_rpm`, `is_monopile`) and calls `CALL self % init_base(config)`, which copies the configuration and builds the paths of the OpenFAST files (`OP_output_dir//rootname//'.outb'` and `'.SD.sum.yaml'` when the names are `'None'` or `'Default'`); a missing file stops the run.

2. OpenFAST (emitters/WindTurbine.f90:read_input, `emitters/OpenFASTReader.f90`). `get_SDsum_variables` parses the SubDyn summary: the member table (the identifiers of the nodes of each member, after the seven leading columns `I, Joint1_ID, Joint2_ID, Prop_I, Prop_J, Mass, Length`) and the node table (`- [ id, x, y, z, ... ]`), and builds `xyz_all(Nmembers, Nnodes, 3)`. `read_input_SD` reads the channels `M<m>N<n>TAxe/TAye/TAze` into `acc(nt, Nmembers, Nnodes, 3)`:

- a `.outb` file is parsed from its header (file identifier, number of channels and of rows, time scaling or start and increment, the scale and offset of every channel, the names and units) and its int16 data, scaled as $(v - \text{offset})/\text{scale}$, in chunks of 4096 rows;
- a `.out` file has 6 header lines, then the names and the units, then the rows;
- the window keeps the rows $\text{int}(N_T\,\texttt{From}) + 1$ to $\text{int}(N_T\,\texttt{Upto})$ and then one out of `skip`.

The time step is $\Delta t = t_2 - t_1$ and the bin width $\Delta f = 1/(N_t \Delta t)$.

3. Frame (`translate_align_with_WindDir`). With $c = \cos\beta$, $s = \sin\beta$, $\beta =$ `WindDir`: the accelerations become $(ca_x - sa_y,\ sa_x + ca_y,\ a_z)$ and the nodes $(cx - sy + X_{\text{axis}},\ sx + cy + Y_{\text{axis}},\ z)$. An automatic barycentre is the mean (x, y) of the nodes.

4. Forces (`compute_force` of the type). The DTU monopile, for instance, removes the nodes that two consecutive members share (emitters/StructureModel.f90:remove_duplicate_nodes), keeps the wet ones ($z \le 0$), takes their accelerations to the frequency domain (`compute_rfft`, next section) and multiplies each node by its mass and added mass,

$$\boldsymbol{F}_j(f) = (m_j + m_{\text{a},j})\,\hat{\boldsymbol{a}}_j(f), \quad m_j = \rho_s \pi \frac{D_o^2 - D_i^2}{4}\Delta s_j, \quad m_{\text{a},j} = \rho\,\pi\frac{D_o^2}{4}\Delta s_j, \tag{13.1}$$

with Δs_j the length of the member assigned to the node (`divide_span`: half the distance to each neighbour). The floating DTU type does the same per column and pontoon (with lids on the columns and a rectangular section for the pontoons). If `filter_freqs` is set, the frequency mask of `filter_frequencies` is applied to `Freqs` and `Force`.

5. Drivetrain filter (emitters/WindTurbine.f90:filter_frequencies). The rpm is `read_curve(path_rpm, WindSpeed)`; the type's `excitation_spectrum` returns the peaks (for the 10 MW drivetrain, emitters/DrivetrainExcitation.f90:drivetrain10MW_excitation_spectrum: 13 shaft and 9 mesh peaks at frequencies interpolated between the values at 6 and 9.6 rpm, with amplitudes shaped by 14 torsional modes); `generate_timeseries_banded_sines` spreads the mesh peaks and those above 10 Hz into 50 lines within $f_0 \pm 4\sigma$, $\sigma = 0.02 f_0$, and returns their frequencies; `filter_non_useful_freqs` keeps every bin below `freqs_over` and every bin within one bin of a line. The synthetic signal that `generate_timeseries_banded_sines` also builds is not used.

6. Radiation correction (`get_impedance_corrected_force` of the type, called by emitters/WindTurbine.f90:turbine_to_source). Every force is multiplied by the magnitude of

$$\alpha(ka) = \frac{-3\mathrm{i}}{\pi (ka)^2\, H_1^{(2)\prime}(ka)}, \qquad H_1^{(2)\prime} = \tfrac{1}{2}\left(H_0^{(2)} - H_2^{(2)}\right), \qquad ka = \frac{2\pi f}{c}\frac{D}{2} \tag{13.2}$$

(emitters/StructureModel.f90:alpha_hankel), with D the diameter of the monopile, of a column, or the equal-area diameter $2\sqrt{AB/\pi}$ of a pontoon. Only $|\alpha|$ is used, so the kind of the Hankel function does not matter. Numerically $|\alpha| = 1.500$ at $ka = 10^{-3}$, 1.514 at 0.1, 1.03 at 1, 0.42 at 2 and 0.038 at 10, decaying as $(ka)^{-3/2}$.

7. Dipoles (`turbine_to_source`). The wet nodes become the sources (`order` = dipole), with the corrected forces, the frequencies, and the reference geometry and names of Tables 6.1 and 7.1.

13.3 The FFT

utils/SignalProcessing.f90:compute_rfft takes `array(nt, N2, N3)` to the one-sided spectrum of every series:

$$X_k = \frac{2}{\sum_n w_n}\sum_{n=0}^{N_t-1}(x_n - \bar{x})\,w_n\,\mathrm{e}^{+2\pi\mathrm{i}kn/N_t}, \qquad w_n = \frac{1}{2} - \frac{1}{2}\cos\frac{2\pi n}{N_t - 1}, \qquad f_k = \frac{k}{N_t \Delta t}, \tag{13.3}$$

computed with FFTW (`fftw_plan_many_dft_r2c`, all the series in one plan) and conjugated, so that a series $A\cos(2\pi f_k t)$ gives $X_k = A$ with the time factor $\mathrm{e}^{-\mathrm{i}\omega t}$. The bins kept are $k = 1, 1 +$ `skipf`, … up to $N_t/2$ (with `remove_zero`, the default; $k = 0$ otherwise). The window is the symmetric Hann window; its equivalent noise bandwidth $N\sum w^2/(\sum w)^2$ tends to `HANN_ENBW` $= 1.5$.

13.4 The farm

`WindFarm_t` holds `turbines(:)`, an array of `TurbineSlot_t`, each with a `CLASS(WindTurbine_t)` (Fortran has no arrays of polymorphic objects), and `src_offset(Nturbines + 1)`: turbine i owns the sources `src_offset(i)+1 : src_offset(i+1)`. emitters/WindFarm.f90:farm_to_source_WindFarm:

1. checks that every turbine has the Δf of the first (relative 10^{-9});
2. maps the frequencies of every turbine to bin numbers $k = \mathrm{nint}(f/\Delta f)$ and takes their union;
3. calls the `to_source` of every turbine (so each applies its own correction) and scatters its sources and forces into the farm arrays, with zero force in the bins it does not have;
4. sets the name, the centre (the mean barycentre), the origin (the mean axis) and the direction, the circular mean of the wind directions β_i of the turbines:

$$\beta = \mathrm{atan2}\Big(\sum_i \sin\beta_i, \sum_i \cos\beta_i\Big).$$

Its datasets are those of every turbine with the suffix `_<i>`, plus `Nturbines` and `Source_offset`.

13.5 Manual sources

`ManualSources_t` (`emitters/ManualSources.f90`) stores what the solver receives (`Nsources`, `order`, `Freqs`, `xyz`, `Force`) and three labels set by the child (`kind_name`, the `Emitter` dataset; `title`, of the `.dat` block; `name_suffix`). Its `to_source` copies the arrays (they are still needed by `save_params` and `write_txt`) and sets the centre and origin to the mean (x, y), the direction to 0 and the name to `Source_Ns<N><name_suffix>`. Its `save_params` writes `Emitter`, `Nsources`, `NFreqs`, `Source_xyz`, `Source_order` and `Source_Force`; its `write_txt` the title, the counts and the positions.

`FrequencySource_t` (`emitters/FrequencySource.f90`): `init` copies the configuration and calls `build_frequency_axis` (list; grid $f_i = f_{\min} + (i-1)\Delta f$ or $10^{\log_{10} f_{\min} + (i-1)\delta}$; or the rows of `freq_file`, then the check of positive and increasing values) and `build_forces` (copy; or the expressions of `f` compiled and evaluated on `Freqs`, real and imaginary parts; or the columns of `force_re_file` and `force_im_file`, with `check_finite_force`). Its `write_txt` adds the description of the input and the force table, or checksums (Section 7.3).

`TimeSource_t` (`emitters/TimeSource.f90`): `init` builds the time axis (`build_time_axis`: uniform, or the rows of `time_file` checked to be uniform to 10^{-6}), the series `Ft(nt, Nsources, 3)` (`build_force_series`: the columns of `force_file` or the expressions of `t`), and their spectrum (`take_to_frequency_domain`: `compute_rfft` and the band `fmin`–`fmax`). It overrides `save_params` (adds `Nt`, `Time_step`, `Source_time`, `Source_Force_time`), `write_txt` (the time axis and checksums of the series) and `get_enbw`.

13.6 The expression parser

`utils/ExpressionParser.f90` turns a text into a list of operations once, and evaluates the list on whole vectors, so that the cost of the parsing does not depend on the length of the series.

Grammar. A recursive descent, one routine per rule (`parse_sum`, `parse_product`, `parse_unary`, `parse_power`, `parse_primary`):

```
sum     := product { (+|-) product }
product := unary { (*|/) unary }
unary   := (+|-) unary | power
power   := primary [ ** unary ]
primary := number | name | name '(' sum { ',' sum } ')' | '(' sum ')'
```

`power` takes a `unary` as exponent, so `**` is right-associative and `a**-b` is valid; the unary minus is below `power`, so `-t**2` is $-(t^2)$.

Compilation (`compile_expression(text, expr, message, variable)`). Each rule *emits* its operation after those of its operands, so the result is the expression in postfix order, stored in `Expression_t`: `code(n)`, `value(n)` for the numbers, and `depth`, the largest stack needed, followed during the compilation. The variable is `t` unless `variable` names another one (`'f'` for the frequency forces); any other name than the variable, `pi` and the 22 functions is an error. The first error is kept with its position and returned in `message`; an empty message means a valid expression.

Evaluation (`evaluate_expression(expr, x, values)`). A stack of `(size(x), depth)` vectors: numbers and the variable push a vector, operators and functions of two arguments pop one, and every operation works on whole vectors. Domain errors give NaN or Inf, which the callers check (`check_finite_force`, the series check of `TimeSource`).

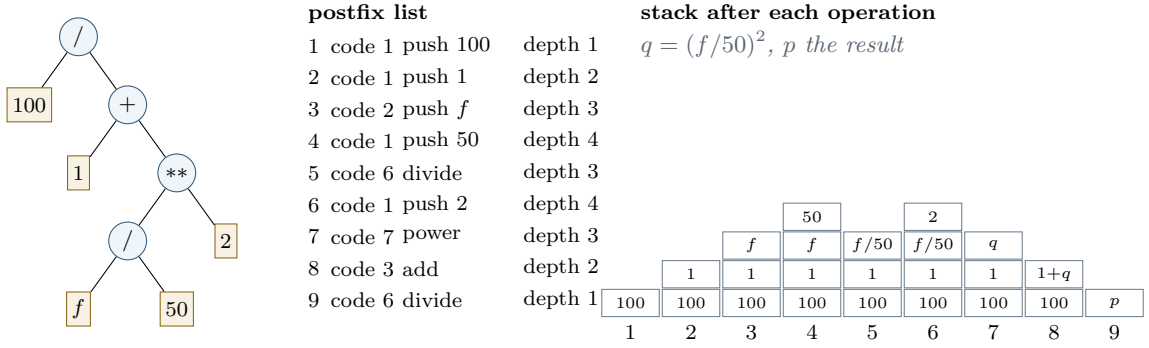


Figure 13.1: Compilation of `'100/(1+(f/50)**2)'`: the parse tree (left), the postfix list that `compile_expression` emits (centre, the codes of `OP_NUMBER`, `OP_VAR` and the operators) and the stack during the evaluation (right), whose largest depth, 4, is `expr % depth`. Evaluated at $f = 0, 50, 100$ Hz the expression gives 100, 50 and 20 (checked with the compiled parser).

Chapter 14

Acoustic methods in depth

14.1 The solver contract

`AcousticSolver_t` (`solvers/AcousticSolver.f90`) defers six bindings to the methods:

Binding	Contract
`init(self, sources, config)`	take `xyz`, `Force`, `Freqs`, `order` with `MOVE_ALLOC`; stop if a source is not a dipole; copy `env`, `run_inputs` and the settings of `solver`; read its `Config_X_t`; set its seabed and call `check_sources_in_water`; `build_save_path`; print `get_name()`; deallocate `sources` and `config`
`get_name(self)`	the name, also stored in `solver_name` (the `Method` dataset)
`compute_pressure(self, observers, block_size, p)`	the pressure (N_f, N_{obs}) at `observers` $(N_{\mathrm{obs}}, 3)$; called only through `compute_pressure_masked`
`save_parameters(self, emitter)`	`check_update`, its datasets, `save_shared_parameters`, `emitter % save_params`, `write_txt_summary`
`build_save_path(self)`	`resolve_save_path(tag)`, with its tag (`_IM30`, `_ANM24`)
`write_txt_summary(self, emitter)`	the `.dat` header: title, emitter block, environment, its own block

14.2 Image method

Image system (solvers/methods/ImageMethod.f90:method_of_images, called by `build_image_system` in `init`). For every real source at height z_0 two chains of `N_images` images are built: the first one starts across the seabed $z_b = -H$, the second across the surface $z_u = 0$, and both then alternate. A reflection across the plane z_p maps $z \mapsto 2z_p - z$, multiplies the sign by the boundary code of the plane (`Lower_BC` or `Upper_BC`), the force by the attenuation of the plane (`lw_R` or `up_R`) and reverses F_z:

$$z_{j+1} = 2z_p - z_j, \qquad s_{j+1} = s_j\, B_p, \qquad \boldsymbol{F}_{j+1} = a_p\,(F_{x,j}, F_{y,j}, -F_{z,j}). \tag{14.1}$$

The images keep the (x, y) of their source. The result is a flat list of $N_s(1 + 2N)$ positions, forces and signs (`ImageSystem_t`); it is rebuilt by `set_N_images` (the sphere sets 0 and `restore_default_images` sets it back).

Kernel (solvers/methods/ImageMethod.f90:dipole_pressure_images). For a block of observers, the sum (5.1) over the list:

$$\hat{p}(\boldsymbol{x}_k, f) = \sum_j s_j\, \frac{\boldsymbol{F}_j(f)\cdot(\boldsymbol{x}_k - \boldsymbol{x}_j)}{4\pi R_j^2} \left(\frac{1}{R_j} - \mathrm{i}k\right) \mathrm{e}^{\mathrm{i}kR_j}, \qquad R_j = \max(|\boldsymbol{x}_k - \boldsymbol{x}_j|, \epsilon). \tag{14.2}$$

The loops are sources, observers and, innermost, frequencies: the factors of each pair are computed once, the frequency loop has unit stride and is vectorised (`!$OMP SIMD`), and $\mathrm{e}^{\mathrm{i}kR}$ is evaluated as $\cos kR + \mathrm{i}\sin kR$. Every observer is summed over the sources in the same order whatever block it falls in, so the result does not depend on the block size or on the number of threads (checked bit by bit: Section 15.4).

Parameters saved : `N_images`, `c_wat`, `rho_wat`, `Lower_HBC` ($-H$), `p_ref`; the method tag is `_IM<N_images>`.

14.3 Analytical normal modes

Modes (solvers/methods/AnalyticalNormalModes.f90:Psi_kzm, `dPsi_kzm_dz`). With $k_{zm} = (2m-1)\pi/(2H)$,

$$\Psi_m(z) = \sqrt{\frac{2\rho}{H}}\sin(k_{zm}z), \qquad \Psi'_m(z) = \sqrt{\frac{2\rho}{H}}\,k_{zm}\cos(k_{zm}z), \tag{14.3}$$

normalised as $\int_{-H}^{0} \Psi_m^2/\rho\,\mathrm{d}z = 1$; $\Psi_m(0) = 0$ and $\Psi'_m(-H) = 0$.

Wavenumbers. $k_{rm} = \sqrt{k^2 - k_{zm}^2}$ when $k^2 \geq k_{zm}^2$, else $\mathrm{i}\sqrt{k_{zm}^2 - k^2}$. At a cut-off ($k = k_{zm}$) the Hankel functions are singular, so $|k^2 - k_{zm}^2|$ is kept at least `CUTOFF_GAP` $\cdot\, k_{zm}^2 = 2\times10^{-6}k_{zm}^2$ (a relative distance 10^{-6} in k).

Kernel. solvers/methods/AnalyticalNormalModes.f90:dipolar_pressure_NM sums, with the horizontal distance $r = \max(\sqrt{\Delta x^2 + \Delta y^2}, \epsilon)$,

$$\hat{p} = \frac{\mathrm{i}}{4}\sum_{m=1}^{M}\Psi_m(z)\left[\frac{F_z\,\Psi'_m(z_s)}{\rho}\,H_0^{(1)}(k_{rm}r) + \frac{\Psi_m(z_s)\,k_{rm}}{\rho}\,H_1^{(1)}(k_{rm}r)\,\frac{F_x\Delta x + F_y\Delta y}{r}\right] \tag{14.4}$$

with $\Delta x = x - x_s$, $\Delta y = y - y_s$ (the factor i/4 is applied once at the end, `p_out = 0.25 * I1 * p_out`). It is the dipole $\hat{p} = -\boldsymbol{F}\cdot\nabla G = \boldsymbol{F}\cdot\nabla_{\boldsymbol{x}_s} G$ of Section 1.3 with the modal Green's function of this waveguide, $G = \frac{\mathrm{i}}{4\rho}\sum_m \Psi_m(z)\Psi_m(z_s)H_0^{(1)}(k_{rm}r)$, which solves $\nabla^2 G + k^2 G = -\delta$ [5] like the free-field G of (1.4); the horizontal term comes from $\partial_{x_s} H_0^{(1)}(k_{rm}r) = k_{rm}H_1^{(1)}(k_{rm}r)\,\Delta x/r$. Section 9.2 checks it against the image method. The Hankel functions of complex argument come from utils/SpecialFunctions.f90:compute_hankel_complex.

Loops. Per block, k_{zm}, Ψ_m at the observers, Ψ_m and Ψ'_m at the sources and k_{rm} for every (mode, frequency) are tabulated once; then the loops are observers, sources, frequencies and, innermost, modes, with no `SIMD` directive.

Truncation. The modal sum stops at the first evanescent mode whose decay exceeds that of the dominant term by `EVAN_CUTOFF` $= 40$ nepers, $(\kappa_m - \kappa_{\mathrm{ref}})\,r > 40$, since κ grows with m. With `Nmodes` $= -999$, $M = \lfloor \frac{1}{2}(4Hf_{\max}/c + 1)\rfloor + 5$, the propagating modes at the highest frequency and five more (`init_AnalyticalNormalModes`).

> **Note** The factor is $+\mathrm{i}/4$: the one of the Green's function above for the time factor $\mathrm{e}^{-\mathrm{i}\omega t}$, and the one that agrees with the image method. The docstring of `dipolar_pressure_NM` said $-\mathrm{i}/4$ until October 2026, when the comment was corrected.

Parameters saved : `Nmodes`, `H`, `c_wat`, `rho_wat`, `Lower_HBC`, `p_ref`; the method tag is `_ANM<Nmodes>`; `print_mode_summary` prints the table of Listing 2.1.

14.4 The water column

The surface is the plane $z = 0$ for every method. The seabed is published by the method through two queries of `AcousticSolver_t`:

- `z_seabed(x, y)`, by default `z_seabed_base`: $-H$;
- `in_water(points)`, by default `in_water_base`: $z_{\text{seabed}}(x, y) - 10^{-9} \leq z \leq 10^{-9}$, for observers or sources.

Every use goes through them: the mask of `compute_pressure_masked`, `check_sources_in_water` (called by every `init`), and the automatic depths of the steps (Section 6.1), which ask $z_{\text{bot}} = z_{\text{seabed}}$ at their reference point and use 0, z_{bot} and $z_{\text{bot}}/2$. No step reads `depth_base` directly. A method with a variable bathymetry $H(x, y)$ only overrides `z_seabed`; the mask, the check of the sources and the defaults of the steps then follow it (Section 17.3).

Chapter 15

The computation engine

This chapter follows the solver from `run()` down to the kernel: how a step builds its observers, how the masked observers are left out, and how the observers are split into blocks and threads. The program flow above it is in Section 11.3.

15.1 The step sequence

solvers/AcousticSolver_steps.f90:run counts the enabled `do_*` flags, prints the banner `RUNNING ACOUSTIC POST-PROCESSING (<n> steps)` and runs the enabled steps in a fixed order: spectrums, polar, decay, line, sphere, sliceXY, sliceXZ, sliceVertical, cylinder. Each one is preceded by a header `[k/n] <title>` (`print_step_header`). The order of the flags in the namelist does not matter. Two steps may switch themselves off while running, so that the `.dat` does not report them: `run_sphere` of a method without a sphere (the parent's `dummy_run_sphere`, which prints `>> [Sphere] skipped, not available for: <method>`), and `run_sliceVertical` when `sliceXZ` already covers the same plane. Which steps make sense for each emitter is decided by the parser through the flags (Section 6.3); `run()` itself does not know the emitter.

15.2 Anatomy of a step

Every `run_*` of `solvers/AcousticSolver_steps.f90` has the same structure; Listing 15.1 is `run_polar`, shortened.

Listing 15.1: The structure of a step: `run_polar`, shortened.

```
bs = self % block_size

! Defaults (resolved once, persisted back into run_inputs)
associate(pp => self % run_inputs % params_polar)
    if (pp % r_polar == DEFAULT_R) pp % r_polar = 500.0_WP
    if (pp % n_polar == DEFAULT_I) pp % n_polar = 72_I32
    if (all(pp % center_polar == DEFAULT_R)) pp % center_polar = self % sources %
    ↪ geometry % center

    ! Water column at the circle centre
    z_bot = self % z_seabed(pp % center_polar(1), pp % center_polar(2))
    if (pp % z_polar == DEFAULT_R) pp % z_polar = 0.5_WP * z_bot
    ...
end associate

! Build observers array (n_theta, 3)
```

```
...
write(*, ...) '>> [Polar]    r=', r, ...

! Compute pressure
CALL self % compute_pressure_masked('Polar', observers, bs, p)
if (self % debug) write(*, '(A,F10.4)') '   Pressure norm : ', pressure_norm(p)

! Save data
CALL save_variable(self % save_path, "Freqs", self % sources % Freqs)
CALL save_variable(self % save_path, "P_polar", p)
...
```

1. **Defaults.** The parameters left at `DEFAULT_R`/`DEFAULT_I` are resolved inside an `associate` on `run_inputs % params_<step>`, so the resolved values are written back and `update_parameters` reports them in the `.dat`. The depths come from `z_seabed` at the reference point of the step (Section 14.4).
2. **Observers**, an array $(N_{\text{obs}}, 3)$. Each grid has its own flattening order: x fastest in `sliceXY` and `sliceXZ` and the azimuth fastest in the sphere (the first index of the saved grid), the depth fastest in the cylinder and `sliceVertical` (the second index).
3. **One line** `>> [<Step>]` with the geometry and the number of observers.
4. **Pressure** through `compute_pressure_masked`, never `compute_pressure` directly; with `debug` the norm of $|\hat{p}|$ over the computed entries (`pressure_norm`, which leaves the NaN out).
5. **Save**: `Freqs`, the pressure and the geometry (Appendix D). The grids are reshaped first: the pressure to (N_f, n_1, n_2) and the observers to $(n_1, n_2, 3)$. The line `Saved to: <path>` closes the step.

15.3 Masked observers

solvers/AcousticSolver.f90:compute_pressure_masked(`label, observers, block_size, p`):

1. `check_observers_distances` flags the observers closer than `mask_distance` to any source (squared distances, an OpenMP loop over the observers that stops at the first source too close), and `in_water` the observers outside the water column;
2. if no observer is masked, it calls `compute_pressure` on the whole array and returns: the result is the same as without the wrapper;
3. otherwise it prints one `WARNING [<label>] <n> of <N> observers ...: masked (NaN)` line per criterion, fills the result with `cmplx(NaN, NaN)`, packs the valid observers into a new array, calls `compute_pressure` on it and scatters the result back.

The grids keep their shape and the masked points appear as holes in the plots (Section 6.2). Packing changes the blocks, so with the normal-mode method the last bits of the valid observers can differ from those of a run without masked observers (Section 15.4).

15.4 Blocks and threads

Both methods split the observers in the same way (`compute_pressure` of each method):

1. the block size b is `block_size` (128 in the shipped `Solver.nml`), clamped to

$$\max\left(1, \min\left(b, \lceil N_{\text{obs}}/N_{\text{threads}} \rceil\right)\right),$$

so that a small set of observers still gives one block to every thread;

2. with `in_cluster = .true.`, an `!$OMP PARALLEL DO` over the blocks: each block calls the kernel on its slice `observers(istart:iend, :)` and adds its result into the columns `istart:iend` of the total, which no other block touches; there is no reduction across threads;
3. with `.false.`, a serial loop over the observers one by one (block size 1), with a line `Progress: i/N` every 100 observers.

The memory of a block is its pressure (N_f, b) plus, for the normal modes, its tables: the modal shapes at its observers (b, M) and at the sources (N_s, M), and k_{rm} (M, N_f); the sources themselves are shared. The image system is built once in `init`, $N_s(1+2N)$ positions and complex forces $(N_f, N_s(1+2N), 3)$, and kept for every step: memory is spent to avoid rebuilding it.

Reproducibility. Measured with the shipped `FrequencySource` case of `input_files/` (Table 15.1): a run is deterministic, and the image method gives the same bits for any block size and number of threads, because each observer is summed by one thread over the sources in a fixed order. The normal-mode method differs in the last bits when the blocks change: with `AnalyticalNormalModes.f90` alone compiled with `-fno-tree-vectorize` the differences disappear, so they come from the automatic vectorisation of that module (most likely of the modal shapes tabulated over the observers of each block, where the elements handled by the vector loop and by its scalar remainder depend on the block length). The number of threads changes the blocks through the clamp above. Bit-for-bit comparisons (Section 18.1) therefore use the same `block_size` and `OMP_NUM_THREADS` on both sides.

Table 15.1: Bits of the pressure against the run with `block_size` = 128 and one thread (shipped `FrequencySource` case with its steps; the clamp makes the blocks of the 4-thread runs differ from the 1-thread ones).

Method	`block_size`	Threads	Result
IM	1, 7	1	identical (69 datasets)
IM	128, 3	4	identical
ANM	1, 7	1	pressures differ; largest difference 3.6×10^{-16} of the peak $\lvert\hat{p}\rvert$
ANM	128, 3	4	pressures differ; largest 1.2×10^{-16} (128) and 3.6×10^{-16} (3)
ANM, module without vectorisation	1, 7; 128, 3	1; 4	identical (63 datasets)
both	3	4	two runs with the same settings: identical

15.5 A whole-step override: the sphere

`run_sphere` is the only step a method overrides. The parent's binding is `dummy_run_sphere`; the image method binds its own solvers/methods/ImageMethod.f90:run_sphere, which sets `N_images` to 0 with `set_N_images` (rebuilding the image system without images), computes the free-field sphere of Section 6.3.9 and calls `restore_default_images`, which rebuilds the system with the original `N_images` only if it was changed. The sphere cannot be a generic step because `AcousticSolver.f90` cannot `USE ImageMethod` (that would be a cycle); a free-field kernel shared by every method would make it generic, and would change the outputs (Chapter 19).

Chapter 16

The output layer

The user's view of the outputs is in Chapter 7. This chapter is about the code that writes them: `utils/HDF5Utils.f90`, the only module that uses HDF5, and the submodule `solvers/AcousticSolver_output.f90`, which manages the file of a run.

16.1 Sequence of a run

Call (from)	What it does to the files
`build_save_path` (`init`)	`resolve_save_path(tag)`: builds the name, creates the folder, deletes a stale `.dat.prev`; an existing HDF5 file is deleted, or kept with its `.dat` renamed `.dat.prev` when `update_existing`
`save_parameters` (`main`)	`check_update`; the datasets of the method; `save_shared_parameters`; `emitter % save_params`; `write_txt_summary` (the `.dat` header, `status='replace'`)
`run` (`main`)	each step writes `Freqs` and its own datasets
`update_parameters` (`main`)	appends the post-processing block and the final separator to the `.dat`; deletes `.dat.prev`

16.2 HDF5Utils

`save_variable(path, name, var)` takes `CLASS(*)` of assumed rank, selects its rank (0 to 3) and type, and calls the matching specific of the generic `save_to_hdf5`: `REAL(WP)`, default `INTEGER` and `COMPLEX(WP)` of ranks 0 to 3, plus `LOGICAL` and `CHARACTER` scalars. Any other type stops with `save_variable: unsupported type for dataset <name>`, any other rank with `unsupported rank`. Every specific:

1. opens the file read-write, or creates it;
2. deletes the dataset if it exists (`h5ldelete_f`) and creates it again;
3. writes `H5T_NATIVE_DOUBLE` for reals, `H5T_NATIVE_INTEGER` for integers and logicals (1/0), and a fixed-length string of the trimmed length (at least 1) for characters; a complex array is copied into a real array with a leading axis of 2 (real, imaginary); a scalar is a 1-element dataset;
4. closes the dataset, the space, the file and the library (`h5close_f`).

Opening and closing the file on every call keeps the module stateless; the cost is one open per dataset. The writers do not check the error codes of the HDF5 calls.

`read_hdf5_real_1d(path, name, data, found)` reads a rank-1 real dataset; `found` is `.false.` if the file, the dataset or rank 1 is missing. `check_update` uses it for `Freqs`; it is the only reader.

File size. A dataset deleted and written again does not always give its space back: three runs of the shipped `FrequencySource` case on the same file, the last two updating it, left 852 396, 853 052 and 853 380 bytes. Compare outputs dataset by dataset, never as files.

16.3 The `.dat` file

`write_txt_summary` of the method writes the header with `status='replace'`: the title and the HDF5 path, `emitter % write_txt(u)`, the `ENVIRONMENT` block (speed of sound, density and depth with `F8.2`, reference pressure with `ES10.2`) and its own `SOLVER` block:

- image method: `N images`, `Lower HBC`, `Upper atten.`, `Lower atten.`, `Block size`, `In cluster`;
- normal modes: `N modes`, `Auto N modes`, `Height H`, `Lower HBC`, `Epsilon` (`ES10.2`), `Block size`, `In cluster`.

`update_parameters` appends `POST-PROCESSING STEPS EXECUTED` and one block per step in the order spectrums, polar, cylinder, decay, line, sliceXY, sliceXZ, sliceVertical, sphere (not the order in which `run()` executes them). A block is a header line `<Step>:` and lines indented by three spaces; it ends at the first blank or non-indented line, which is how `write_previous_block` finds it in `.dat.prev`.

16.4 Updating a file

solvers/AcousticSolver_output.f90:check_update runs first in `save_parameters`, so nothing has been written yet. If the HDF5 file exists:

1. it reads the old `Freqs`: missing, of another size, or with $\max|f_{\text{old}} - f| > 10^{-9} \max|f|$ gives the reasons *no Freqs in the file*, *different number of frequencies*, *different frequencies*;
2. without `.dat.prev` the reason is *no previous .dat to compare with*; otherwise it writes the new header (`write_txt_summary`) and `compare_dat_headers` compares it with the header of `.dat.prev`: the lines before `POST-PROCESSING STEPS EXECUTED`, without the dashed separator before it and without the lines starting with `Block size` or `In cluster`, read into 256-character buffers; the first different line gives *different setting: "<old>" -> "<new>"*, a different count *different .dat header length*;
3. with a reason it prints `WARNING check_update: <file> not updated (<reason>): deleted and rebuilt` and deletes the HDF5 file and `.dat.prev`; otherwise `---> Same case: only the steps run now are rewritten in <file>`.

The header is therefore the definition of "the same case": whatever changes the results and is not in it at enough precision is not seen.

> **Pitfall** Checked with the shipped `FrequencySource` case and the image method (all steps, then `polar` only with one setting changed): the second run reported *Same case*, rewrote `P_polar` with the new setting and kept the other steps computed with the old one, for each of these changes:
>
> - `UpperBC` $-1 \to 1$ and `LowerBC` $1 \to -1$, which the `SOLVER` block of the image method does not print;

- `epsilon` of the image method ($10^{-6} \rightarrow 900$ m), not printed either;
- `mask_distance` ($1 \rightarrow 900$ m), which no `.dat` prints;
- `depth_base` $30.0 \rightarrow 30.004$ m, printed as `30.00` in both runs.

Until these settings are in the header at full precision, delete the output (or use `update_existing` = `.false.`) after changing any of them.

16.5 Rules for new outputs

- Dataset names, the `.dat` format and the namelist names are an interface: the plotting reads them and `check_update` compares the header. They are never renamed; new ones are added.
- A new dataset of a step goes in its `run_*`, after `compute_pressure_masked`; a new parameter of a step gets a line in its block of `update_parameters`.
- Every input that changes the results must appear in the header (in `write_txt` of the emitter or in `write_txt_summary` of the method), at full precision or as a checksum at full precision, as the manual sources do for their forces (Section 7.3).
- A new dataset that the plotting needs must be listed in Appendix D, in `ARCHITECTURE.md` §6.1, and read in `plotting/readers/`.

Chapter 17

Extending the code

Each section is a checklist of every file that a kind of extension touches. The compiler finds a missing deferred binding, but not a missing `CASE`, a missing Makefile rule order or a missing call that the contract asks for; the items marked *(not enforced)* are those. After the change, follow the protocol of Section 18.1. `fpm build` finds a new source file by itself; the Makefile lists every object.

17.1 A turbine type

1. `src/emitters/TurbineTypes/X.f90`: `X_t EXTENDS(WindTurbine_t)` with its geometry and the deferred bindings of `WindTurbine_t` (`excitation_spectrum` included). Its `init` sets `path_rpm` (and any data path) and `is_monopile`, and calls `CALL self % init_base(config)` *(not enforced)*.
2. A new drivetrain spectrum goes in `emitters/DrivetrainExcitation.f90`, called from `excitation_spectrum`.
3. One `CASE` in `new_turbine` (`emitters/TurbineFactory.f90`) *(not enforced: an unknown `case_ type` stops at run time)*.
4. Optionally its short label in `type_alias` (`emitters/WindFarm.f90`), for the default name of a farm file.
5. Makefile: the object in `OBJS` and its rule, before `TurbineFactory`.

17.2 An emitter

Point sources given in a new way (for example an input HDF5 file): extend `ManualSources_t`.

1. `src/emitters/X.f90`: `X_t EXTENDS(ManualSources_t)` with `init(self, config)`, which fills `Nsources`, `xyz`, `order`, `Freqs`, `Force` and sets `kind_name = 'X'`, `title` and `name_suffix`; `to_source`, the common datasets and the positions in the `.dat` come from the parent.
2. Optionally `save_params` and `write_txt`, which call `save_params_base` / `write_txt_base` first and add what is its own. `write_txt` must list every input value that matters, or checksums of it at full precision: `check_update` compares this block, and an input missing from it would reuse an old output after being changed *(not enforced)*.
3. `get_enbw`, if its forces come from `compute_rfft` (Section 13.3).

4. Its configuration type in `types/ConfigTypes.f90` and its field in `Config_t`; its reader in `input/InputEmitters.f90`, reusing `read_source_file`, `check_source_positions`, `check_file_exists`, `read_table` and `compile_expression`; its `CASE` in `read_input_files` (`input/Args_parser.f90`).
5. Its input files `Emitters/X.nml` and `ComputationSetups/X_CS.nml`, named exactly as its `simulation_type` (the parser builds both names from it).
6. Its branch in `app/main.f90`: a variable of its concrete type (the abstract `active_emitter` can neither be allocated nor call `init`), `init`, `to_source`, and `MOVE_ALLOC` into `active_emitter` at the end of the branch.
7. Makefile: the object in `OBJS` and its rule after `ManualSources`, with every module it uses among the prerequisites.
8. Plotting: `readers/hdf5_io.EMITTERS` maps its `Emitter` marker to the script that reads it (`'source'` for the manual sources); what is specific to it goes in `readers/source.py` and `wrappers/source.py`.

Any other emitter (a new kind of structure): `X_t EXTENDS(Emitter_t)` with `to_source`, `save_params` (which writes `Emitter = 'X'` and everything the plotting needs), `write_txt` (with the rule of item 2) and, if needed, `get_enbw`; then items 4 to 8, with a reader and a wrapper of its own in `plotting/`. If some steps make no sense for it, `read_do_config` (`input/InputPostProcessing.f90`) switches them off, as it does with `sphere` and `decay` for a farm.

A new input mode of the manual sources follows the pattern of Section 3.4.3: one more variable of the namelist with its sentinel, one more entry in the list of ways of its item in the reader (two ways of giving the same item stop the run, with a message built by `join_given`), and the resolution in `init` of the emitter. The same physical input must give bit-identical datasets in every mode, and `write_txt` must show the checksums of the input when it is not written by hand.

17.3 An acoustic method

1. `src/solvers/methods/X.f90`: `X_t EXTENDS(AcousticSolver_t)` with the six deferred bindings of Section 14.1; `save_parameters` calls `check_update` first and `save_shared_parameters` *(not enforced).*
2. Its `init` calls `CALL self % check_sources_in_water()` once the environment of the method is set (its seabed included) and before anything is saved *(not enforced: without it a source below the seabed is computed and gives finite but wrong pressures).*
3. Its `write_txt_summary` prints every setting that changes the results, at full precision *(not enforced; Section 16.4 shows what happens otherwise).*
4. `Config_X_t EXTENDS(Config_Method_t)` in `types/ConfigTypes.f90`.
5. `read_X_config` in `input/InputSolver.f90`, its `CASE` in `read_input_files` (`input/Args_parser.f90`), which allocates `config % method` with the new type, and its file `AcousticMethods/X.nml`, named exactly as its `method`.
6. Its `CASE` in `new_solver` (`solvers/SolverFactory.f90`).
7. Makefile: the object before `SolverFactory`.

The `Method` dataset is written by `save_shared_parameters`. The free surface is always the plane $z = 0$: the queries, the readers ($z \leq 0$) and the steps assume it. A method with its own seabed overrides `z_seabed` only (Section 14.4); it never overrides the `run_*` steps, except as the image method does with its sphere (Section 15.5).

17.4 A post-processing step

1. `run_X` in `solvers/AcousticSolver_steps.f90` with the structure of Section 15.2; its interface body and binding in `solvers/AcousticSolver.f90`; its call in `run()`.
2. `Config_X_t` and the flag `do_X` in `Config_DoRun_t` and `Config_Computation_t` (`types/ConfigTypes.f90`).
3. `read_X_config` and the flag in `read_do_config` (`input/InputPostProcessing.f90`), and its namelist group in every `ComputationSetups/*_CS.nml` *(not enforced: the group is read only when the step runs, so a file without it fails only then).*
4. Its block in `update_parameters` (`solvers/AcousticSolver_output.f90`), with `write_previous_block` for updates.
5. Its plots in `plotting/`.

Chapter 18

Verification and style

18.1 The verification protocol

Any change that is meant not to change the results (a refactor, a style change, a new input mode that must reproduce an old one) is accepted only if the outputs stay bit-identical. The protocol, from `ARCHITECTURE.md` §8 and `Programming_style_guidelines.txt` §8:

1. **Reference.** Keep a copy of the original sources and build the reference binary from them. Run everything in an isolated copy, never in `FORTRAN/` or in its output folders: a run deletes and rewrites the outputs of the same name.
2. **Builds.** `make clean && make -j8`; `fpm build --flag "-O3 -fopenmp"`; `make clean && make FFLAGS="-O3 -fopenmp -Wall"`, which must give no warning that the reference does not give. Keep `-fopenmp` when overriding `FFLAGS` (Section 2.2). The accepted `-Wall` warnings are the nine `-Wunused-dummy-argument` of bindings that ignore arguments by design: `get_enbw` of `Emitter_t`, `WindTurbine_t`, `WindFarm_t` and `TimeSource_t` (`self`); `z_seabed_base` (`x`, `y`); `to_source_ManualSources` (`config`); `excitation_spectrum` of `DTU10MWMonopile_t` and `DTU10MWFloating_t` (`self`).
3. **Debug build.** `make FFLAGS="-O0 -g -fopenmp -fcheck=all -fbacktrace"` must run the cases without a runtime error.
4. **Cases**, with `OMP_NUM_THREADS=1` and the same `block_size` on both sides (Section 15.4): the turbine and farm cases of the refactor harness (DTU 10 MW monopile with all steps and both methods, an `update_existing` sequence, the floating DTU, SAITEC and a two-turbine farm), which need the OpenFAST data, and the shipped `FrequencySource` and `TimeSource` cases with both methods, which need nothing else and are the quick regression of every change in the input, emitter or solver layers.
5. **Acceptance.** Every HDF5 dataset of the reference exists with the same type, shape and bytes, with no extra dataset except those the change adds on purpose; the `.dat` files are identical; the standard output is identical except the line `Total elapsed time`. Compare datasets, not files (Section 16.2). A difference is never accepted: its cause is found.
6. **Rebuild** after editing a module before running: `make` follows the dependencies, and a binary older than its sources has already cost a false alarm (a "segfault" that was a `main.f90` with the old `simulation_type` labels).

> **Note** `Programming_style_guidelines.txt` §8.3 says that multi-threaded runs are not bit-reproducible from run to run. Measured with the shipped `FrequencySource` case, they are (Table 15.1): the blocks are independent and there is no reduction across threads. What changes the last bits is a change of the blocks, and only in the normal-mode method. Running

with one thread remains the safe choice for comparisons.

18.2 Style rules

`Programming_style_guidelines.txt` is the reference; every new or changed line of Fortran follows it. Its golden rule: a style change never changes a calculation, the program flow, or a byte of the outputs, and the HDF5 names, the `.dat` contents and the namelist names are never renamed. The rules applied most often:

Topic	Rule
Case	declarations and structure in UPPERCASE (`SUBROUTINE`, `REAL`, `INTENT`, `CALL`, `ALLOCATE`, `SELECT CASE`); control flow, I/O and intrinsics in lowercase (`if`, `do`, `write`, `size`, `trim`); OpenMP directives and clauses in UPPERCASE; kind suffixes `_WP`, `_I32`
Layout	4 spaces, no tabs, no trailing blanks, one final newline; `CASE` at the level of its `SELECT`; at most 120 characters per line; continuations aligned with the first argument
Blank lines	none between a procedure line and its docstring; one after it; exactly two between the last declaration and the first executable statement; two between procedures; one before `END MODULE`
Spacing	`self % a % b`; `x**2` without spaces, other operators with; a space after every comma except in `write(*,*)`, shape specs and formats; aligned `=` in related assignments
Declarations	"comma-aligned" blocks (first comma and `::` aligned); attribute order type, `INTENT`, `OPTIONAL`, `ALLOCATABLE`; every dummy and component with a comment `! [unit] Description`; locals after `! Local variables`; `INTEGER(I32)` except in the HDF5 routines
Literals	`1.0e-6_WP`, never `1e-6_WP`; `_I32` on integer literals in assignments, comparisons and arguments, not in loop bounds, subscripts or intrinsic arguments; `[ ]` constructors; `write`, never `print`; `end if`, `end do`
Comments	English; docstrings inside every procedure and type, between lines of 73 dashes, without argument sections; file headers of 79 characters; level-2 separators `! --- 1. Title --- !`; printed strings only get spelling fixes

18.3 Pitfalls met in this code

- **Parent bindings of an abstract type.** gfortran 13 rejects `CALL self % Parent_t % proc()` when `Parent_t` is `ABSTRACT` (*Base object for type-bound procedure call at (1) is of ABSTRACT type*); the parent publishes `proc_base` aliases instead (Section 11.2).
- **Namelists.** A `/` outside a string and outside a `!` comment ends the group (a `//` written as a comment does); the final `/` needs a newline after it; an unknown variable stops the read with *Cannot match namelist object name* (Section 10.2).
- **Character array constructors.** `buffer = [buffer, line]` with elements of different lengths is accepted at `-O3` and stops a `-fcheck=all` build with *Different CHARACTER lengths (6/256) in array constructor*; `read_header_lines` hit it on every update. The typed constructor `[CHARACTER(len=256) :: buffer, line]` is the fix, in the code since October 2026.
- **`SELECT CASE` without `CASE DEFAULT`** leaves the variables it sets possibly uninitialised for `-Wmaybe-uninitialized`; a `CASE DEFAULT` with `error stop` removes the warning and catches the impossible case.

- **Vectorised intrinsics.** At `-O3 -fopenmp` array `sin`, `exp`, `**` use vector versions whose last bit can differ from the scalar ones: compare a Fortran expression with the parser only when both are compiled alike (Section 9.4), and the normal-mode results only with the same blocks (Section 15.4).
- **The `.dat` header defines the case**: a setting that changes the results and is missing from it breaks `update_existing` (Section 16.4).

Chapter 19

Roadmap

The plan agreed on 2 October 2026 (`ARCHITECTURE.md` §7) and its state on 8 October 2026 (`NEXT_STEPS.md`). Nothing in this chapter is implemented unless it says so.

19.1 Manual sources

Done. `ManualSources_t` with `FrequencySource` and `TimeSource`; one multipole-order label per source (`Source_order`); the input modes of Sections 3.4.3 and 3.4.4, which already remove the limit on the number of sources and frequencies; the plotting of both.

Open design point: the monopole strength. Which quantity defines a monopole (volume velocity, or pressure at 1 m) and where it is stored. The decision was a label per source and no new arrays, so a convention is needed, for example the strength in `Force(:, s, 1)` with components 2 and 3 unused. The readers write `SOURCE_DIPOLE` for every source today, and both methods stop in `init` on any other order.

Work items, in order.

1. **Monopole kernels**: the images of a monopole in the image method and its modal sum in the normal modes; each method replaces its dipole-only check by a check of the orders it supports; an input for the order of each source (a column of `source_file`, or the HDF5 input below).
2. **An input HDF5 file** with `xyz`, `order`, `Freqs` and `Force`, as a new child of `ManualSources_t` (Section 17.2). `HDF5Utils` needs readers of real 2-D, integer 1-D and complex 3-D datasets (it has `read_hdf5_real_1d` only). The item is about size and convenience: the text files already have no limit.
3. **Verification**: the refactor cases stay identical; the dipoles of a turbine, exported and read back as manual sources, reproduce its pressures (the corrected forces handed to the solver are not saved today, so they must first be written out); a free-field monopole and dipole match the closed form; the two methods agree for monopoles as they do for dipoles.

19.2 Methods with a non-uniform environment

- `Config_Environment_t` gains optional arrays: $c(z)$, $\rho(z)$, attenuation, bathymetry $H(x, y)$, sediment. It keeps the base scalars, so the two present methods keep working; each method checks in `init` that the data it needs is present.

- The boundaries belong to each method: constants in the image method and the normal modes, arrays in a more complex method. A method publishes its seabed by overriding `z_seabed` (Section 14.4); the masks, the check of the sources and the defaults of the steps follow it.
- Adding the method follows Section 17.3. Overriding a whole step stays the exception.

19.3 The sphere

The sphere stays a step of the image method only (Section 15.5). A free-field kernel shared by every method would make it generic; it would change the outputs, so it is a future decision.

19.4 Items found while writing this guide

These were found by reading the code and checking it with runs. The calculation code was not changed; the first two items are listed for a decision in `NEXT_STEPS.md` §3.

- `update_existing` does not see the settings missing from the `.dat` header (Section 16.4).
- The normal-mode results depend in their last bits on the observer blocks (Section 15.4).
- Comments that disagreed with the code (the factor of the normal-mode sum, Section 14.3; the automatic radius and centre of the sphere; the unit of `f_farfield_sphere` and of the `epsilon` of the image method; the sign of $\boldsymbol{F} = m\boldsymbol{a}$ of the monopile; the order of `zlim_xz`) were corrected in October 2026; only the comments changed.

Appendix A

Symbols

Table A.1: Symbols used in this guide, with their unit and the input or code name when there is one.

Symbol	Unit	Meaning
a_s, a_b	–	attenuation per reflection at the surface and at the seabed (`attenuation_surface`, `attenuation_bottom`); a_p for the plane p in Section 14.2
B_p	–	boundary code of the plane p: -1 pressure release, $+1$ rigid (`UpperBC`, `LowerBC`)
c	m/s	speed of sound (`speed_sound_base`)
f	Hz	frequency; f_m the cut-on frequency of mode m
Δf	Hz	bin width of a spectrum
$\boldsymbol{F}$, $\hat{F}$	N	complex peak amplitude of the force on the fluid, $\mathrm{e}^{-\mathrm{i}\omega t}$
G	1/m	Green's function, $\nabla^2 G + k^2 G = -\delta$
H	m	water depth (`depth_base`); the seabed is $z = -H$
$H_n^{(1)}$	–	Hankel function of the first kind of order n
i	–	imaginary unit (`I1` in the code)
k	1/m	acoustic wavenumber ω/c
k_{zm}, k_{rm}	1/m	vertical and horizontal wavenumbers of mode m, $k_{rm}^2 = k^2 - k_{zm}^2$
κ_m	1/m	decay rate of an evanescent mode, $k_{rm} = \mathrm{i}\kappa_m$
L, L_p	dB	sound pressure level re p_{ref}
M	–	number of modes (`Nmodes`)
N	–	number of images on each side (`N_images`)
N_s, N_f, N_t, N_{obs}	–	number of sources, frequencies, time samples and observers
$\hat{p}$	Pa	complex peak amplitude of the pressure, $\mathrm{e}^{-\mathrm{i}\omega t}$
p_{ref}	Pa	reference pressure (`reference_pressure`, $1\,\mu$Pa)
R, $\hat{\boldsymbol{r}}$	m, –	distance from a source to an observer and the unit vector along it
r	m	horizontal distance from a source to an observer (normal modes)
s_j	–	sign of image j, the product of the boundary codes of its reflections
t	s	time
W	W	radiated power through a closed surface
$\boldsymbol{x}$, $\boldsymbol{x}_s$	m	observer and source positions; z_s the height of a source

Symbol	Unit	Meaning
x, y, z	m	global frame; z upwards, $z = 0$ the free surface
α	dB/km	absorption (Thorp) used by the plots
ϵ	m	floor of the distance in the kernels (`epsilon`)
θ	deg	azimuth, anticlockwise from $+x$
ρ	$\mathrm{kg/m^3}$	density of the water (`density_base`)
Ψ_m	$\sqrt{\mathrm{kg}}/\mathrm{m}^2$	shape of mode m, $\int_{-H}^{0} \Psi_m^2/\rho \, \mathrm{d}z = 1$
ω	rad/s	angular frequency $2\pi f$
ENBW	bins	equivalent noise bandwidth of the window of the forces (1.5 for Hann)

Appendix B

Glossary

Bin One frequency of a spectrum; for the forces of an FFT, a multiple of $\Delta f = 1/T$ with T the length of the record.

Block
A set of observers handed to the kernel at once, `block_size` at most; blocks are the unit of parallel work (Section 15.4).

Case What an output file holds: one emitter, one environment and one method with their settings. Two runs are the same case when their `Freqs` and their `.dat` headers agree (Section 16.4).

Cut-on frequency
The frequency above which a mode propagates, $f_m = (2m-1)c/(4H)$ (Section 5.2).

Dipole
A point source of force: the field of a force $\boldsymbol{F}$ applied to the fluid at a point, (1.4). Every source of this version is a dipole.

Emitter
The object that produces the sources: a turbine, a farm, or a set of manual sources (Chapter 4).

ENBW
Equivalent noise bandwidth of a window, in bins: 1.5 for the Hann window of `compute_rfft` (Section 6.4).

Evanescent mode
A mode below its cut-on frequency; it decays as $\mathrm{e}^{-\kappa_m r}$ instead of propagating.

Free field
Water without boundaries; the image method with `N_images = 0`.

Header
The part of the `.dat` before `POST-PROCESSING STEPS EXECUTED`: emitter, environment and method.

Image
A mirrored copy of a source across the surface or the seabed, which imposes the boundary condition of that plane (Section 5.1).

Manual sources
Point dipoles given by the user, in frequency (`FrequencySource`) or in time (`TimeSource`).

Mask
The observers that are not computed and get NaN: too close to a source (`mask_distance`) or outside the water (Section 6.2).

Method
The acoustic propagation model: the image method or the analytical normal modes.

Method tag
The part of the output name that names the method and its resolution, `_IM30`, `_ANM24`.

Mode
A solution of the waveguide with a fixed shape in depth, $\Psi_m(z)$, travelling horizontally (Section 5.2).

Observer
A point where the pressure is computed.

Peak amplitude
The modulus of a complex amplitude with the time factor $\mathrm{e}^{-\mathrm{i}\omega t}$; the rms value of a tone is the peak amplitude divided by $\sqrt{2}$.

Sentinel
The value -999 (`DEFAULT_R`, `DEFAULT_I`) that marks an input left automatic; the step that uses it resolves it at run time.

Step One post-processing geometry of observers (`polar`, `sliceXY`, ...) and its outputs (Chapter 6).

Waveguide
The water layer between the free surface and the seabed.

Wet node
A node of the structure below the free surface; only the wet nodes become sources.

Appendix C

Command line and error messages

C.1 Command line

Call	Effect
`./off_coustics`	reads `./input_files/`
`./off_coustics <dir>`	reads `<dir>/`, adding the final `/` if it is missing, and prints `» Reading the input files from: <dir>/`
two or more arguments	`ERROR STOP get_input_path(): Too many arguments. Usage: ./off_coustics [directory_of_the_input_files]`
an empty argument (`''`)	`ERROR STOP get_input_path(): The directory of the input files is an empty argument`
`OMP_NUM_THREADS=<n>`	number of OpenMP threads (Section 15.4)

The output paths are relative to the working directory, not to the input directory (Section 7.1).

C.2 How a run stops

Every error stops the run at once, with exit status 1. The message has one of two forms:

- `ERROR STOP <routine>(): <message>`, from an `error stop` with a text;
- a line `ERROR: <message>` or `ERROR <emitter>: <message>`, followed by `ERROR STOP`.

Both end with `Error termination. Backtrace:` and the backtrace of the call. Lines that start with `WARNING` do not stop the run. The messages below were produced by runs of the binary with one wrong input each, on copies of `input_files/` (Section 9.4).

C.3 Messages

Table C.1: Error messages of common wrong inputs, as printed (`<...>`: the values of the case).

Wrong input	Message
Input files	
directory `./nowhere` or its `Solver.nml` missing	`ERROR: Could not open input file: ./nowhere/Solver.nml`
emitter file missing	`ERROR: Could not open input file: ./input_files/Emitters/FrequencySource.nml`

Wrong input	**Message**
unknown variable (a typo)	`ERROR: Failed to read &environment_config: Cannot match namelist object name depht_base`
no newline after the final `/`	`ERROR: Failed to read &environment_config: End of file`
group of a step that runs is missing	`ERROR: Failed to read &polar_config: End of file`
`simulation_type` unknown	`ERROR STOP read_input_files(): Unknown simulation_type: Turbine`
`method` unknown	`ERROR STOP read_input_files(): Unknown method Images`
Solver, environment, method, steps	
`mask_distance` < 0	`ERROR STOP read_solver_config(): mask_distance must be >= 0 (0 = no masking)`
`depth_base` ≤ 0	`ERROR: &environment_config: depth_base, speed_sound_base, density_base and reference_pressure must be > 0`
`Nmodes` $= 0$	`ERROR STOP AnalyticalNormalModes % init(): Nmodes must be > 0 (or -999 for automatic)`
`n_polar` $= 0$	`ERROR: n_polar = 0 (use -999 for the automatic value, or at least 1)`
a source below the seabed	`ERROR: 1 of 1 sources are outside the water column; the first one, source 1, is at z = -45.00 m (the seabed is at -30.00 m and the surface at 0 m)`
`FrequencySource`	
a file missing	`ERROR: &frequency_source_config: freq_file not found: nofile.txt`
a row with too few values	`ERROR: line 2 of re.txt has 5 values, 6 are required`
rows of a force file	`ERROR FrequencySource: force_re_file re.txt has 3 rows and the frequency axis has 4 frequencies`
two ways of one item	`ERROR: &frequency_source_config: the frequency axis is given in more than one way (freqs and freq_file): give only one of them`
positions given twice	`ERROR: &frequency_source_config: the positions are given twice, in source_xyz and in source_file`
frequencies not increasing (list)	`ERROR: &frequency_source_config: Freqs must be positive and strictly increasing`
frequencies not increasing (file)	`ERROR FrequencySource: the frequencies of freq_file f.txt must be positive and strictly increasing, and frequency 3 ( 2.00000E+01 Hz) is not`
`fmin` $\geq$ `fmax`	`ERROR: &frequency_source_config: fmax must be greater than fmin: 1.00000E+02 <= 1.00000E+02`
a grid without `Nfreqs`	`ERROR: &frequency_source_config: Nfreqs = -999 (a grid from fmin to fmax needs at least 2 frequencies)`
`freq_spacing` wrong	`ERROR: &frequency_source_config: freq_spacing = 'logarithmic' (it must be 'linear' or 'log')`
`t` in an expression of `f`	`ERROR: &frequency_source_config: force_re_expr(1, Fz): unknown name 't' (use f, pi or a function) at position 14 of 'sin(2*pi*50*t)'`
syntax error	`ERROR: &frequency_source_config: force_im_expr(1, Fy): ')' expected at position 17 of '100/(1+(f/50)**2'`

Wrong input	**Message**
an expression not finite	`ERROR FrequencySource: the real part (force_re_expr) of the force Fz of source 1 is not finite at f = 5.00000E+01 Hz`
more than 200 frequencies written	`ERROR: &frequency_source_config: force_re and force_im hold at most 200 frequencies and Nfreqs = 201: give the forces in files or as expressions of f`
more than 20 sources with expressions	`ERROR: &frequency_source_config: force_re_expr and force_im_expr hold at most 20 sources and source_file has 25: give the forces in files`
`Nsources` against `source_file`	`ERROR: &frequency_source_config: Nsources = 3 and source_file has 2 rows`
a source above the surface	`ERROR: &frequency_source_config: source 1 is at z = 5.00 m, above the free surface (z = 0)`
`TimeSource`	
time file not uniform	`ERROR TimeSource: the time file is not uniformly spaced (its time step varies by 5.00E-01 of its mean value); the FFT needs a constant time step`
time file too short	`ERROR TimeSource: the time file has 3 samples (at least 4 are required)`
`fmin`, `fmax` outside the spectrum	`ERROR TimeSource: no frequency of the spectrum ( 5.000E-01 to 5.000E+02 Hz) is inside fmin and fmax`
more than 20 sources with expressions	`ERROR: &time_source_config: force_expr holds at most 20 sources and source_file has 25: give the forces in force_file`

A `/` written as a comment (`//`) is not an error: the group ends there and the rest is silently not read (Section 10.2).

Appendix D

HDF5 datasets

Every dataset is at the root of the file. The shapes are the Fortran ones; `h5py` and other row-major readers see the axes reversed, and a complex array as real with a last axis of 2 (real, imaginary): `P_polar` (N_f, n) complex is $(n, N_f, 2)$ in `h5py` (Section 7.2). Scalars are datasets of one element; *int* is a 32-bit integer, *str* a fixed-length byte string, *flag* an integer 0 or 1. The shapes were checked on outputs of the four emitters (`FrequencySource` with both methods, the DTU monopile alone and in a farm); those of the DTU floating type come from its code.

Table D.1: Datasets of an output file, by writer.

Dataset	Fortran shape	Unit	Meaning
Method (`save_parameters`) and shared (`save_shared_parameters`)			
`Method`	str	–	`"Images Method"` or `"Analytical Normal Modes"`
`ENBW`	scalar	bins	1.5; only for forces from a windowed FFT (turbine, farm, `TimeSource`)
`c_wat`, `rho_wat`	scalar	m/s, kg/m^3	speed of sound, density
`p_ref`	scalar	Pa	reference pressure
`Lower_HBC`	scalar	m	$-H$, the seabed
`N_images`	int	–	image method: images on each side
`Nmodes`, `H`	int, scalar	–, m	normal modes: modes and depth
Manual sources (`save_params_ManualSources`, `save_params_TimeSource`)			
`Emitter`	str	–	`"FrequencySource"` or `"TimeSource"`
`Nsources`, `NFreqs`	int	–	number of sources and of frequencies
`Source_xyz`	$(N_s, 3)$	m	positions
`Source_order`	(N_s) int	–	multipole order (1, dipole)
`Source_Force`	$(N_f, N_s, 3)$ complex	N	forces handed to the solver
`Nt`, `Time_step`	int, scalar	–, s	`TimeSource`: samples and time step
`Source_time`	(N_t)	s	`TimeSource`: times
`Source_Force_time`	$(N_t, N_s, 3)$	N	`TimeSource`: force series
Turbine (`save_turbine_params`, `save_specific_params`); in a farm with the suffix `_<i>`			
`case_type`, `Case_type`	str	–	turbine type (written twice, by the two routines)
`WindSpeed`, `WindDir`	scalar	m/s, deg	wind speed and direction
`Depth`	scalar	m	water depth
`AxisPos`, `BariPos`	(2)	m	turbine axis and barycentre in (x, y)
`Structure_nodes`	$(N_{\text{members}}, N_{\text{nodes}}, 3)$	m	nodes of the structural model

Dataset	Fortran shape	Unit	Meaning
`rho_mat`, `wet_area`	scalar	kg/m^3, m^2	DTU types: material density, wetted area
`D`	scalar	m	DTU monopile: diameter
`col_D`, `XsecA`, `XsecB`, `t`	scalar	m	DTU floating: column diameter, pontoon width and height, wall thickness
`col_members`, `pon_members`	$(3,2)$, $(3,1)$ int	–	DTU floating: member indices (from 0)
Farm (`save_params_WindFarm`)			
`Nturbines`	int	–	number of turbines
`Source_offset`	$(N_{\text{turbines}}+1)$ int	–	the sources of turbine i are offset$(i)+1,\ldots,$offset$(i+1)$
Every step			
`Freqs`	(N_f)	Hz	frequencies
`run_spectrums`			
`P_spectrums`	(N_f, n) complex	Pa	pressure
`Obs_spectrums`	$(n,3)$	m	observers
`run_polar`			
`P_polar`	(N_f, n) complex	Pa	pressure
`R_polar`, `Z_polar`	scalar	m	radius, depth
`Theta_deg_polar`	(n)	deg	azimuths
`Obs_polar`	$(n,3)$	m	observers
`Center_polar`	(2)	m	centre
`run_cylinder`			
`P_cylinder`	(N_f, n_θ, n_z) complex	Pa	pressure
`R_cylinder`	scalar	m	radius
`Z_cylinder`	(n_z)	m	depths, seabed to surface
`Theta_deg_cylinder`	(n_θ)	deg	azimuths
`Obs_cylinder`	$(n_\theta, n_z, 3)$	m	observers
`Center_cylinder`	(2)	m	axis
`dA_cylinder`	scalar	m^2	area of an interior cell
`run_decay`			
`P_decay`	(N_f, n) complex	Pa	pressure
`Z_decay`	scalar	m	depth
`Distance_decay`	(2)	m	first and last distance
`Distances_decay`	(n)	m	distances
`Obs_decay`	$(n,3)$	m	observers
`Logspace_decay`	flag	–	logarithmic spacing
`run_line`			
`P_line`	(N_f, n) complex	Pa	pressure
`P1_line`, `P2_line`	(3)	m	end points
`Obs_line`	$(n,3)$	m	observers
`Distance_line`	scalar	m	length of the line
`Distances_line`	(n)	m	distances from `P1_line`
`Logspace_line`	flag	–	logarithmic spacing
`run_sliceXY`			

Dataset	Fortran shape	Unit	Meaning
`P_slicexy`	(N_f, n_x, n_y) complex	Pa	pressure
`Obs_slicexy`	$(n_x, n_y, 3)$	m	observers
`X_slicexy`, `Y_slicexy`	(n_x), (n_y)	m	grid lines
`Z_slicexy`	scalar	m	depth
`Center_slicexy`	(2)	m	centre
`Nx_slicexy`, `Ny_slicexy`	int	–	grid size
`run_sliceXZ`			
`P_slicexz`	(N_f, n_x, n_z) complex	Pa	pressure
`Obs_slicexz`	$(n_x, n_z, 3)$	m	observers
`X_slicexz`, `Z_slicexz`	(n_x), (n_z)	m	grid lines
`Y_slicexz`	scalar	m	position of the plane
`Nx_slicexz`, `Nz_slicexz`	int	–	grid size
`run_sliceVertical`			
`P_sliceVertical`	(N_f, n_u, n_z) complex	Pa	pressure
`Coords_sliceV`	$(n_u, n_z, 3)$	m	observers
`U_sliceV`, `Z_sliceV`	(n_u), (n_z)	m	span coordinate and depths
`Azimuth_sliceV`	scalar	deg	azimuth of the normal
`Center_sliceV`	(2)	m	centre
`run_sphere` (image method)			
`P_sphere`	(N_f, n_θ, n_z) complex	Pa	free-field pressure
`Obs_sphere`	$(n_\theta, n_z, 3)$	m	observers
`Theta_sphere`	(n_θ)	deg	azimuths of the cell centres
`Z_sphere`	(n_z)	m	heights of the cell centres
`R_sphere`	scalar	m	radius
`Center_sphere`	(3)	m	centre
`dA_sphere`	scalar	m^2	area of a cell

The SAITEC type writes its own geometry datasets next to `Case_type` and `Structure_nodes`, with the same suffix rule in a farm.

References

[1] C. Bak, F. Zahle, R. Bitsche, T. Kim, A. Yde, L. C. Henriksen, M. H. Hansen, J. P. A. A. Blasques, M. Gaunaa and A. Natarajan. *The DTU 10-MW Reference Wind Turbine*. DTU Wind Energy Report I-0092, Technical University of Denmark, 2013.

[2] M. Frigo and S. G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231, 2005.

[3] F. J. Harris. On the use of windows for harmonic analysis with the discrete Fourier transform. *Proceedings of the IEEE*, 66(1):51–83, 1978.

[4] The HDF Group. *Hierarchical Data Format, version 5*. `https://www.hdfgroup.org/solutions/hdf5/`.

[5] F. B. Jensen, W. A. Kuperman, M. B. Porter and H. Schmidt. *Computational Ocean Acoustics*. Springer, New York, 2nd edition, 2011.

[6] National Renewable Energy Laboratory. *OpenFAST*. `https://github.com/OpenFAST/openfast`.

[7] W. H. Thorp. Analytic description of the low-frequency attenuation coefficient. *The Journal of the Acoustical Society of America*, 42(1):270, 1967.

Index of the input variables

&AnalyticalNormalModes_config (group), 20
attenuation_bottom, 4, 28, 31, 82
    ImagesMethod_config, 19
attenuation_surface, 28, 82
    ImagesMethod_config, 19
AxisPos, 25, 26, 32
    turbine_config, 11
    turbine, 12
azimuth_sv, 9, 35
    sliceVertical_config, 18

BariPos, 26, 32
    turbine_config, 11
    turbine, 12
Binary, 51
    turbine_config, 11
    turbine, 12
block_size, 6, 69, 70, 77, 84
    solver_config, 9

case_type, 24, 51, 74
    turbine_config, 10
    turbine, 12
center_cyl
    cylinder_config, 17
center_polar
    polar_config, 17
center_sphere
    sphere_config, 19
center_sv
    sliceVertical_config, 18
center_xy, 33
    sliceXY_config, 18
&cylinder_config (group), 17

debug, 6, 10, 69
    solver_config, 9
&decay_config (group), 17
density_base, 83
    environment_config, 10
depth_base, x, 2, 11, 35, 51, 73, 82, 87
    environment_config, 10
distance_decay
    decay_config, 17
do_all
    do_config, 16
&do_config (group), 16, 32
do_cylinder
    do_config, 16
do_decay
    do_config, 16
do_line
    do_config, 16
do_polar
    do_config, 16
do_sliceVertical
    do_config, 16
do_sliceXY
    do_config, 16
do_sliceXZ
    do_config, 16
do_spectrums
    do_config, 16
do_sphere
    do_config, 16

&environment_config (group), 10
epsilon, 29, 30, 40, 73, 81, 83
    AnalyticalNormalModes_config, 20
    ImagesMethod_config, 19
extra_label, 9, 37
    solver_config, 9

f_farfield_sphere, 35, 81
    sphere_config, 19
&farm_config (group), 11, 12
farm_name, 9, 27
    farm_config, 12
filter_freqs, 25
    farm_config, 12
    turbine_config, 11
fmax, 13, 16, 27, 87, 88
    frequency_source_config, 13
    time_source_config, 15

fmin, 13, 15, 16, 27, 87, 88
    frequency_source_config, 13
    time_source_config, 15
force_expr, 15
    time_source_config, 15
force_file, 27
    time_source_config, 15
force_im, 13
    frequency_source_config, 13
force_im_expr, 13, 15
    frequency_source_config, 14
force_im_file, 13, 14
    frequency_source_config, 14
force_re, 13
    frequency_source_config, 13
force_re_expr, 13, 15
    frequency_source_config, 14
force_re_file, 13, 14
    frequency_source_config, 14
freq_file, 6, 13, 14
    frequency_source_config, 13
freq_spacing, 13, 87
    frequency_source_config, 13
freqs, 13
    frequency_source_config, 13
freqs_over, 26
    farm_config, 12
    turbine_config, 11
&frequency_source_config (group), 12, 13
From, 24
    farm_config, 12
    turbine_config, 11

&ImagesMethod_config (group), 19
in_cluster, 70
    solver_config, 9

&line_config (group), 17
logspace_decay
    decay_config, 17
logspace_line
    line_config, 18
LowerBC, 28, 40, 72, 82
    ImagesMethod_config, 19

margin_sv, 18, 35
    sliceVertical_config, 19
mask_distance, 10, 29, 32, 40, 51, 69, 73, 85, 87
    solver_config, 10
method, 8, 28, 50, 75, 87
    solver_config, 9
n_cyl
    cylinder_config, 17
n_decay
    decay_config, 17
N_images, 28, 82
    ImagesMethod_config, 19
n_line
    line_config, 17
n_observers
    spectrums_config, 16
n_polar, 87
    polar_config, 17
n_theta_sphere
    sphere_config, 19
Nfreqs, 13, 87
    frequency_source_config, 13
Nmembers, 24
    turbine_config, 11
    turbine, 12
Nmodes, 30, 82, 87
    AnalyticalNormalModes_config, 20
Nnodes, 24
    turbine_config, 11
    turbine, 12
Nsources, 13, 88
    frequency_source_config, 13
    time_source_config, 14
Nt, 15
    time_source_config, 14
nu_sv
    sliceVertical_config, 19
nx_xy
    sliceXY_config, 18
nx_xz
    sliceXZ_config, 18
ny_xy
    sliceXY_config, 18
nz_cyl
    cylinder_config, 17
nz_sphere
    sphere_config, 19
nz_sv
    sliceVertical_config, 19
nz_xz
    sliceXZ_config, 18

observers_x, 33
    spectrums_config, 16
observers_y, 33
    spectrums_config, 17
observers_z, 33

spectrums_config, 17
OP_output_dir, 51
turbine_config, 10
turbine, 12
OP_output_filename, 51
turbine_config, 10
turbine, 12
OP_SubDyn_filename
turbine_config, 11
turbine, 12

p1_line, 17
line_config, 17
p2_line, 17
line_config, 17
&polar_config (group), 17

r_cyl
cylinder_config, 17
r_polar
polar_config, 17
r_sphere
sphere_config, 19
reference_pressure, 3, 82
environment_config, 10
rootname, 12, 26
turbine_config, 10
turbine, 12

save_path, 6, 37
solver_config, 9
simulation_type, 6, 8, 20, 50, 75, 77, 87
solver_config, 9
skip, 24
farm_config, 12
turbine_config, 11
skipf, 16, 25, 27
farm_config, 12
time_source_config, 15
turbine_config, 11
&sliceVertical_config (group), 18
&sliceXY_config (group), 18
&sliceXZ_config (group), 18
&solver_config (group), 9
source_file, 6, 13, 14, 80, 88
frequency_source_config, 13
time_source_config, 14
source_xyz, 13
frequency_source_config, 13
time_source_config, 14
&spectrums_config (group), 16
speed_sound_base, 3, 82
environment_config, 10
&sphere_config (group), 19

t_end, 15
time_source_config, 14
t_start, 14, 15
time_source_config, 14
time_file, 14
time_source_config, 15
&time_source_config (group), 14
&turbine (group), 11, 12
&turbine_config (group), 10

update_existing, 6, 40, 52, 71, 73, 77, 79, 81
solver_config, 9
UpperBC, 28, 40, 72, 82
ImagesMethod_config, 19
Upto, 24
farm_config, 12
turbine_config, 11

width_sv
sliceVertical_config, 18
WindDir, 25, 26, 32
turbine_config, 11
turbine, 12
WindSpeed, 25
turbine_config, 11
turbine, 12

xlim_xy
sliceXY_config, 18
xlim_xz
sliceXZ_config, 18

y_xz
sliceXZ_config, 18
ylim_xy
sliceXY_config, 18

z_decay
decay_config, 17
z_polar
polar_config, 17
z_xy
sliceXY_config, 18
zlim_sv
sliceVertical_config, 18
zlim_xz, 81
sliceXZ_config, 18